

МІНІСТЕРСТВО ОСВІТИ І НАУКИ УКРАЇНИ
НАЦІОНАЛЬНИЙ ТЕХНІЧНИЙ УНІВЕРСИТЕТ УКРАЇНИ
«КИЇВСЬКИЙ ПОЛІТЕХНІЧНИЙ ІНСТИТУТ ІМЕНІ ІГОРЯ СІКОРСЬКОГО»

КРОСПЛАТФОРМНА РОЗРОБКА МОБІЛЬНИХ ЗАСТОСУНКІВ

Навчальний посібник

Рекомендовано Методичною радою КПІ ім. Ігоря Сікорського
як навчальний посібник для здобувачів ступеня бакалавра
за спеціальністю 121 Інженерія програмного забезпечення

Укладачі: О.Л. Недашківський, І.І. Гусєва

Електронне мережеве навчальне видання

Київ
КПІ ім. Ігоря Сікорського
2023

УДК 004.43
Н42

Укладачі: *Недашківський Олексій Леонідович*, д-р техн. наук, доц.
Гусєва Ірина Ігорівна, канд. екон. наук, доц.

Рецензенти *Кравченко Ю. В.*, д-р техн. наук, проф., завідувач кафедри мережевих технологій факультету інформаційних технологій Київського національного університету імені Тараса Шевченка
Артемчук В. О., д-р техн. наук, с.н.с., професор кафедри комп'ютеризованих систем управління Національного авіаційного університету, заступник директора з науково-організаційної роботи Інституту проблем моделювання в енергетиці ім.Г.Є.Пухова НАН України

Відповідальний редактор *Коваль О. В.*, д-р техн. наук, доцент

*Гриф надано Методичною радою КПІ ім. Ігоря Сікорського
(протокол № 4 від 01.02.2024 р.)
за поданням вченої ради Навчально-наукового інституту атомної та теплової енергетики
(протокол № 14 від 29.06.2023 р.)*

Н42 Кросплатформна розробка мобільних застосунків. [Електронний ресурс] : навч. посіб. для здобувачів ступеня бакалавра за освіт. програмою «Інженерія програмного забезпечення інтелектуальних кібер-фізичних систем в енергетиці» спец. 121 Інженерія програмного забезпечення / КПІ ім. Ігоря Сікорського ; уклад.: О. Л. Недашківський, І. І. Гусєва. – Електрон. текст. дані (1 файл). – Київ : КПІ ім. Ігоря Сікорського, 2023. – 221 с.

Навчальний посібник призначений для здобувачів ступеня бакалавр за спеціальністю 121 Інженерія програмного забезпечення освітньої програми «Інженерія програмного забезпечення інтелектуальних кібер-фізичних систем в енергетиці» Національного технічного університету України «Київського політехнічного інституту імені Ігоря Сікорського» при вивченні дисципліни «Кросплатформна розробка мобільних застосунків». У посібнику приділено увагу вивченню кросплатформної мови програмування мобільних застосунків DART. Навчальний посібник буде також корисний для тих, хто цікавиться розробкою програмного забезпечення мовою програмування DART за іншими спеціальностями галузі знань 12 Інформаційні технології.

УДК 004.43

Реєстр. № НП 23/24-019. Обсяг 6,2 авт. арк.

Національний технічний університет України
«Київський політехнічний інститут імені Ігоря Сікорського»
проспект Берестейський, 37, м. Київ, 03056
<https://kpi.ua>

Свідectво про внесення до Державного реєстру видавців, виготовлювачів
і розповсюджувачів видавничої продукції ДК № 5354 від 25.05.2017 р.

© КПІ ім. Ігоря Сікорського, 2023

ЗМІСТ

ВСТУП.....	7
1. ПОРІВНЯННЯ DART ТА JAVA ДЛЯ КРОСПЛАТФОРМНОЇ МОБІЛЬНОЇ РОЗРОБКИ.....	8
1.1. Коротка довідка про Dart	8
1.2. Створення простого класу Dart.....	8
1.3. Використання необов'язкових параметрів замість перевантаження (overloading)	15
1.4. Створення фабрик	17
1.5. Використання інтерфейсів	20
1.6. Використання Dart для функціонального програмування.....	21
Резюме	23
Контрольні запитання і завдання	24
2. НАЛАШТУВАННЯ ЗАСОБІВ РОЗРОБКИ МОВОЮ DART ТА ПОЧАТОК РОБОТИ	26
2.1. Мова Dart. Перша програма.....	26
Резюме	30
Контрольні запитання і завдання	31
3. ОСНОВИ МОВИ DART	32
3.1. Структура програми.....	32
3.2. Змінні і константи	33
3.3. Примітивні типи даних.....	37
3.4. Значення null, nullable-типи і null-безпека	39
3.5. Арифметичні операції.....	43
3.6. Побітові операції і операції зсуву	46
3.7. Умовні вирази.....	48
3.8. Операції присвоєння і пріоритет операцій	51
3.9. Умовні конструкції	52
3.10. Цикли.....	57
Резюме	59
Контрольні запитання і завдання	60

4. ФУНКЦІОНАЛЬНЕ ПРОГРАМУВАННЯ	67
4.1. Визначення функцій	67
4.2. Параметри функції	68
4.3. Повернення результату з функції	73
4.4. Функція як об'єкт	75
4.5. Анонімні і вкладені функції	77
4.6. Замикання	79
Резюме	82
Контрольні запитання і завдання	83
5. ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ	84
5.1. Класи і об'єкти	84
5.2. Константи і константні конструктори класів	97
5.3. Успадковування	99
5.4. Абстрактні класи та методи	103
5.5. Реалізація інтерфейсів	105
5.6. Міксини	108
5.7. Статичні члени класів	110
5.8. Generics	114
5.9. Фабричний конструктор	119
5.10. Перевизначення операторів	122
5.11. Перерахування	125
Резюме	128
Контрольні запитання і завдання	128
6. ЗАСТОСУВАННЯ КОЛЕКЦІЙ	132
6.1. List	132
6.2. Set	137
6.3. Map	140
Резюме	144
Контрольні запитання і завдання	144
7. ОБРОБКА ВИНЯТКІВ	146

7.1. Блок try..catch.....	146
7.2. Класи винятків.....	150
7.3. Оператор throw і створення своїх типів винятків.....	153
Резюме.....	156
Контрольні запитання і завдання	156
8. АСИНХРОННІСТЬ	157
8.1. Вступ до асинхронності і Future	157
8.2. Робота з класом Future	159
8.3. Асинхронні функції і оператори async і await.....	168
Резюме	172
Контрольні запитання і завдання	172
9. ПРИКЛАДИ ТА ВПРАВИ	174
9.1. Інтерполяція рядків.....	174
9.2. Nullable змінні	175
9.3. Null-aware оператори	176
9.4. Умовний доступ до властивостей.....	176
9.5. Колекції	177
9.6. Синтаксис стрілки (=>).....	179
9.7. Каскади.....	180
9.8. Гетери та сетери	181
9.9. Необов'язкові позиційні параметри	182
9.10. Необов'язкові іменовані параметри	184
9.11. Винятки	185
9.12. Використання this в конструкторах.....	188
9.13. Ініціалізація списків (list)	189
9.14. Іменовані конструктори.....	190
9.15. Фабричні конструктори	191
9.16. Перенаправлення конструкторів	192
9.17. Константні конструктори	194
Резюме	194

Контрольні запитання і завдання	195
ПЕРЕЛІК ПОСИЛАНЬ	197
СПИСОК РЕКОМЕНДОВАНОЇ ЛІТЕРАТУРИ І ЕЛЕКТРОННИХ РЕСУРСІВ	200
ВІДПОВІДІ НА КОНТРОЛЬНІ ЗАВДАННЯ.....	201
ПРЕДМЕТНИЙ ПОКАЖЧИК ТЕРМІНІВ ТА СИНТАКСИЧНИХ КОНСТРУКЦІЙ.....	211
ДОДАТОК А. ПРИКЛАДИ КРОСПЛАТФОРМНОЇ РОЗРОБКИ МОБІЛЬНИХ ЗАСТОСУНКІВ	213

ВСТУП

Навчальний посібник «Кросплатформна розробка мобільних застосунків» присвячено мові програмування загального призначення Dart від компанії Google, яка призначена насамперед для кросплатформної розробки мобільних і вебзастосунків.

Обсяг матеріалу становить необхідний мінімум в підготовці слухачів за освітньою програмою «Інженерія програмного забезпечення кіберфізичних систем в енергетиці» спеціальності 121 Інженерія програмного забезпечення при вивченні дисципліни: «Кросплатформна розробка мобільних застосунків», а також може використовуватись у дисциплінах «Мульти- та кросплатформне програмне забезпечення» і «Кросплатформне програмування» та інших, де використання мови програмування Dart є корисним.

В навчальному посібнику на основі переробленої, систематизованої і впорядкованої інформації з різних джерел подано загальний та практичний матеріал, опис програмних засобів, наведені рекомендації для поглибленого вивчення як самої мови програмування так і інструментальних засобів розробки й тестування. В додатку наведені приклади створення мобільних застосунків мовою Dart.

Основним завдання навчального посібника, згідно з вимогами програм зазначених навчальних дисциплін, є формування у студентів після засвоєння кредитного модуля таких знань та умінь.

Знання:

- основних принципів застосування мови Dart;
- структури Dart програми;
- типів даних, змінних, констант та керуючих конструкцій мови;
- визначення, створення та використання функцій;
- базових класів, методів та інтерфейсів;
- засобів обробки потоків та файлів;

Уміння:

- проектувати компоненти кросплатформних мобільних застосунків мовою Dart;
- застосовувати засоби мови програмування, опису інформаційних ресурсів, специфікацій під час проектування та створення інформаційних систем;
- програмно реалізувати алгоритми розв'язування задач;
- проектувати компоненти програмного забезпечення;
- виконувати аналіз коректності програм.

1. ПОРІВНЯННЯ DART ТА JAVA ДЛЯ КРОСПЛАТФОРМНОЇ МОБІЛЬНОЇ РОЗРОБКИ

1.1. Коротка довідка про Dart

Dart - це мова програмування для Flutter, набору інструментів інтерфейсу Google для створення нативно скомпільованих мобільних, вебзастосунків та настільних програм з єдиною кодовою базою.

В цьому розділі розглядається Dart з акцентом на функціях, які розробники на мові Java можуть не очікувати. Буде розглянуто, як просто та швидко створювати функції, сценарії та програми у Dart. А саме, буде розглянуто:

- як створювати конструктори;
- різні способи задавання параметрів;
- коли і як створювати гетери та сетери;
- як Dart обробляє межі видимості;
- як створювати фабрики;
- як функціональне програмування працює в Dart;
- та інші основні концепції Dart.

Для вивчення матеріалу буде потрібен браузер. Писати та запускати всі приклади можна за допомогою DartPad¹⁾ [1], інтерактивного браузерного інструмента, який дозволяє оперувати з функціями мови Dart та основними бібліотеками. Якщо ж віддається перевага інтегрованим середовищам розробки, то можна замість DartPad використовувати IDE, наприклад WebStorm, IntelliJ з плагіном Dart або Visual Studio Code з розширенням Dart Code extension²⁾ [2].

1.2. Створення простого класу Dart

Почнемо з побудови простого класу Dart з тією ж функціональністю, що і клас **Bicycle**³⁾ [3] з підручника Java. Клас **Bicycle** містить деякі приватні змінні екземпляра з гетерами та сетерами. Метод **main()** створює екземпляр **Bicycle** і виводить його на консоль.

¹⁾ <https://dart.dev/tools/dartpad>

²⁾ <https://marketplace.visualstudio.com/items?itemName=Dart-Code.dart-code>

³⁾ <https://docs.oracle.com/javase/tutorial/java/javaOO/variables.html>

1.2.1. Запуск DartPad

DartPad - це онлайн кодова лабораторія, яка дозволяє писати та запускати код на мові Dart для виконання будь-якого набору вправ. Посилання [**https://dart.dev/#try-dart**](https://dart.dev/#try-dart)¹⁾ [4] відкриває новий екземпляр, який містить приклад «Привіт, Світ» за замовчуванням, зовнішній вигляд якого наведено на рисунку 1.1.

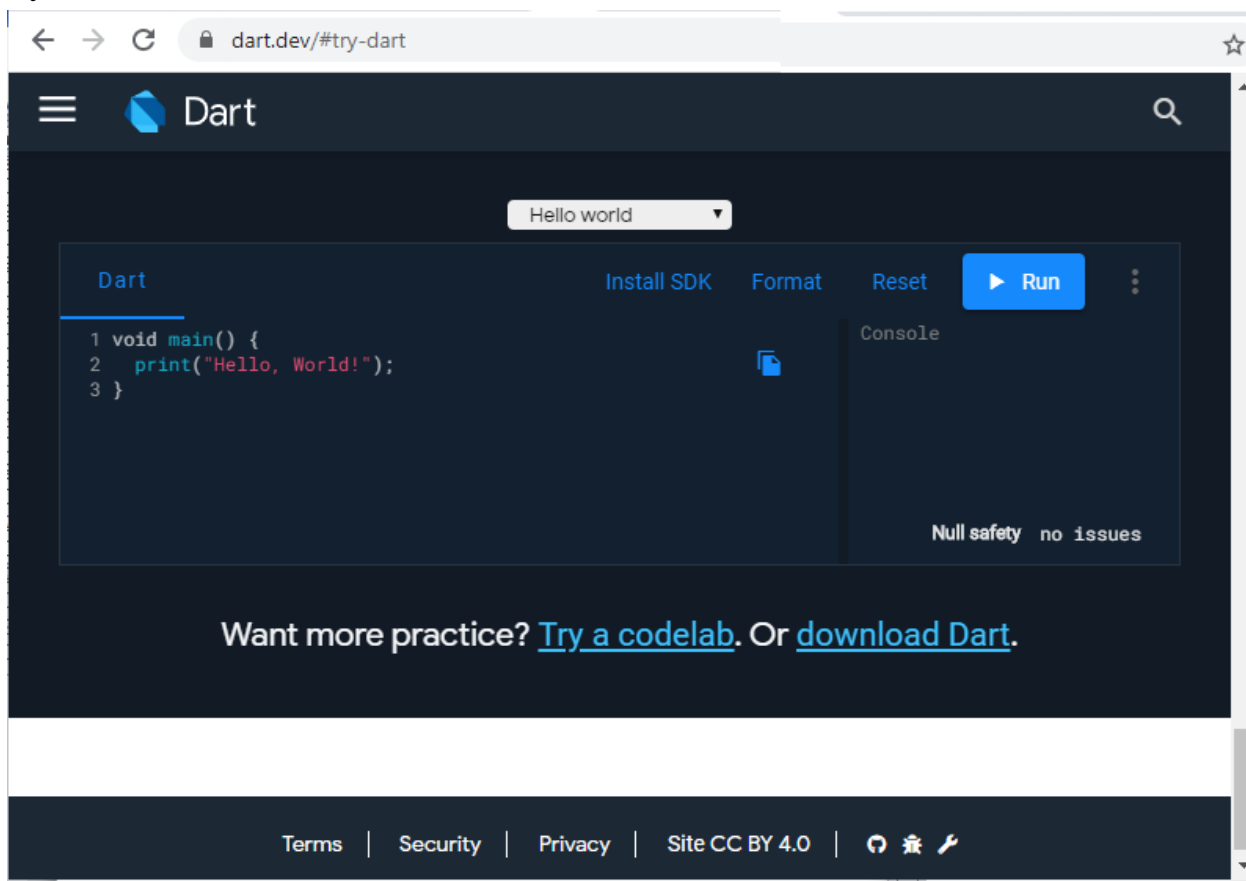


Рисунок 1.1. Середовище розробки

Можна продовжувати використовувати один і той самий DartPad примірник у всіх прикладах та вправах, але якщо натиснути кнопку **Скинути (Reset)**, DartPad поверне до прикладу за замовчуванням, при цьому втратиться вся попередня робота.

Примітка: Для зручності вгорі кожної сторінки кодової лабораторії DartPad знаходиться екземпляр DartPad, який відображає стан на початку кожної вправи.

Тож, спочатку відкриваємо DartPad за посиланням: <https://dartpad.dev/0ca5b334ec413c084575f575e0240501>.

¹⁾ <https://dart.dev/#try-dart>

1.2.2. Визначення класу

Над функцією **main()** додаємо клас **Bicycle** із трьома змінними екземпляра. Також видалимо вміст з **main()**, як показано в наступному фрагменті коду:

```
1  class Bicycle {  
2      int cadence;  
3      int speed;  
4      int gear;  
5  }  
6  
7  void main() {  
8  }
```

Основний метод у Dart називається **main()**. Якщо потрібен доступ до аргументів командного рядка, можна додати їх, наприклад так: **main (List <String> args)**.

Метод **main()** розташовується на вищому рівні. У Dart можна визначити код поза класами. Змінні, функції, гетери та сетери можуть існувати поза класами.

Оригінальний приклад Java оголошує змінні приватного екземпляра за допомогою приватного тегу **private**, який Dart не використовує. Детальніше про межі видимості розглядається у розділі «Додавання змінних лише для читання».

Ні **main()**, ні **Bicycle** не оголошуються загальнодоступними **public**, оскільки у Dart всі ідентифікатори є загальнодоступними за замовчуванням. Dart не має ключових слів для загальнодоступних (**public**), приватних (**private**) чи захищених (**protected**) ідентифікаторів.

У цьому прикладі аналізатор Dart видає помилку, повідомляючи про те, що змінні повинні бути ініціалізовані, оскільки вони не мають мати нульового значення (**non-nullable**). Виправлення та обробка цієї ситуації розглянуті в наступному розділі.

Dart використовує двозначний відступ за домовленістю замість чотирьох. Окрім цього, не потрібно турбуватися про правила відступу у Dart, завдяки зручному інструменту, який називається **dartfmt**¹⁾ [5]. Як сказано в конвенціях коду Dart (**Effective Dart**²⁾ [6]), «Офіційні правила обробки пробілів для Dart - це те, що здійснює **dartfmt**».

¹⁾ <https://dart.dev/tools/dartfmt>

²⁾ <https://dart.dev/guides/language/effective-dart>

1.2.3. Визначення конструктора

Додаймо наступний конструктор до класу **Bicycle**:

```
1 Bicycle(this.cadence, this.speed, this.gear);
```

Цей конструктор не має тіла, що дозволено в Dart.

В разі якщо не написати крапку з комою (;) у кінці конструктора без тіла (**no-body constructor**), DartPad відображає таку помилку: «**A function body must be provided** - Необхідно надати тіло функції.»

Використання **this** у списку параметрів конструктора - це зручний вказівник для призначення значень змінним екземпляра.

Наведений вище код еквівалентний наступному:

```
1 Bicycle(int cadence, int speed, int gear) {  
2   this.cadence = cadence;  
3   this.speed = speed;  
4   this.gear = gear;  
5 }
```

1.2.4. Форматування коду

Переформатувати код у Dart можна у будь-який час, натиснувши кнопку **Format** у верхній частині інтерфейсу DartPad. Переформатування особливо корисно, коли вставляємо скопійований код у DartPad і вирівнювання вимкнено.

1.2.5. Створення та виведення на консоль екземпляра класу

Додаймо наступний код до функції **main()**:

```
1 void main() {  
2   var bike = new Bicycle(2, 0, 1);  
3   print(bike);  
4 }
```

Видалимо необов'язкове ключове слово **new**:

```
1 var bike = Bicycle(2, 0, 1);
```

Ключове слово **new** стало необов'язковим у Dart 2.

Якщо знаємо, що значення змінної не буде змінюватись, тоді можемо використовувати ключове слово **final** замість **var**.

1.2.6. Запуск програми прикладу

Запустимо код прикладу на виконання, натиснувши кнопку **Run** у верхній частині вікна DartPad. Якщо кнопка **Run** не активна, необхідно подивитись на розділ **Problems** далі на цій сторінці.

Ми повинні побачити результат, який наведено на рисунку 1.2.

Instance of 'Bicycle'

Рисунок 1.2. Результат роботи програми

Помилки та попереджень не повинно з'являтися, що вказує на те, що інтерфейс типу працює, і що аналізатор припускає, що твердження, яке починається з **var bike =**, визначає екземпляр **Bicycle**.

1.2.7. Покращення результату

Незважаючи на те, що результат «Екземпляр велосипеда» (Instance of 'Bicycle') правильний, він не дуже інформативний. Усі класи у Dart мають метод **toString()**, який можемо замінити, щоб забезпечити більш корисний результат.

Додаймо такий метод як **toString()** де завгодно в класі **Bicycle**:

```
1 @override
2 String toString() => "Bicycle: $speed mph";
```

Анотація **@override** повідомляє аналізатор, що необхідно навмисно перевизначити елемент. Аналізатор поверне помилку, якщо не зможе виконати заміну належним чином.

Dart підтримує одинарні або подвійні лапки при визначення рядків.

Можна використовувати інтерполяцію рядків, щоб розташувати значення виразу всередині рядкового літералу: **\${expression}**. Якщо вираз є ідентифікатором, то можемо пропустити дужки: **\$variableName**.

Також можемо скоротити однорядкові функції або методи, використовуючи позначення жирної стрілки (**=>**).

1.2.8. Запуск коду прикладу

Натискаємо на кнопку **Run**.

Результат має бути таким, який наведено на рисунку 1.3.

Bicycle: 0 mph

Рисунок 1.3. Результат роботи програми

В разі помилки, можна перевірити свій код за посиланням: https://dartpad.dev/f176b4c31b4be8a454107d0946fe2fa3?null_safety=true.

1.2.9. Додавання змінних лише для читання

Оригінальний приклад Java визначає **speed** як змінну, призначену лише для читання - вона оголошена приватною та реалізовано лише гетер. Далі надаймо той самий функціонал програмі у Dart.

Відкриємо **bicycle.dart**¹⁾ [7] у DartPad (або продовжимо використовувати попередню створену власну копію).

Щоб позначити ідентифікатор Dart як приватний для своєї бібліотеки, почнемо його ім'я з підкреслення (_). Можемо оголосити змінну **speed** лише для читання, змінивши її назву та додавши гетер.

Позначимо **speed** приватною змінною екземпляра, тобто лише для читання.

Для цього у конструкторі **Bicycle** видалимо параметр **speed**:

```
1 Bicycle(this.cadence, this.gear);
```

У **main()** видалимо другий параметр (**speed**) із виклику конструктора **Bicycle**:

```
1 var bike = Bicycle(2, 1);
```

Змінімо решту викликів поля **speed** на **_speed** у двох місцях.

Порада. Якщо використовується інтегроване середовище розробки, наприклад, JetBrains IDE, можна одночасно перейменувати всі екземпляри змінної, натиснувши правою кнопкою миші на імені та вибравши **Refactor>Rename** у контекстному меню.

Ініціалізуємо **_speed** значенням 0:

```
1 int _speed = 0;
```

Додаймо наступний гетер до класу **Bicycle**:

```
1 int get speed => _speed;
```

Кожна змінна (навіть якщо це число) повинна бути або ініціалізована, або оголошена **nullable** шляхом додавання **?** під час декларації її типу.

Компілятор Dart забезпечує дотримання конфіденційності бібліотеки для будь-якого ідентифікатора, який має префікс підкреслення. Конфіденційність бібліотек (**Library privacy**) загалом означає, що ідентифікатор видно лише всередині файлу (а не лише класу), в якому визначений ідентифікатор.

¹⁾ <https://dartpad.dev/f176b4c31b4be8a454107d0946fe2fa3>

За замовчуванням Dart створює неявні гетери та сетери для всіх відкритих (**public**) змінних екземпляра. Тобто не потрібно визначати власні гетери чи сетери, якщо не хочемо застосувати змінні лише для читання або лише для запису, обчислювати або перевіряти значення або оновлювати значення деінде.

Оригінальний зразок Java містить гетери та сетери для **cadence** та **gear**. Приклад на мові Dart не потребує явних гетерів і сетерів для них, тому він просто використовує змінні екземпляра.

Можна почати з простого поля, наприклад **bike.cadence**, а пізніше рефакторингувати його, щоб використовувати гетери та сетери. API залишається незмінним. Іншими словами, перехід від поля до гетера та сетера не є суттєвою зміною в мові Dart.

Завершимо реалізацію швидкості **speed** як змінну екземпляра лише для читання.

Додаймо наступні методи до класу Велосипед:

```
1 void applyBrake(int decrement) {
2     _speed -= decrement;
3 }
4
5 void speedUp(int increment) {
6     _speed += increment;
7 }
```

Остаточний приклад Dart схожий на оригінальний Java¹⁾ [8], але є більш компактним на 23 рядки замість 40:

```
1 class Bicycle {
2     int cadence;
3     int _speed = 0;
4     int get speed => _speed;
5     int gear;
6
7     Bicycle(this.cadence, this.gear);
8 }
```

¹⁾ <https://docs.oracle.com/javase/tutorial/java/javaOO/variables.html>

```

9   void applyBrake(int decrement) {
10     _speed -= decrement;
11   }
12
13   void speedUp(int increment) {
14     _speed += increment;
15   }
16
17   @override
18   String toString() => "Bicycle: $_speed mph";
19 }
20
21 void main() {
22   var bike = Bicycle(2, 1);
23   print(bike);
24 }

```

В разі помилки, можна перевірити свій код за посиланням: <https://dartpad.dev/a81cf6bde4d52bb8b774d87aff842b56>.

1.3. Використання необов'язкових параметрів замість перевантаження (overloading)

Наступний приклад визначає клас **Rectangle class**¹⁾ [9], ще один приклад із підручника Java.

У коді Java показано перевантаження конструкторів (**overloading constructors**), що є звичайною практикою на Java, коли конструктори мають однакові назви, але відрізняються кількістю або типом параметрів. Dart не підтримує перевантаження конструкторів і по іншому обробляє цю ситуацію, як буде показано в наступному розділі.

1.3.1. Відкриття прикладу класу Rectangle у DartPad.

Відкриємо приклад класу Rectangle²⁾ [10] у DartPad за посиланням <https://dartpad.dev/68fb4923eee0e9e3d3af42730c1d3503>.

¹⁾ <https://docs.oracle.com/javase/tutorial/java/javaOO/objectcreation.html>

²⁾ <https://dartpad.dev/68fb4923eee0e9e3d3af42730c1d3503>

1.3.2. Додавання конструктора **Rectangle**

Додаймо один порожній конструктор, який замінює всі чотири конструктори в прикладі Java:

```
1 Rectangle({this.origin = const Point(0, 0),  
  this.width = 0, this.height = 0});
```

Цей конструктор використовує необов'язкові іменовані параметри.

Конструкції **this.origin**, **this.width** та **this.height** використовують скорочений трюк для присвоєння змінних екземпляра всередині оголошення конструктора.

Конструкції **this.origin**, **this.width** та **this.height** є необов'язковими іменованими параметрами. Іменовані параметри розташовані у фігурних дужках (**{}**).

Синтаксис **this.origin = const Point (0, 0)** визначає значення **Point (0,0)** за замовчуванням для змінної первинного (**origin**) екземпляра. Значення за замовчуванням має бути константою часу компіляції. Цей конструктор надає значення за замовчуванням для всіх трьох змінних екземпляра.

Покращимо код. Додаймо функцію **toString ()** до класу **Rectangle**:

```
1 @override  
2 String toString() => "Origin: (${origin.x},  
  ${origin.y}), width: $width, height: $height";
```

Використаємо конструктор. Для цього замінимо **main()** таким кодом, щоб переконатися, що можна створити екземпляр **Rectangle**, використовуючи лише ті параметри, які нам потрібні:

```
1 main() {  
2   print(Rectangle(origin: const Point(10, 20), width:  
  100, height: 200));  
3   print(Rectangle(origin: const Point(10, 10)));  
4   print(Rectangle(width: 200));  
5   print(Rectangle());  
6 }
```

Конструктор у Dart для **Rectangle** - це один рядок коду, порівняно з 16 рядками коду для еквівалентних конструкторів у версії Java.

Запустимо код програми прикладу на виконання. Результат має бути таким, який наведено на рисунку 1.4.


```
Origin: (10, 20), width: 100, height: 200
Origin: (10, 10), width: 0, height: 0
Origin: (0, 0), width: 200, height: 0
Origin: (0, 0), width: 0, height: 0
```

Рисунок 1.4. Результат роботи програми

В разі помилки, можна перевірити свій код за посиланням: <https://dartpad.dev/bdc6a0c88482c4492912cc2849aa5121>.

1.4. Створення фабрик

Фабрики, загальноживаний шаблон дизайну в Java, мають кілька переваг перед безпосереднім створенням об'єкта, наприклад, приховування деталей створення екземпляра, надання можливості повернути підтип фабричного типу та необов'язково повернення існуючого, а не нового об'єкта.

Розглянемо два способи реалізації фабрики створення фігури:

- ✓ варіант 1: створення функції верхнього рівня;
- ✓ варіант 2: створення фабричного конструктора.

Для цієї вправи будемо використовувати приклад **Shapes**, який створює екземпляри фігур та виводить на консоль їх обчислену площу:

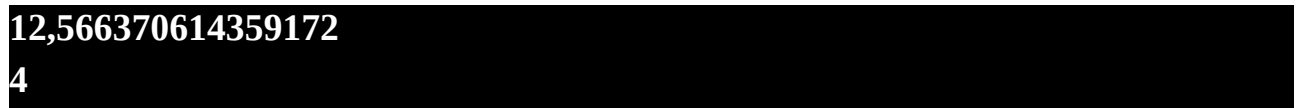
```
1   import 'dart:math';
2
3   abstract class Shape {
4     num get area;
5   }
6
7   class Circle implements Shape {
8     final num radius;
9     Circle(this.radius);
10    num get area => pi * pow(radius, 2);
11  }
12
13  class Square implements Shape {
14    final num side;
15    Square(this.side);
16    num get area => pow(side, 2);
```

```

17     }
18
19     main() {
20         final circle = Circle(2);
21         final square = Square(2);
22         print(circle.area);
23         print(square.area);
24     }

```

Відкриємо приклад Shapes¹⁾ [11] у DartPad. В результаті відпрацювання цього коду, у області консолі повинні бути отримані результати обчислення площі кола і квадрата, який наведено на рисунку 1.5.



```

12,566370614359172
4

```

Рисунок 1.5. Результат роботи програми

Таки чином бачимо, що Dart підтримує абстрактні класи. Окрім цього можна визначити кілька класів в одному файлі.

Бібліотека **dart: math** - одна з основних бібліотек Dart. Інші основні бібліотеки включають **dart: core**, **dart: async**, **dart: convert** та **dart: collection**.

За домовленістю, константи бібліотек Dart позначаються у стилі **lowerCamelCase** (наприклад, **pi** замість **PI**). Якщо цікаві міркування, якими керувались розробники мови, то можна переглянути рекомендації щодо стилю **PREFER using lowerCamelCase for constant names**²⁾ [12].

Наступний код демонструє реалізацію двох гетерів, які обчислюють значення площі кола та квадрата:

```

1  num get area => pi * pow (radius, 2); // Отримання
    площі кола та
2  num get area => pow (side, 2); // Отримання площі
    квадрата.

```

1.4.1. Варіант 1: Створення функції верхнього рівня

Реалізуємо фабрику, як функцію верхнього рівня, додавши наступну функцію на найвищому рівні (поза будь-яким класом):

¹⁾ <https://dartpad.dev/a98247f43448f5e0eac4c7f483c173a5>

²⁾ <https://dart.dev/guides/language/effective-dart/style#prefer-using-lowercamelcase-for-constant-names>

```

1 Shape shapeFactory(String type) {
2   if (type == "circle") return Circle(2);
3   if (type == "square") return Square(2);
4   throw 'Can\'t create $type. ';
5 }

```

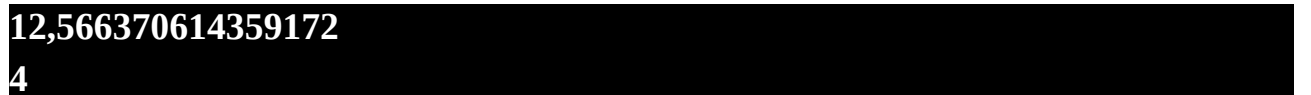
Викликаємо фабричну функцію, замінивши перші два рядки методом **main()**:

```

1 final circle = shapeFactory("circle");
2 final square = shapeFactory("square");

```

Запустимо код програми прикладу. Консоль повинна виглядати так само, як і раніше. А саме так, як наведено на рисунку 1.6.



```

12,566370614359172
4

```

Рисунок 1.6. Результат роботи програми

Якщо функція викликається з будь-яким рядком, крім «**circle**» або «**square**», вона згенерує виняток.

Dart SDK визначає класи для багатьох поширених винятків, окрім цього, можна самостійно створювати і реалізувати клас **Exception** для створення більш конкретних винятків або (як у цьому прикладі) можна створити рядок, що описує проблему, яка виникла.

Коли зустрічається виняток, **DartPad** повідомляє **Uncaught**. Щоб побачити кориснішу інформацію, необхідно обернути код у конструкцію **try-catch** та надрукувати виняток у консолі. В якості додаткової вправи можна ознайомтеся з наступним прикладом у **DartPad**¹⁾ [13].

Щоб використовувати одинарні лапки всередині рядка (**string**), необхідно використовувати екранування за допомогою косої риски ('**Can\'t create \$type.**'), або визначити рядок (**string**) за допомогою подвійних лапок ("**Can't create \$type.**").

В разі помилки, можна перевірити свій код за посиланням: <https://dartpad.dev/a6abf3864fcd73a4400b8155f624d899>.

¹⁾ <https://dartpad.dev/63e040a3fd682e191af40f6427eaf9ef>

1.4.2. Варіант 2: Створення фабричного конструктора

Застосуємо ключове слово **factory** для створення фабричного конструктора.

Додаймо фабричний конструктор до абстрактного класу **Shape**:

```
1 abstract class Shape {
2     factory Shape(String type) {
3         if (type == 'circle') return Circle(2);
4         if (type == 'square') return Square(2);
5         throw 'Can\'t create $type.';
6     }
7     num get area;
8 }
```

Замінімо перші два рядки **main()** наступним кодом для створення екземплярів фігур:

```
1 final circle = Shape('circle');
2 final square = Shape('square');
```

Та видалимо попередньо додану функцію **shapeFactory()**.

Сам код програми у фабричному конструкторі ідентичний коду, який використовується у функції **shapeFactory()**.

В разі помилки, можна перевірити свій код за посиланням: <https://dartpad.dev/8e2ad295a1b7c8254414afb43fc7d280>.

1.5. Використання інтерфейсів

Мова Dart не містить ключового слова **interface**, оскільки кожен клас визначається як інтерфейс (**every class defines an interface**).

Відкриємо приклад **Shapes**¹⁾ [14] у **DartPad** (або продовжимо використовувати попередній приклад).

Додаймо клас **CircleMock**, який реалізує інтерфейс **Circle**:

```
1 class CircleMock implements Circle {}
```

В результаті, повинна згенеруватися помилка «Відсутність конкретних реалізацій» («**Missing concrete implementations**»), оскільки **CircleMock** не успадковує реалізацію **Circle** - він використовує лише її інтерфейс. виправимо цю помилку, визначивши змінні екземпляра площі (**area**) та радіуса (**radius**):

¹⁾ <https://dartpad.dev/8e2ad295a1b7c8254414afb43fc7d280>

```

1  class CircleMock implements Circle {
2    num area = 0;
3    num radius = 0;
4  }

```

В цьому прикладі, незважаючи на те, що клас **CircleMock** не визначає поведінки, це є дозволеним кодом у Dart - аналізатор не викликає помилок.

Змінна **area** екземпляра **CircleMock** реалізує гетер **area** для **Circle**.

В разі помилки, можна перевірити свій код за посиланням: <https://dartpad.dev/75fa871ed532225710fa537d983e775e>.

1.6. Використання Dart для функціонального програмування

У функціональному програмуванні можна виконувати такі речі:

- передавати функції як аргументи;
- призначати функцію змінній;
- деконструювати функцію, яка приймає кілька аргументів у послідовність функцій, кожна з яких приймає один аргумент (так званий каррінг (**currying**));
- створювати безіменну функцію, яка може використовуватися як значення константи (так звані лямбда-вирази (**lambda expression**), які були додані до Java у випуску JDK 8¹) [15]).

Dart підтримує всі ці функції. У Dart навіть функції є об'єктами і мають тип **Function**. Це означає, що функції можуть бути призначені змінним або передані як аргументи іншим функціям. Також можна викликати екземпляр класу Dart так, ніби це функція, як у наступному прикладі:

```

1  class WannabeFunction {
2    String call(String a, String b, String c) => '$a $b
    $c!';
3  }
4
5  var wf = WannabeFunction();
6  var out = wf('Hi', 'there, ', 'gang');
7
8  main() => print(out);

```

Вигляд результату роботи програми на консолі наведено на рисунку 1.7.

Hi there, gang!

¹) <http://www.oracle.com/technetwork/java/javase/8-whats-new-2157071.html>

Рисунок 1.7. Результат роботи програми

У наступному прикладі використовується імперативний (не функціональний) код:

```
1 String scream(int length) => "A${'a' * length}h!";
2
3 main() {
4   final values = [1, 2, 3, 5, 10, 50];
5   for (var length in values) {
6     print(scream(length));
7   }
8 }
```

Відкриємо та запустимо на виконання приклад "**Scream**"¹⁾ [16] у DartPad. Результат повинен виглядати так, як наведено на рисунку 1.8.



Рисунок 1.8. Результат роботи програми

При використанні інтерполяції рядків рядок `$ {'a' * length}` отримує значення повторення символу 'a' у кількості `length` разів.

Перетворимо імперативний код у функціональний. Для цього видалимо імператив для циклу `for() {...}` у `main()` і замінимо його одним рядком коду, який використовує ланцюжок методів:

```
1 values.map(scream).forEach(print);
```

Запустимо приклад на виконання та перевіримо його роботу. Отже тут функціональний підхід реалізує друк тих самих шість «галасів», що й імперативний приклад.

В разі помилки, можна перевірити свій код за посиланням: <https://dartpad.dev/368c0634feb20f47011f435d911456c2>.

¹⁾ <https://dartpad.dev/9bcb7dba6dca46e2a9dbfb300298461e>

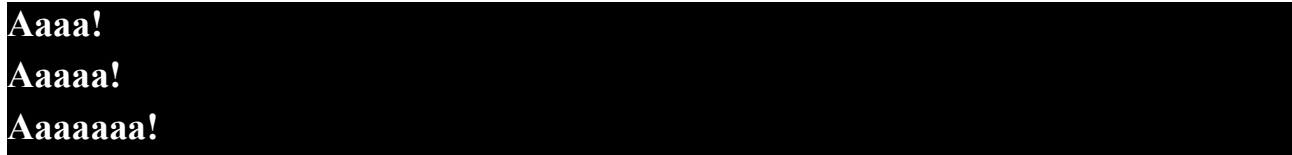
1.6.1. Використання Iterable функцій

Основні класи **List** та **Iterable** підтримують **fold ()**, **where ()**, **join ()**, **skip ()** та багато іншого. Dart також має вбудовану підтримку **maps** та **sets**.

Замінімо рядок **values.map ()** у **main ()** наступним чином:

```
1 values.skip(1).take(3).map(scream).forEach(print);
```

Запустимо код програми прикладу на виконання. Результат повинен виглядати так, як наведено на рисунку 1.9.



```
Aaaa!  
Aaaaa!  
Aaaaaaa!
```

Рисунок 1.9. Результат роботи програми

В цьому прикладі:

- **skip (1)** пропускає перше значення, 1, у літералі значень **values** списку (**list**);
- **take (3)** отримує наступні 3 значення - 2, 3 і 5 - у літералі значень **values** списку (**list**);
- решта значень пропускаються.

В разі помилки, можна перевірити свій код за посиланням: <https://dartpad.dev/bd0d627c9af8d264c2b3dde7c0939375>.

Резюме

Ознайомлення із цим розділом дозволить отримати знання про деякі відмінності між Java та Dart. Dart досить легко вивчити, і, крім того, його основні бібліотеки [17] та багатий набір доступних пакетів [18] підвищують продуктивність розробника програм. Dart добре масштабується до великих застосунків. Сотні інженерів Google використовують Dart для написання критично важливих програм, які приносять значну частину доходу Google.

Наведений матеріал в цьому розділі недостатньо глибокий, щоб показати всі відмінності між Java та Dart. Наприклад, в цьому розділі не розглядалися:

- **async / await**, що дозволяють писати асинхронний код так, ніби він синхронний. Ознайомтесь із цим можна на прикладі DartPad за посиланням [19]: <https://dartpad.dev/fae22cffa7b184b4d27cd96dd633a5af>, який візуалізує обчислення перших п'яти знаків після коми числа π ;

- каскади методів [20], про що можна дізнатися за посиланням: <http://news.dartlang.org/2012/02/method-cascades-in-dart-posted-by-gilad.html>;

- нульові (Null aware) оператори [21], про що можна дізнатися за посиланням: <http://news.dartlang.org/2015/08/dart-112-released-with-null-aware.html>.

Побачити Dart-технологію в дії можна у Flutter codelabs [22], яка доступна за посиланням: <https://flutter.dev/docs/codelabs>.

Дізнатися набагато більше про Dart можна із наступних статей, ресурсів та вебсайтів:

Статті:

1. Чому Флатер використовує Dart [23]: <https://hackernoon.com/why-flutter-uses-dart-dd635a054ebf>;

2. Анонс Dart 2: Оптимізація для розробки на стороні клієнта [24]: <https://medium.com/dartlang/announcing-dart-2-80ba01f43b6>;

3. Чому я перейшов з Java на Dart [25]: <https://hackernoon.com/why-i-moved-from-java-to-dart-8f3802b1d652>.

Ресурси:

1. Екскурсія мовою Dart [26]
2. Екскурсія по основних бібліотеках [27]
3. Ефективний Dart [28]

Вебсайти:

1. dart.dev: Мова програмування Dart [29]
2. flutter.dev: Набір інструментів інтерфейсу Flutter [30]

Контрольні запитання і завдання

1. Що таке Dart?
2. Який набір інструментів потрібен для написання та виконання програм на Dart?
3. Що таке DartPad?
4. Як в Dart визначаються класи?
5. Як в Dart визначаються конструктори?
6. Як в Dart створюються екземпляри класу?
7. Чи обов'язкове використання ключового слова `new` у Dart?
8. Як у Dart визначаються рядки? Які типи лапок дозволені?
9. Як в Dart позначаються змінні?

10. Що таке перевантаження конструкторів (**overloading constructors**)?
11. Що таке фабрики?
12. Що таке абстрактні класи?
13. Як у Dart використовуються інтерфейси? Чи містить мова Dart ключове слово `interface`?
14. Які можливості надає Dart при функціональному програмуванні?
15. Що таке Iterable функції?
16. Створіть клас власного транспортного засобу.
17. Наведіть приклад використання необов'язкових параметрів замість перевантаження конструкторів (**overloading constructors**).
18. Наведіть приклад використання фабрик у Dart.
19. Наведіть приклад використання поняття інтерфейс у Dart.
20. Наведіть приклад застосування Iterable функцій.

2. НАЛАШТУВАННЯ ЗАСОБІВ РОЗРОБКИ МОВОЮ DART ТА ПОЧАТОК РОБОТИ

2.1. Мова Dart. Перша програма

Dart є мовою програмування загального призначення від компанії Google, яка призначена насамперед для розробки вебзастосунків (як на стороні клієнта, так і на стороні сервера) і мобільних застосунків. Це також означає, що одну і ту ж програму на Dart можна компілювати під різні платформи - Windows (x86 / 64), Android, iOS.

Dart - об'єктно-орієнтована мова. Всі значення, які використовуються в програмі на Dart, є об'єктами.

У своєму розвитку Dart зазнав впливу більш ранніх мов, таких як Smalltalk, Java, JavaScript. Її синтаксис схожий на синтаксис інших С-подібних мов.

Dart швидко розвивається. Поточна версія - 3.0.

2.1.1. Установка Dart

Для роботи з Dart необхідно встановити Dart SDK. Для цього потрібно завантажити zip-архів з SDK з адреси <https://dart.dev/tools/sdk/archive> вигляд сторінки, якої наведено на рисунку 2.1, і розпакувати його на жорсткому диску.

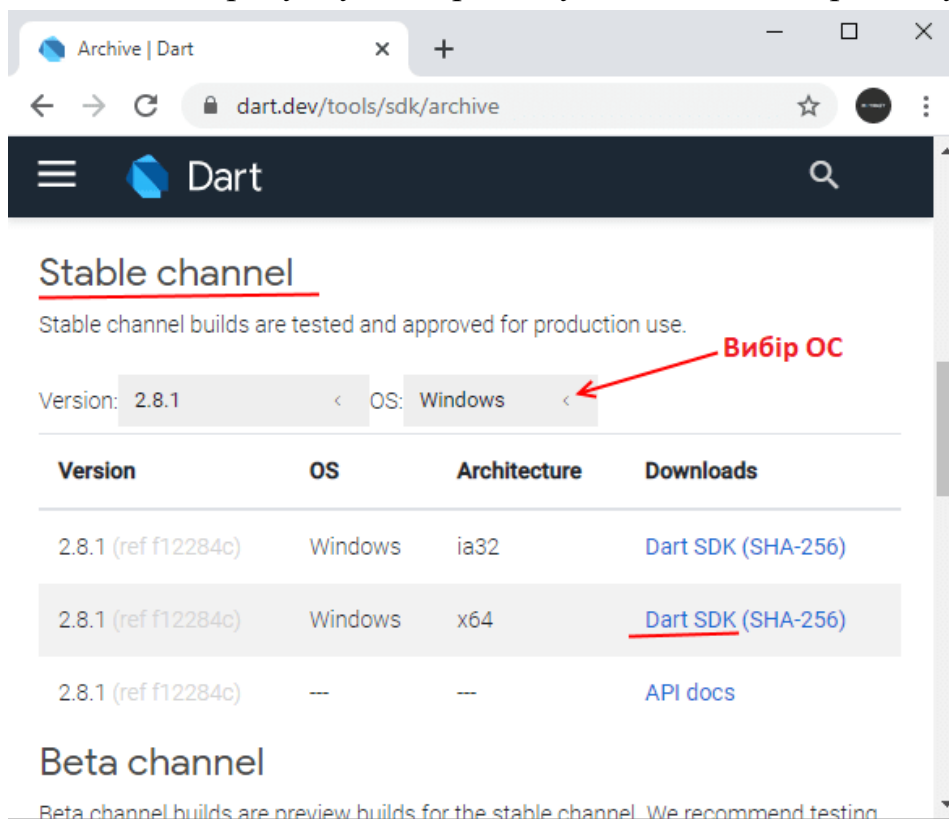


Рисунок 2.1. Вигляд вікна вибору версії SDK

На сторінці завантаження є пакети для Windows, Linux, MacOS. Також варто відзначити, що доступні різні збірки: бета-збірки і збірки для розробників. В нашому ж випадку будемо використовувати стабільну збірку, яка рекомендується до використання в розробці, тому при завантаженні вибираємо потрібний пакет в секції «Stable channel».

Припустимо, в нашому випадку Dart SDK розпакований по шляху **C:\dart-sdk**. Якщо перейдемо в розпакованому SDK до теки **bin**, то можемо побачити там таку програму як **dart.exe**, як показано на рисунку 2.2.

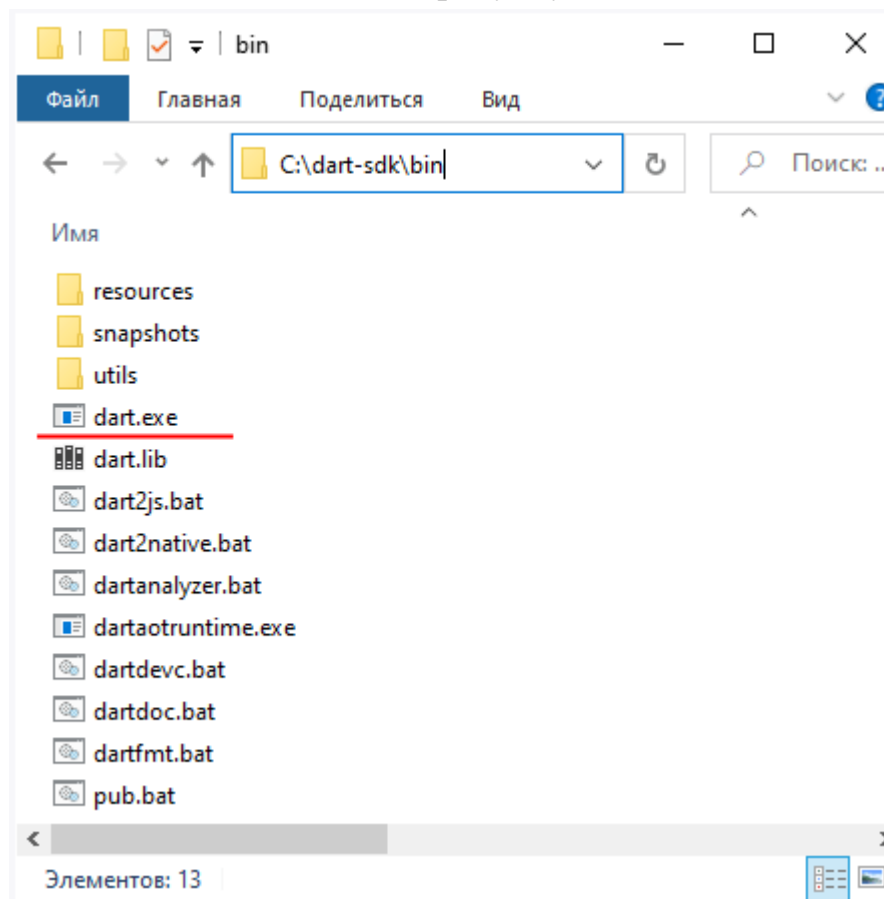


Рисунок 2.2. Вигляд теки C:\dart-sdk\bin

За допомогою цієї утиліти власне і будуть запускатися скрипти на мові Dart.

2.1.2. Перша програма

Напишемо першу найпростішу програму. Для цього визначимо на жорсткому диску каталог для файлів. Наприклад, в наведеному випадку нехай це буде каталог **C:\dart**. Створимо в каталозі новий текстовий файл, який буде називатися **main.dart** (файли з кодом Dart зазвичай мають розширення ***.dart**). Відкриємо цей файл в будь-якому текстовому редакторі, вигляд якого показано на рисунку 2.3, і визначимо в цьому файлі наступний найпростіший код:

```

1  void main() {
2    print("Hello, Dart!");
3  }

```

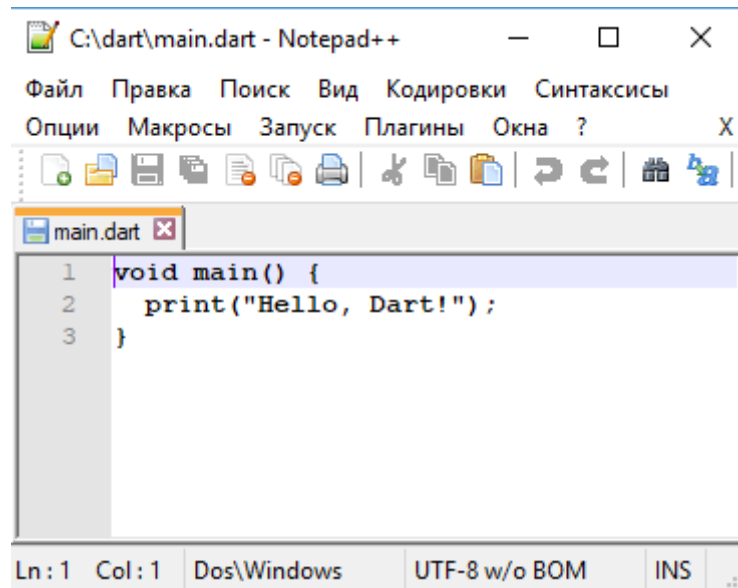


Рисунок 2.3. Вигляд текстового редактора при написанні коду програм

Будь-який застосунок на мові Dart повинна мати функцію, яка називається **main**. Ця функція має тип **void** і не приймає ніяких параметрів, тому після назви функції йдуть порожні дужки.

Тіло функції розташовується у фігурних дужках. Зокрема, в наведеному випадку у функції **main** викликається інша функція - **print()**, яка виводить на консоль деякий рядок. В наведеному прикладі це рядок «Hello, Dart!»

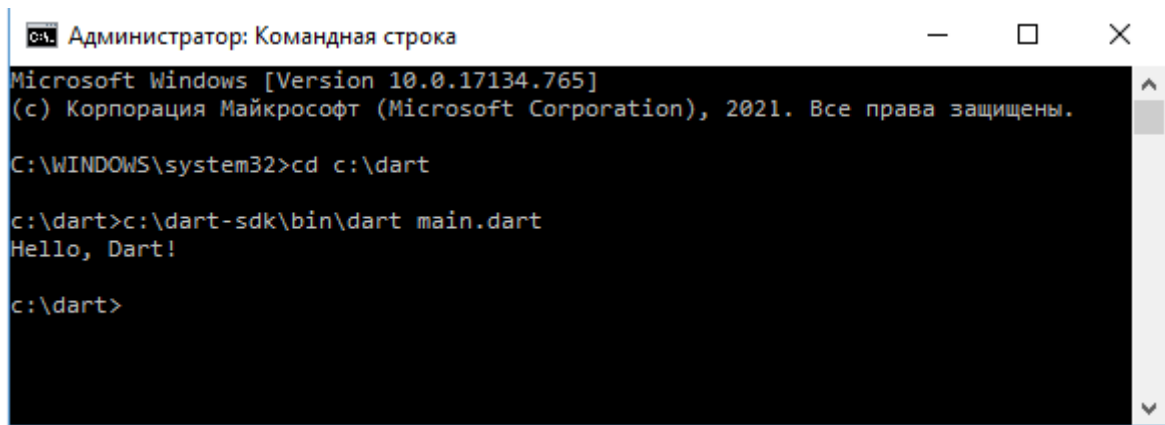
Тепер запусимо цю програму. Для цього відкриємо командний рядок/термінал і перейдемо до теки з файлом **main.dart** за допомогою команди **cd**.

Потім введемо в командний рядок наступний текст для запуску скрипта, як показано на рисунку 2.4.

```
c: \dart-sdk\bin\dart main.dart
```

Рисунок 2.4. Вигляд командного рядку

В нашому випадку мається на увазі, що **dart sdk** розпаковано по шляху **c:\dart-sdk**. І в цьому SDK викликається програма **dart.exe**, якій передається наш скрипт **main.dart**. І після цього буде виконуватися програма, яка виведе на консоль рядок, як показано на рисунку 2.5.



```
Администратор: Командная строка
Microsoft Windows [Version 10.0.17134.765]
(c) Корпорация Майкрософт (Microsoft Corporation), 2021. Все права защищены.

C:\WINDOWS\system32>cd c:\dart

c:\dart>c:\dart-sdk\bin\dart main.dart
Hello, Dart!

c:\dart>
```

Рисунок 2.5. Результат работы программы

Щоб кожного разу при запуску програми не треба було вводити повний шлях до утиліти **dart.exe**, можна додати шлях до утиліти в змінні середовища, як показано на рисунку 2.6.

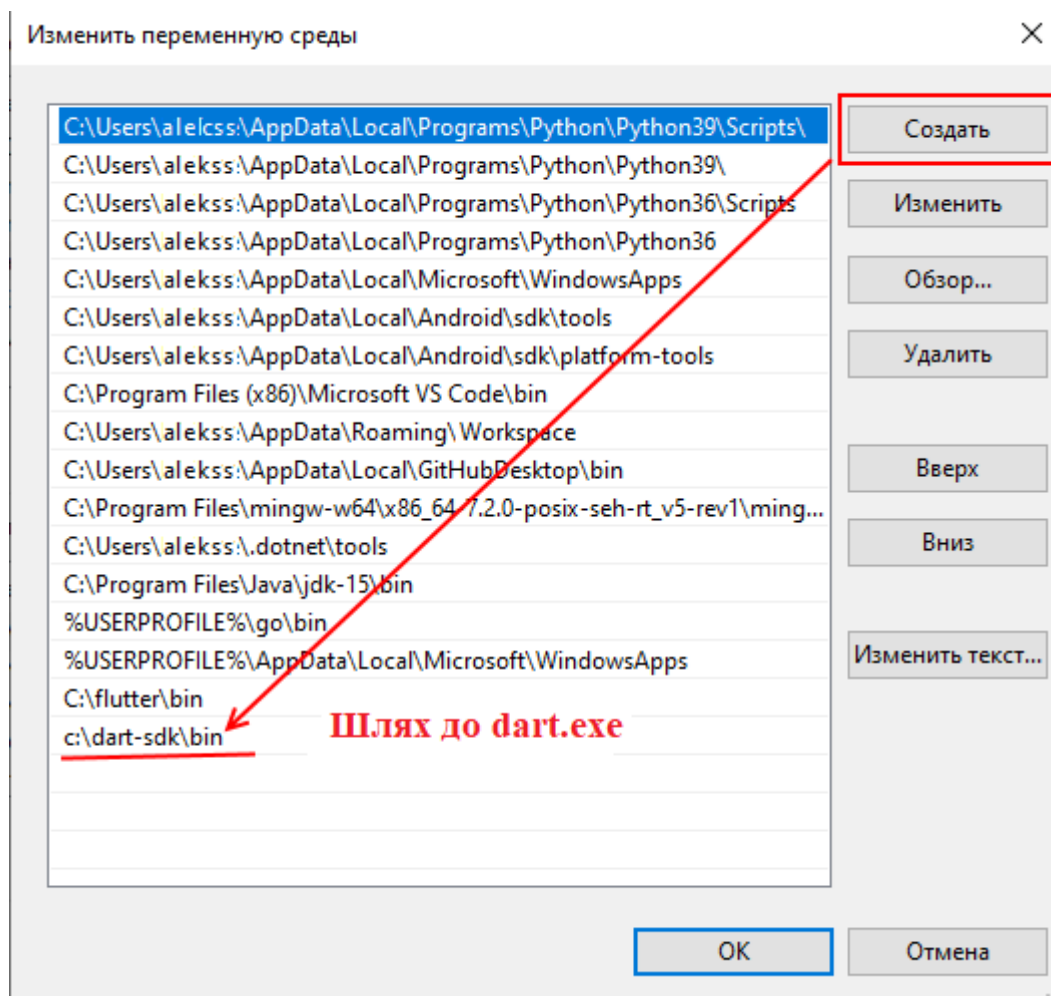


Рисунок 2.6. Вікно додавання змінних середовища виконання

Відповідно після цього можна буде запускати програму на Dart так, як показано на рисунку 2.7.

```
dart main.dart
```

Рисунок 2.7. Команда запуску програми на виконання

2.1.3. Компіляція програми

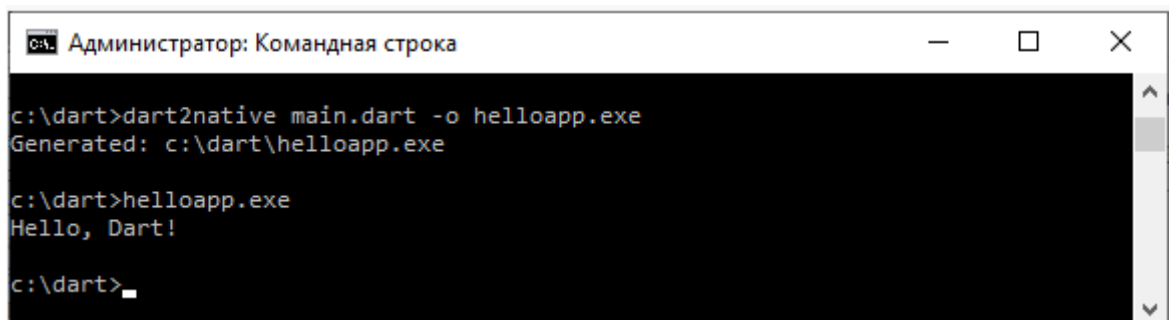
За допомогою утиліти **dart.exe** можемо запустити програму, проте як можна створити якийсь виконуваний файл, щоб в будь-який час його можна було запускати (без звернення до **dart.exe**) і переносити на інший комп'ютер з тієї ж операційною системою? Для цього в SDK є друга утиліта - **dart2native** (в тій же теці, що і **dart.exe**), яка дозволяє скомпільовати нативний виконуваний файл програми. Як параметр вона приймає вихідний файл, який необхідно скомпільовати. А після прапорця **-o** можна вказати шлях і назву файлу, який буде скомпільовано, як показано на рисунку 2.8.

```
dart2native main.dart -o helloapp.exe
```

Рисунок 2.8. Вигляд командного рядку

В наведеному прикладі буде скомпільовано файл **helloapp.exe**.

Отже, скомпілюємо і запустимо файл програми, результат роботи якої показано на рисунку 2.9.



```
c:\dart>dart2native main.dart -o helloapp.exe
Generated: c:\dart\helloapp.exe

c:\dart>helloapp.exe
Hello, Dart!

c:\dart>
```

Рисунок 2.9. Результат роботи програми

Після цього в теці програми з'явиться файл **helloapp.exe**, який можна буде запускати незалежно від **dart.exe**.

Резюме

В цьому розділі було розглянуто питання встановлення Dart для різних платформ та перелік платформ, під які можна скомпільовати програми на Dart.

Розглянуто структуру програми та перелік команд для компіляції та запуску її на виконання.

Показано яким чином компілюється код у нативний виконуваний файл програми.

Контрольні запитання і завдання

1. Як встановити Dart для різних платформ?
2. Під які платформи можна скомпілювати програми на Dart?
3. Мова Dart об'єктно-орієнтована?
4. Чи всі значення в Dart є об'єктами? Наведіть приклади виключень, якщо вони є.
5. Що таке функції **main** у Dart?
6. Який вигляд має команда для запуску програм *.dart?
7. Як зробити, аби кожного разу при запуску програми не треба було вводити повний шлях до утиліти **dart.exe**?
8. Як в Dart скомпілювати нативний виконуваний файл програми?
9. Створіть програму привітання з вашим початком вивчення мови Dart.
10. Скомпілюйте та запустіть програму з п.9.
11. Обміняйтеся програмою з п.9. з одногрупниками з іншою платформою.
12. Створіть виконуваний файл, та перевірте його роботу на інших комп'ютерах.

3. ОСНОВИ МОВИ DART

3.1. Структура програми

Вхідною точкою в програму на мові Dart є функція **main**. Саме з неї починається виконання програми. Вона обов'язково має бути присутня в програмі. Її найпростіше визначення може бути наступним:

```
1 void main() {  
2  
3 }
```

Слово **void** на початку визначення функції вказує, що вона не повертає ніякого значення, просто виконує деякі дії. Далі йде власне ім'я функції - **main** і потім порожні дужки, які вказують, що функції не приймає параметрів. Хоча також можемо використовувати і іншу форму функції **main**, яка приймає параметри - і через ці параметри передати функції дані ззовні під час запуску програми.

Після порожніх дужок йде блок функції у вигляді фігурних дужок {}, які відкриваються і закриваються, всередині яких міститься набір виконуваних інструкцій - ті дії, які і буде виконувати програма.

Інструкції (statement) є основним будівельним блоком у програмі на Dart. Кожна інструкція виконує деяку дію, наприклад, виклики методів, оголошення змінних і присвоєння їм значень. Після завершення інструкції в Dart ставиться крапка з комою (;). Цей знак свідчить про кінець інструкції, наприклад:

```
1 print("Hello Dart!");
```

Цей рядок здійснює виклик функції **print**, яка виводить на консоль рядок «Hello Dart!». В наведеному прикладі виклик функції є інструкцією і тому завершується крапкою з комою.

Крім окремих інструкцій поширеною конструкцією є блок коду. Блок коду містить набір інструкцій, він розташовується у фігурних дужках, а інструкції містяться між фігурними дужками, які відкриваються і закриваються:

```
1 {  
2     print("Hello!");  
3     print("Welcome to Dart!");  
4 }
```

У цьому блоці коду дві інструкції, які виводять на консоль певний рядок.

Функція **main** по суті також є блоком коду і також може містити інші блоки коду:


```

1  void main() {
2
3      {
4          print("Hello!");
5          print("Welcome to Dart!");
6      }
7  }

```

3.1.1. Коментарі

Код програми може містити коментарі. Коментарі дозволяють зрозуміти зміст програми, що виконують ті чи інші її частини. При компіляції коментарі ігноруються і не мають жодного впливу на роботу програми та на її розмір.

В Dart є два типи коментарів: **однорядковий** і **багаторядковий**. **Однорядковий** коментар розташовується на одному рядку після подвійного слеша //. А **багаторядковий** коментар розташовується між символами **/* текст коментаря */**. Він може розташовуватися на декількох рядках.

Наприклад:

```

1  /*
2      багаторядковий коментар
3      Перша програма
4      на мові Dart
5  */
6  // визначення функції main
7  void main () {           // початок оголошення функції
8
9      print("Hello Dart!"); // друк рядка на
      консоль
10 } // кінець оголошення функції

```

3.2. Змінні і константи

Для зберігання даних в мові Dart, як і в інших мовах програмування, використовуються змінні. Змінна визначається наступним чином:

```

1  Тип_даних ім'я_змінної;

```

Тип даних є одним з вбудованих типів або типів, визначених програмістом. Ім'я змінної є довільною назвою, яка може містити алфавітно-цифрові символи і символ підкреслення та яка не має збігатися з одним з ключових слів мови: **abstract, else, import, super, as, enum, in, switch, assert, export, interface, sync,**

async, extends, is, this, await, extension, library, throw, break, external, mixin, true, case, factory, new, try, catch, false, null, typedef, class, final, on, var, const, finally, operator, void, continue, for, part, while, covariant, Function, rethrow, with, default, get, return, yield, deferred, hide, set, do, if, show, dynamic, implements, static.

Наприклад, визначення змінної **name** типу **String**:

```
1 String name;
```

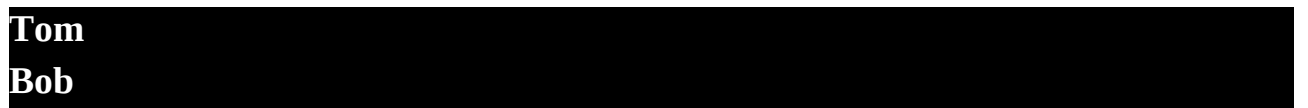
Для іменування змінних застосовується стиль **lowerCamelCase**, тобто назва змінної починається з маленької літери. Якщо ім'я змінної складається з двох і більше слів, то перше слово починається з маленької, а всі наступні слова з великої літери, наприклад, **userAge**, або **oldUserAge**.

3.2.1. Значення змінної

За допомогою операції присвоєння змінній можна присвоїти значення, яке відповідає її типу. Наприклад, тип **String** є рядком. Відповідно змінній цього типу ми можемо присвоїти рядок:

```
1 void main() {  
2     String name = "Tom";  
3     print(name);  
4     name = "Bob";  
5     print(name);  
6 }
```

Результат роботи цієї програми показано на рисунку 3.1.



Tom
Bob

Рисунок 3.1. Результат роботи програми

3.2.2. Var

Для визначення змінної також можна використовувати ключове слово **var**:

```
1 var name = "Tom";
```

В цьому прикладі Dart сам визначає тип змінної, виходячи з присвоєного їй значення.

Ще одним способом визначення змінної є використання оператора **dynamic**:

```
1 dynamic name = "Tom";
```

Цей оператор, як і **var**, дозволяє визначити тип змінної виходячи з присвоєного їй значення. Але на відміну від **var**, **dynamic** дозволяє змінювати тип змінної. Наприклад, якщо в наступному прикладі при виконанні програми ми отримаємо помилку:

```
1 void main() {
2     var name = "Tom";
3     print(name);
4     name = 45;      // Помилка!!! name є рядком, а не
    цілим числом
5     print(name);
6 }
```

В цьому випадку виходячи з присвоєного змінній **name** початкового значення (рядок «Tom») для неї визначається тип **String**, який є рядком. Однак далі ми намагаємося присвоїти змінній число - 45. Це значення не є рядком, тому не вписується в діапазон допустимих значень для типу **String** - типу змінної **name**, тому програма викидає помилку.

Щоб можна було динамічно змінювати тип даних, застосовується оператор **dynamic**. Так, наступна програма виконатися без помилок:

```
1 void main() {
2     dynamic name = "Tom";
3     print(name);
4     name = 45;
5     print(name);
6 }
```

3.2.3. Const і final

Одним з характерних якостей змінних є те, що протягом роботи програми можна скільки завгодно змінювати їх значення:

```
1 void main() {
2     var name = "Tom";
3     print(name);    // Tom
4     name = "Bob";
5     print(name);    // Bob
6     name = "Sam";
7     print(name);    // Sam
8 }
```

Якщо ж значення змінної міняти не потрібно, а іноді навіть небажано, то замість змінних можна використовувати константи. Константи визначаються за допомогою операторів **const** і **final**:

```
1 void main () {
2
3     const name = "Tom";
4     print(name);    // Tom
5     //name = "Bob";
6     // Помилка - значення константи змінювати не
    можна!
7 }
```

Якщо спробувати змінити саме значення константи, то в процесі компіляції програми зіткнемося з помилкою.

Використання **final** аналогічне:

```
1 void main () {
2
3     final name = "Tom";
4     print(name);    // Tom
5     //name = "Bob";    // Помилка - значення
    константи змінювати не можна!
6 }
```

Головна відмінність між **const** і **final** полягає в тому, що значення **const** має бути визначено при компіляції, а значення константи **final** визначається під час виконання. При визначенні та використанні констант **const** або **final** в контексті функції **main** між ними великих відмінностей немає, проте в інших контекстах (наприклад, при використанні інших функцій і класів) є деякі синтаксичні відмінності, які будуть розглянуті в наступних відповідних темах.

Приклади доступних способів визначення констант:

```
1 void main () {
2
3     const a = 5;
4     const int b = 6;
5     final int c = 8;
6     final d = 9;
7     print("a:$a b:$b c:$c d:$d");
8 }
```

Для виведення на консоль в наведеному прикладі застосовується такий прийом як **інтерполяція**, яка дозволяє після символу **\$** вказати ім'я змінної, значення якої буде виводитися на консоль.

3.3. Примітивні типи даних

Тип даних визначає, які саме значення може зберігати об'єкт. Dart має декілька примітивних вбудованих типів:

- ✓ логічний тип **bool**;
- ✓ числові типи (**int**, **double**);
- ✓ строкові типи (**String**, **Runes**);
- ✓ **symbol**.

3.3.1. Bool

Тип **bool** позначає два значення: **true** і **false**.

```
1 var isEnabled = false;
2 bool isAlive = true;
```

3.3.2. Int

Типом **int** є цілі числа, які займають не більше 64 біт, точний розмір залежить від платформи. Наприклад, для Dart VM значення варіюються від -2^{63} до $2^{63} - 1$. Dart, який компілюється в JavaScript, використовує систему числових типів JavaScript з діапазоном значень від -2^{53} до $2^{53} - 1$.

Будь-які цілі числа трактуються як значення типу **int**:

```
1 int x = 8;
2 var y = -5;
```

3.3.3. Double

Типом **double** є числа з плаваючою крапкою, які займають в пам'яті 64 біта.

Всі дробові числа з крапкою як роздільник цілої та дробової частини трактуються як значення типу **double**:

```
1 double x = 8.8;
2 var y = -5.3;
3 var z = 0.09;
```

Змінній типу **double** також можна присвоїти цілочисельне значення, в цьому випадку воно автоматично перетвориться в дробове:

```
1 double x = 8;    // x = 8.0
2 print(x);       // 8.0
```

3.3.4. String

Рядки представлені типом **String**, який є послідовністю символів в кодуванні UTF-16. Для визначення рядка можна використовувати одинарні та подвійні лапки:

```
1 String tom = "Tom";
2 String sam = 'Sam';
3 var kate = "Kate";
4 var alice = 'Alice';
```

Якщо необхідно визначити багаторядковий текст, то він розташовується в потрійних лапках:

```
1 void main() {
2     var multiline = '''
3         Багаторядковий
4         текст
5     ''';
6     print(multiline);
7     String users = """
8         Список користувачів
9         Том
10        Сэм
11    """;
12     print(users);
13 }
```

За допомогою **інтерполяції** можна вводити в рядок значення інших змінних:

```
1 String name = "Tom";
2 int age = 35;
3 String info = "Name: $name Age: $age";
```

Для введення значень в рядок перед змінною ставиться знак долара **\$** (**\$name**, **\$age**), в результаті замість **\$name** вставлятиметься значення змінної **name**, а замість **\$age** - значення змінної **age**.

3.3.5. Runes

Тип **Runes** також є рядком, але на відміну від **String**, **Runes** - це послідовність символів в кодуванні UTF-16. Оскільки за замовчуванням всі рядки в лапках (як одинарних, так і подвійних) мають тип **String**, то для визначення змінної **Runes** потрібен спеціальний синтаксис:

```
1 void main() {
2     Runes input = new
    Runes ('\u041F\u0440\u0438\u0432\u0456\u0442');
3     Runes text = new Runes ('Привіт');
4     print(text);
5     print(String.fromCharCode(input));
6     print(String.fromCharCode(text));
7 }
```

Для визначення значення типу **Runes** застосовується конструктор даного типу. Тобто на початку йде оператор **new**, потім назва типу - **Runes**, після якого в дужках вказується аргумент, що передається конструктору. Можна передати в конструктор або набір кодових одиниць в кодуванні UTF-16 (**new Runes ('\u041F \u0440 \u0438 \u0432 \u0456 \u0442')**), або звичайний набір символів (**new Runes ('Привіт')**).

Щоб отримати текстове подання об'єкта **Runes**, застосовується метод **String.fromCharCode**.

3.3.6. Symbol

Тип **Symbol**, найменш використовуваний з усіх типів, і позначає символні ідентифікатори, які, як правило, застосовуються для посилання на якісь елементи API, наприклад, бібліотеки і класи. Для визначення об'єкта цього типу застосовується символ решітки **#**:

```
1 Symbol libName = #foo_lib;
2 var className = #foo;
```

3.4. Значення null, nullable-типи і null-безпека

В мові Dart є спеціальна константа **null**, яка говорить, що у змінної або константи по суті не вказано ніякого значення. Однак при використанні цієї константи слід враховувати, що, починаючи з версії 2.12 в Dart за замовчуванням застосовується **null-безпека** або **null-safety**. Розглянемо що це означає.

Розглянемо, наприклад, таку програму:

```

1  void main() {
2      String name;
3      print(name);
4      name = "Tom";
5      print(name);
6  }

```

Тут змінній **name** при її визначенні не присвоюють ніякого значення. І, починаючи з версії Dart 2.12, при компіляції програми в цьому разі отримаємо помилку, як показано на рисунку 3.2.

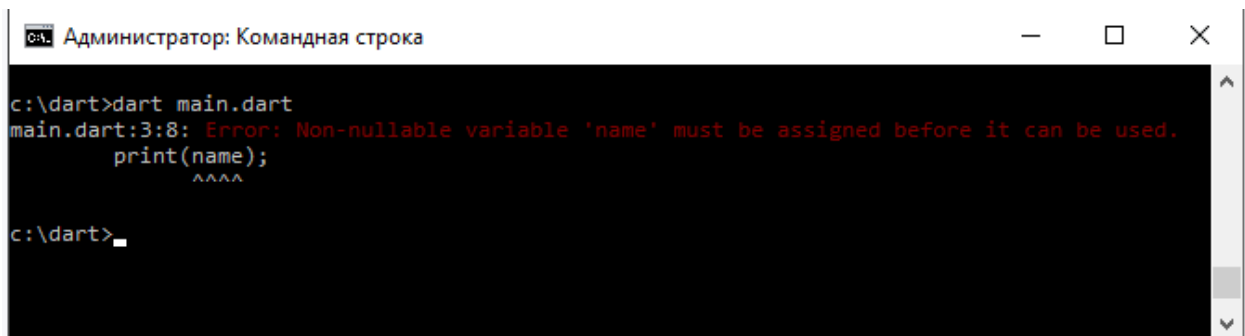


Рисунок 3.2. Результат роботи програми

Помилка говорить про те, що було намагання використовувати змінну, яка не може приймати значення **null**, до присвоєння їй значення.

За замовчуванням всі примітивні вбудовані типи розцінюються як типи, які не допускають значення **null**, тому обов'язково необхідно присвоювати їх змінним і константам значення перед першим використанням:

```

1  void main() {
2      String name = "Bob";
3      print(name);
4      name = "Tom";
5      print(name);
6  }

```

3.4.1. Оператор ? і nullable-типи

Однак, в деяких ситуаціях може знадобитися вказати, що змінна або константа не має ніякого значення. Тобто необхідно, щоб вона могла приймати значення **null**. У цьому випадку можна використовувати, так звані **nullable-типи**. Вони є по суті двійниками звичайних типів за тим винятком, що їх змінні можуть приймати значення **null**.

Щоб визначити тип як **nullable** (яке допускає значення **null**), після назви типу ставиться оператор **?** (знак питання). Наприклад:

```
1 void main() {
2     String? name;
3     print(name);
4     name = "Tom";
5     print(name);
6 }
```

Тип **String?** вказує, що його змінна може містити або рядок, або значення **null**. І при виконанні цієї програми отримаємо вигляд консолі, як показано на рисунку 3.3.



```

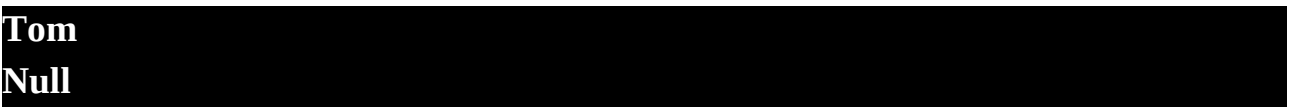
null
Tom
```

Рисунок 3.3. Результат роботи програми

Змінним таких типів також можна явно присвоювати **null**:

```
1 void main() {
2     String? name = "Tom";
3     print(name);
4     name = null;
5     print(name);
6 }
```

Вигляд результату роботи програми показано на рисунку 3.4.



```

Tom
Null
```

Рисунок 3.4. Результат роботи програми

3.4.2. Оператор ??

Оператор **??** при присвоєнні змінній значень виконує перевірку на **null**. З'ясуємо навіщо він потрібен. Розглянемо наступний приклад:

```

1 void main() {
2     int? num1 = 23;
3     int num2 = num1;
4     print(num2);
5
6     num1 = null;
7     num2 = num1;
8     print(num2);
9 }

```

Тут визначені дві змінні: **num1** і **num2**. Обидві вони можуть зберігати цілі числа. Однак крім того, змінна **num1** може також зберігати значення **null**. Змінній **num2**, яка НЕ може приймати **null**, передається значення **num1**, яке може зберігати **null**. Відповідно тут можна зіткнутися з потенційною помилкою, в разі якщо **num1 = null**.

Для вирішення подібної проблеми можна використовувати оператор **??**:

```

1 value1 ?? value2

```

Якщо значення **value1** (значення зліва від оператора **??**) не дорівнює **null**, то оператор повертає саме це значення **value1**. Якщо ж це значення дорівнює **null**, то оператор **??** повертає значення **value2** (праворуч від оператора).

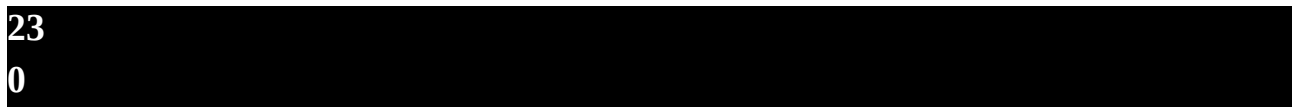
```

1 void main() {
2     int? num1 = 23;
3     int num2 = num1 ?? 0;
4     print(num2);           // 23
5
6     num1 = null;
7     num2 = num1 ?? 0;
8     print(num2);           // 0
9 }

```

При першому присвоєнні змінна **num2** отримає значення змінної **num1** (число 23), так як **num1** не дорівнює **null**. При другому присвоєнні **num2** отримає значення 0, так як **num1** дорівнює **null**.

Вигляд результату роботи програми на консолі показано на рисунку 3.5.



```

23
0

```

Рисунок 3.5. Результат роботи програми

3.4.3. Оператор !

У деяких ситуаціях змінна може бути визначена як змінна **nullable-типу**, проте при цьому можуть бути виключені ситуації, що дана змінна буде зберігати **null**. І якщо точно впевнені, що ця змінна в процесі роботи програми не отримає значення **null**, то в цьому випадку можна використовувати оператор **!**, який ставиться після назви змінної:

```
1 void main() {
2     int? a = 23;
3     int b = a!; // ми впевнені, що a не дорівнює null
4     print(b);  // 23
5 }
```

У той же час, якщо такої впевненості немає, то краще не зловживати цим оператором і не використовувати його.

3.5. Арифметичні операції

Мова Dart підтримує всі поширені арифметичні операції. В арифметичних операціях беруть участь числа. Є бінарні арифметичні операції (виконуються над двома операндами) і унарні (виконуються над одним операндом). До бінарних операцій відносять наступні:

• +

Операція додавання двох чисел:

```
1 int a = 10;
2 int b = 7;
3 int c = a + b; // 17
4 int d = 4 + b; // 11
```

• -

Операція віднімання двох чисел:

```
1 int a = 10;
2 int b = 7;
3 int c = a - b; // 3
4 int d = 4 - a; // -6
```

• *

Операція множення двох чисел:

```
1 int a = 10;
2 int b = 7;
3 int c = a * b; // 70
4 int d = b * 5; // 35
```

- /

Операція ділення двох чисел:

```
1 int a = 20;
2 int b = 5;
3 double c = a / b; // 4.0
4 double d = 22.5 / 4.5; // 5.0
```

При діленні варто враховувати, що результат операції завжди є типом **double**.

- ~/

Операція цілочисельного ділення двох чисел. Результат операції завжди є типом **int**:

```
1 int a = 33;
2 int b = 5;
3 int c = a ~/ b; // 6
4 int d = 22 ~/ 4; // 5
```

- %

Отримання залишку від ділення двох чисел:

```
1 int a = 33;
2 int b = 5;
3 int c = a % b; // 3
4 int d = 22 % 4; // 2 (22 - 4*5 = 2)
```

Також є дві унарні арифметичні операції, які здійснюються над одним числом: ++ (інкремент) і -- (декремент). Кожна з операцій має два різновиди: префіксний і постфіксний:

- ++ (префіксний інкремент)

Передбачає збільшення змінної на одиницю, наприклад, **z = ++ y** (спочатку значення змінної **y** збільшується на 1, а потім її значення присвоюється змінній **z**):

```
1 int a = 8;
2 int b = ++a;
3 print(a); // 9
4 print(b); // 9
```

- ++ (постфіксний інкремент)

Також виконує збільшення змінної на одиницю, наприклад, **z = y ++** (спочатку значення змінної **y** присвоюється змінній **z**, а потім значення змінної **y** збільшується на 1):

```

1  int a = 8;
2  int b = a++;
3  print(a);    // 9
4  print(b);    // 8

```

- -- (префіксний декремент)

Зменшення змінної на одиницю, наприклад, $z = --y$ (спочатку значення змінної y зменшується на 1, а потім її значення присвоюється змінній z):

```

1  int a = 8;
2  int b = --a;
3  print(a);    // 7
4  print(b);    // 7

```

- -- (постфіксний декремент)

Також виконує зменшення змінної на одиницю, наприклад $z = y--$ (спочатку значення змінної y присвоюється змінній z , а потім значення змінної y зменшується на 1):

```

1  int a = 8;
2  int b = a--;
3  print(a);    // 7
4  print(b);    // 8

```

3.5.1. Пріоритет арифметичних операцій

Одні операції мають більший пріоритет ніж інші і тому виконуються першими. Операції в порядку зменшення пріоритету:

++ (постфіксний інкремент), -- (постфіксний декремент)

++ (префіксний інкремент), -- (префіксний декремент)

* (множення), / (ділення), % (залишок від ділення)

+ (Додавання), - (віднімання)

Пріоритет операцій слід враховувати при виконанні запису арифметичних виразів:

```

1  int a = 8;
2  int b = 7;
3  int c = a + 5 * ++b;
4  print(c);    // 48

```

Спочатку буде виконуватися операція інкремента $++b$, яка має більший пріоритет - вона збільшить значення змінної b і поверне його в якості результату. Потім виконується множення $5 * ++b$, і тільки в останню чергу виконується складання $a + 5 * ++b$.

Дужки дозволяють перевизначити порядок обчислень:

```
1  int a = 8;
2  int b = 7;
3  int c = (a + 5) * ++b;
4  print(c);    // 104
```

Хоча операція додавання має менший пріоритет, але спочатку буде виконуватися саме складання, а не множення, так як операція складання укладена в дужки.

3.6. Побітові операції і операції зсуву

Побітові операції і операції зсуву здійснюються над окремими розрядами або бітами числа. Щоб зрозуміти подібні операції, розглянемо двійкове подання чисел. Кожне число має певне двійкове подання. Наприклад, число 4 в двійковій системі 100, а число 5 - 101 і так далі.

Для запису чисел зі знаком застосовується **додатковий код (two's complement)**, при якому старший розряд є знаковим. Якщо його значення дорівнює 0, то число позитивне, і його двійкове подання не відрізняється від позначення беззнакового числа. Наприклад, 0000 0001 в десятковій системі 1.

Якщо старший розряд дорівнює 1, то маємо справу з негативним числом. Наприклад, 1111 1111 в десятковій системі є -1. Відповідно, 1111 0011 позначає -13.

3.6.1. Логічні операції

Логічні операції над числами є порозрядними операціями. В цьому випадку числа розглядаються в двійковому поданні, наприклад, 2 в двійковій системі дорівнює 10 і має два розряди, число 7 - 111 і має три розряди.

- **&** (Логічне множення):

Множення здійснюється порозрядно, і якщо у обох операндах значення розрядів дорівнює 1, то операція повертає 1, інакше повертається число 0. Наприклад:

```
1  int a1 = 2;           //010
2  int b1 = 5;           //101
3  print(a1&b1);        //000 - результат 0
4
5  int a2 = 4;           //100
6  int b2 = 5;           //101
7  print(a2 & b2);       //100 - результат 4
```

У першому прикладі у нас два числа 2 і 5. 2 в двійковому вигляді є числом 010, а 5 - 101. Порозрядне множення чисел ($0*1$, $1*0$, $0*1$) дає результат 000.

У другому прикладі у нас замість двійки число 4, у якого в першому розряді 1, так само як і у числа 5, тому тут результатом операції ($1*1$, $0*0$, $0*1$) = 100 буде число 4 в десятковому форматі.

- | (Логічне додавання):

Ця операція також проводиться над двійковими розрядами, але тепер повертається одиниця, якщо хоча б у одного числа в конкретному розряді є одиниця (операція «логічне АБО»). Наприклад:

```
1  int a1 = 2;      //010
2  int b1 = 5;      //101
3  print(a1|b1);    //111- результат 7
4  int a2 = 4;      //100
5  int b2 = 5;      //101
6  print(a2 | b2);  //101 - результат 5
```

- ^ (Логічне виключення АБО):

Також цю операцію називають XOR, нерідко її застосовують для простого шифрування:

```
1  int number = 45; // 1001 Значення, яке необхідно
   зашифрувати - в двійковому поданні 101101
2  int key = 102;  //Ключ шифрування - в двійковому
   поданні 1100110
3  int encrypt = number ^ key; //Результатом буде число
   1001011 або ж 75
4  print("Зашифроване число: $encrypt");
5
6  int decrypt = encrypt ^ key; // Результатом буде
   початкове число 45
7  print("Розшифроване число: $decrypt");
```

Тут також здійснюється порозрядні операції. Якщо значення поточного розряду в обох числах різні, то повертається 1, інакше повертається 0. Наприклад, результатом виразу 9^5 буде число 12. А щоб розшифрувати число, необхідно застосувати зворотну операцію до результату.

- ~ (Логічне заперечення):

Порозрядна операція, яка інвертує всі розряди числа: якщо значення розряду дорівнює 1, то воно стає рівним нулю, і навпаки.

```

1  int a = 12;           // 0000 1100
2  print(~a);           // 1111 0011    або ж -13

```

3.6.2. Операції зсуву

Операції зсуву також здійснюються над розрядами чисел. Зсув може здійснюватися праворуч і ліворуч.

- **a<<b** - зсуває число **a** ліворуч на **b** розрядів. Наприклад, вираз **4<<1** зсуває число 4 (яке в двійковому поданні 100) на один розряд ліворуч, в результаті виходить число 1000 або число 8 в десятковому поданні.

- **a>>b** - зсуває число **a** праворуч на **b** розрядів. Наприклад, **16>>1** зсуває число 16 (яке в двійковій системі 10000) на один розряд праворуч, тобто в результаті виходить 1000 або число 8 в десятковому поданні.

3.7. Умовні вирази

Умовні вирази є деякими умовами і повертають значення типу **bool**, тобто значення **true** (якщо умова істинна), або значення **false** (якщо умова помилкова). До умовних виразів належать операції порівняння і логічні операції.

3.7.1. Операції порівняння

В операціях порівняння порівнюються два операнда, і повертається значення типу **boolean** - **true**, якщо вираз вірний, і **false**, якщо вираз невірний.

- **==**

Порівнює два операнда на рівність і повертає **true** (якщо операнди рівні) і **false** (якщо операнди не рівні):

```

1  int a = 10;
2  int b = 4;
3  bool c = a == b;           // false
4  bool d = a == 10;          // true

```

- **!=**

Порівнює два операнда і повертає **true**, якщо операнди НЕ рівні, і **false**, якщо операнди рівні:

```

1  int a = 10;
2  int b = 4;
3  bool c = a != b;           // true
4  bool d = a != 10;          // false

```

- **<** (менше ніж)

Повертає **true**, якщо перший операнд менше другого, інакше повертає **false**:

```
1 int a = 10;
2 int b = 4;
3 bool c = a < b; // false
```

- > (більше ніж)

Повертає **true**, якщо перший операнд більше другого, інакше повертає **false**:

```
1 int a = 10;
2 int b = 4;
3 bool c = a > b; // true
```

- >= (більше або дорівнює)

Повертає **true**, якщо перший операнд більше другого або дорівнює другому, інакше повертає **false**:

```
1 bool c = 10 >= 10; // true
2 bool b = 10 >= 4;  // true
3 bool d = 10 >= 20; // false
```

- <= (менше або дорівнює)

Повертає **true**, якщо перший операнд менше другого або дорівнює другому, інакше повертає **false**:

```
1 bool c = 10 <= 10; // true
2 bool b = 10 <= 4;  // false
3 bool d = 10 <= 20; // true
```

3.7.2. Логічні операції

Крім того, в мові Dart є логічні операції, які також є умовами і повертають **true** або **false** і зазвичай об'єднують кілька операцій порівняння. До логічних операцій відносять наступні:

- |

c = a | b; (с дорівнює **true**, якщо або **a**, або **b** (або і **a**, і **b**) дорівнюють **true**, інакше **c** дорівнюватиме **false**);

- &

c = a & b; (с дорівнює **true**, якщо і **a**, і **b** дорівнюють **true**, інакше **c** дорівнюватиме **false**);

- !

c = ! b; (с дорівнює **true**, якщо **b** дорівнює **false**, інакше **c** дорівнюватиме **false**);

- $\wedge$

$c = a \wedge b$; (с дорівнює **true**, якщо або **a**, або **b** (але не одночасно) дорівнюють **true**, інакше с дорівнюватиме **false**);

- $\parallel$

$c = a \parallel b$; (с дорівнює **true**, якщо або **a**, або **b** (або і **a**, і **b**) дорівнюють **true**, інакше с дорівнюватиме **false**);

- $\&\&$

$c = a \&\& b$; (с дорівнює **true**, якщо і **a**, і **b** дорівнюють **true**, інакше с дорівнюватиме **false**).

Отже, є дві пари операцій $|$ і $\parallel$ (а також $\&$ і $\&\&$), які виконують схожі дії, проте вони не рівнозначні.

Вираз $c = a | b$; буде обчислювати спочатку обидва значення - **a** і **b** і на їх основі виводити результат.

А у виразі $c = a \parallel b$; спочатку буде обчислюватися значення **a**, і якщо воно дорівнює **true**, то обчислення значення **b** вже не має сенсу, адже у нас в будь-якому випадку **c** вже дорівнюватиме **true**. Значення **b** буде обчислюватися тільки в тому випадку, якщо **a** дорівнює **false**.

Те ж саме стосується пари операцій $\&$ / $\&\&$. У виразі $c = a \& b$; будуть обчислюватися обидва значення - **a** і **b**.

А у виразі $c = a \&\& b$; спочатку буде обчислюватися значення **a**, і якщо воно дорівнює **false**, то обчислення значення **b** вже не має сенсу, адже значення **c** в будь-якому випадку дорівнюватиме **false**. Значення **b** буде обчислюватися тільки в тому випадку, якщо **a** дорівнює **true**.

Таким чином, операції $\parallel$ і $\&\&$ більш зручні в обчисленнях, дозволяючи скоротити час на обчислення значення виразу і тим самим підвищуючи продуктивність. А операції $|$ і $\&$ більше підходять для виконання порозрядних операцій над числами.

Приклади:

```

1  bool a1 = (5 > 6) || (4 < 6); // 5 > 6 - false, 4 < 6
   - true, тому повертається true
2  bool a2 = (5 > 6) || (4 > 6); // 5 > 6 - false, 4 > 6
   - false, тому повертається false
3  bool a3 = (5 > 6) && (4 < 6); // 5 > 6 - false, 4 < 6
   - true, тому повертається false
4  bool a4 = (50 > 6) && (4 / 2 < 3); // 50 > 6 - true,
   4/2 < 3 - true, тому повертається true
5  bool a5 = (5 > 6) ^ (4 < 6); // 5 > 6 - false, 4 < 6 -
   true, тому повертається true
6  bool a6 = (50 > 6) ^ (4 / 2 < 3); // 50 > 6 - true,
   4/2 < 3 - true, тому повертається false

```

3.8. Операції присвоєння і пріоритет операцій

3.8.1. Операції присвоєння.

На завершення розглянемо операції присвоєння, які в основному є комбінацію простого присвоєння з іншими операціями:

=	// c = b (Просто змінній c присвоюється значення b)
+=	// c += b ; (Змінній c присвоюється результат додавання c і b)
-=	// c -= b ; (Змінній c присвоюється результат віднімання b з c)
*=	// c *= b ; (Змінній c присвоюється результат множення c і b)
/=	// c /= b ; (Змінній c присвоюється результат ділення c на b)
~/=	// c ~/= b ; (Змінній c присвоюється результат цілочисельного ділення c на b)
%=	// c %= b ; (Змінній c присвоюється залишок від ділення c на b)
&=	// c &= b ; (Змінній c присвоюється значення c & b)
=	// c = b ; (Змінній c присвоюється значення c b)
^=	// c ^= b ; (Змінній c присвоюється значення c ^ b)
<<=	// c <<= b ; (Змінній c присвоюється значення c << b)
>>=	// c >>= b ; (Змінній c присвоюється значення c >> b)
??=	// c ??= b ; (Якщо змінна c має значення null , то їй присвоюється значення змінної b , якщо змінна c НЕ дорівнює null , то вона зберігає своє значення).

Приклади застосування операцій присвоєння:

```

1  int a = 5;
2  a += 10;      // 15
3  a -= 3;       // 12
4  a *= 2;       // 24
5  a ~/= 6;      // 4
6  a <<= 4;      // 64
7  a >>= 2;      // 16
8  print(a);     // 16

```

3.8.2. Пріоритет операцій

При роботі з операціями важливо розуміти їх пріоритет, який можна описати таблицею 3.1.

Таблиця 3.1

Пріоритет операцій

expr++ expr--
-expr !expr ~expr ++expr --expr
* / % ~/
+ -
<< >> >>>
&
^
< > <= >= as is is!
== !=
&&
??
? : (тернарний оператор)
.. (spread-оператор)
= += -= *= /= %= ~/= &= ^= = <<= >>= (оператори присвоєння)

3.9. Умовні конструкції

Умовні конструкції дозволяють в залежності від певних умов направити роботу програми по одній із колій.

У мові Dart використовуються наступні умовні конструкції: **if..else** і **switch..case**, а також оператори **?:** та **??**.

3.9.1. Конструкція if / else

Вираз **if/else** перевіряє істинність деякої умови і залежно від результатів перевірки виконує певний код:

```
1  int num1 = 6;
2  int num2 = 4;
3  if(num1>num2) {
4      print("Перше число більше за друге");
5  }
```

Після ключового слова **if** в дужках йде умова. Якщо ця умова виконується і повертає **true**, то спрацьовує код, який розташований далі в блоці **if** після фігурних дужок. В наведеному прикладі в якості умови виступає операція порівняння двох чисел.

Оскільки тут перше число більше за друге, то вираз **num1>num2** вірний і повертає значення **true**. Отже, управління переходить в блок коду після фігурних дужок і починає виконувати інструкції, які там розташовані, а саме метод **print("Перше число більше за друге")**. Якби перше число виявилось б менше за друге або дорівнювало б йому, то інструкції в блоці **if** не виконувалися б.

Але що, якщо необхідно, щоб при недотриманні умови також виконувалися якісь дії? У цьому випадку можна додати блок **else**:

```
1  int num1 = 6;
2  int num2 = 4;
3  if(num1>num2) {
4      print("Перше число більше за друге");
5  }
6  else{
7      print("Перше число менше за друге");
8  }
```

Але при порівнянні чисел можна нарахувати три стани: перше число більше за друге, перше число менше за друге і числа рівні. За допомогою виразу **else if**, можна обробляти додаткові умови:

```
1  int num1 = 6;
2  int num2 = 8;
3  if(num1>num2) {
4      print("Перше число більше за друге");
5  }
```

```

6  else if(num1<num2) {
7      print("Перше число менше за друге");
8  }
9  else{
10     print("Числа дорівнюють одне одному");
11 }

```

Використовуючи логічні оператори, можна об'єднати відразу кілька умов:

```

1  int num1 = 8;
2  int num2 = 6;
3  if(num1 > num2 && num1>7) {
4      print("Перше число більше за друге и більше 7");
5  }

```

Тут блок **if** буде виконуватися, якщо **num1>num2** дорівнює **true** і одночасно **num1>7** дорівнює **true**.

3.9.2. Конструкція switch

Конструкція **switch/case** дозволяє обробити відразу кілька умов:

```

1  int num = 8;
2  switch(num) {
3
4      case 1:
5          print("число дорівнює 1");
6          break;
7      case 8:
8          print("число дорівнює 8");
9          num++;
10         break;
11     case 9:
12         print("число дорівнює 9");
13         break;
14     default:
15         print("число не дорівнює 1, 8, 9");
16 }

```

Після ключового слова **switch** в дужках йде вираз порівняння. Значення цього виразу послідовно порівнюється зі значеннями, розташованими після оператора **case**. Якщо збіг буде знайдено, то буде виконуватися певний блок **case**.

Варто відзначити, що в кінці блоку **case** повинен йти оператор **break**, або один з наступних операторів: **continue**, **throw**, **return**. Якщо в кінці блоку **case** немає жодного з цих операторів, тоді згенерується помилка, як в наступному прикладі:

```
1  case 8:
2      print("число дорівнює 8");
3      num++;  // Помилка ! Відсутній оператор break
4  case 9:
5      print("число дорівнює 9");
6      break;
```

Якщо необхідно також обробити ситуацію, коли збігів не буде знайдено, то можна додати блок **default**, як в прикладі вище. Хоча блок **default** необов'язковий.

Також можна визначити одну дію відразу для декількох блоків **case** поспіль:

```
1  int num = 3;
2  int output = 0;
3  switch(num) {
4
5      case 1:
6          output = 3;
7          break;
8      case 2:
9      case 3:
10     case 4:
11         output = 6;
12         break;
13     case 5:
14         output = 12;
15         break;
16     default:
17         output = 24;
18 }
19 print(output);
```

Замість оператора **break** можна використовувати оператор **continue**, вказавши мітку для переходу:

```

1  int num = 8;
2  switch (num) {
3
4      case 1:
5          print("число дорівнює 1");
6          break;
7      case 8:
8          print("число дорівнює 8");
9          num++;
10         continue N9;      // перехід на мітку N9
11     N9: case 9:
12         print("число дорівнює 9");
13         break;
14     default:
15         print("число не дорівнює 1, 8, 9");
16 }

```

Тут оператор **continue** здійснює перехід на мітку **N9**, яка за змістом є наступним блоком **case**. Назва мітки довільна. В цьому прикладі дія оператора **continue** схожа на дію оператора **goto** в деяких мовах програмування. Тільки тут мітка повинна вести всередину конструкції **switch** на один з блоків **case** або блок **default**.

3.9.3. Тернарна операція

Тернарна операція має наступний синтаксис: **[перший операнд - умова]? [Другий операнд]: [третій операнд]**. Таким чином, в цій операції беруть участь відразу три операнда. Залежно від умови тернарна операція повертає другий або третій операнд: якщо умова дорівнює **true**, то повертається другий операнд; якщо умова дорівнює **false**, то третій. Розглянемо приклад:

```

1  int x=3;
2  int y=2;
3  int z = x<y? (x+y) : (x-y);
4  print(z);

```

Тут результатом тернарної операції є змінна **z**. Спочатку перевіряється умова **x<y**. І якщо вона вірна, то **z** дорівнюватиме другому операнду - **(x+y)**, інакше **z** дорівнюватиме третьому операнду.

3.10. Цикли

Цикли дозволяють в залежності від певних умов виконувати певну дію багато разів. У мові Dart є такі види циклів:

for;
while;
do ... while.

3.10.1. Цикл for

Цикл **for** має наступне формальне визначення:

```
1  for ([ініціалізація лічильника]; [умова]; [зміна  
    лічильника])  
2  {  
3      // дії  
4  }
```

Розглянемо стандартний цикл **for**:

```
1  for (int i = 1; i < 9; i++) {  
2      print("Квадрат числа $i дорівнює ${i * i} \n");  
3  }
```

Перша частина оголошення циклу - **int i = 1** створює і ініціалізує лічильник **i**. Тобто перед виконанням циклу значення лічильника буде дорівнювати **1**. В цьому прикладі це те ж саме, що і оголошення змінної.

Друга частина - умова, за якої буде виконуватися цикл. В наведеному прикладі цикл буде виконуватися, поки **i** не досягне 9.

І третя частина - приріст лічильника на одиницю. При цьому необов'язково збільшувати на одиницю. Можна і зменшувати: **i--**.

В результаті блок циклу спрацює 8 разів, поки значення **i** не стане дорівнювати 9. І кожен раз це значення буде збільшуватися на 1.

Вказувати всі умови при оголошенні циклу необов'язково. Можна писати так, як наведено в наступному прикладі:

```
1  int i = 1;  
2  for (; ;){  
3      print("Квадрат числа $i дорівнює ${i * i} \n");  
4  }
```

Визначення циклу залишилося тим самим, тільки тепер блоки у визначенні порожні: **for (; ;)**. Тепер немає ініціалізованої змінної-лічильника, немає умови, тому цикл буде працювати вічно - нескінченний цикл.

Або можна опустити кілька блоків:

```

1  int i = 1;
2  for (; i<9;){
3      print("Квадрат числа $i дорівнює ${i * i} \n");
4      i++;
5  }

```

Цей приклад еквівалентний першому прикладу: тут є лічильник, тільки створений він поза циклом. Тут є умова виконання циклу. І є приріст лічильника вже в самому блоці **for**.

Цикл **for** може визначати відразу кілька змінних і управляти ними:

```

1  int n = 10;
2  for(int i=0, j = n - 1; i < j; i++, j--){
3
4      print(i * j);
5  }

```

3.10.2. Цикл do

Цикл **do** спочатку виконує код циклу, а потім перевіряє умову в інструкції **while**. І поки ця умова вірна, цикл повторюється. Розглянемо наступний приклад:

```

1  int j = 7;
2  do{
3      print(j);
4      j--;
5  }
6  while (j > 0);

```

В наведеному прикладі код циклу спрацює 7 разів, поки **j** не дорівнюватиме нулю. Важливо відзначити, що цикл **do** гарантує хоча б одноразове виконання дії, навіть якщо умова в інструкції **while** не буде вірна. Отже, можна написати так:

```

1  int j = -1;
2  do{
3      print(j);
4      j--;
5  }
6  while (j > 0);

```

Хоча змінна **j** спочатку менше 0, цикл все одно один раз виконається.

3.10.3. Цикл **while**

Цикл **while** відразу перевіряє вірність деякої умови, і якщо умова вірна, то код циклу виконується:

```
1  int j = 6;
2  while (j > 0) {
3
4      print(j);
5      j--;
6  }
```

3.10.4. Оператори **continue** і **break**

Оператор **break** дозволяє вийти з циклу в будь-який його момент, навіть якщо цикл не закінчив свою роботу:

Наприклад:

```
1  for (int i = 0; i < 10; i++) {
2      if (i == 5)
3          break;
4      print(i);
5  }
```

Коли лічильник дорівнюватиме 5, спрацює оператор **break**, і цикл завершиться.

Тепер зробимо так, щоб якщо число дорівнює 5, цикл не завершувався, а просто переходив до наступної ітерації. Для цього застосовується оператор **continue**:

```
1  for (int i = 0; i < 10; i++) {
2      if (i == 5)
3          continue;
4      print(i);
5  }
```

Тут, коли значення лічильника дійде до числа 5, яке не задовольняє умові перевірки, програма просто пропустить це число і перейде до наступної ітерації.

Резюме

В цьому розділі було розглянуто основи мови Dart, а саме: структуру програми; змінні та константи; примітивні типи даних; арифметичні, побітові операції та операції присвоєння.

Розглянуто умовні вирази, умовні конструкції та цикли, які підтримуються мовою Dart.

Показано яким чином ці елементи застосовуються при написанні програм.

Контрольні запитання і завдання

1. Яку структуру мають програми в мові Dart?
2. Яке призначення та зміст слова **void**?
3. Що таке **Інструкції (statement)**?
4. Що таке блок коду?
5. Як в Dart позначаються коментарі?
6. Як в Dart позначаються змінні і константи?
7. Які є способи визначення змінної в Dart?
8. Які є способи визначення константи в Dart?
9. Що таке інтерполяція в Dart?
10. Які примітивні типи даних є у Dart?
11. Який зміст вкладається в значення поняття **null**, **nullable-типи** і **null-безпека**?
12. Як в Dart визначити тип як **nullable**?
13. Навіщо потрібен оператор **??**?
14. Які арифметичні операції підтримує Dart?
15. Які побітові операції підтримує Dart?
16. Які умовні вирази підтримує Dart?
17. Які умовні конструкції підтримує Dart?
18. Які види циклів підтримує Dart?
19. Наведіть приклади інструкцій.
20. Наведіть приклади інструкцій.
21. Наведіть приклади блоку коду.
22. Наведіть приклади застосування коментарів.
23. Наведіть приклади визначення змінної.
24. Наведіть приклади визначення константи.
25. Наведіть приклади використання інтерполяції.
26. Наведіть приклади використання типу **Runes**.
27. Наведіть приклади використання типу **Symbol**.
28. Наведіть приклади застосування арифметичних операцій.
29. Наведіть приклади застосування побітових операцій.
30. Наведіть приклади застосування логічних операцій.
31. Наведіть приклади застосування умовних виразів.

32. Наведіть приклади застосування умовних конструкцій.

33. Наведіть приклади застосування циклів.

34. Які змінні в коді, що наведено нижче, визначені неправильно:

```
1  var name1 = "Bob";
2  String name2 = "Sam";
3  dynamic name3 = "Tom";
4  var name4: String = "Mike";
5  name5 = "Alex";
```

35. Що буде виведено на консоль при виконанні наступної програми:

```
1  void main () {
2
3      var name = "Tom";
4      name = "45";
5      print(name);
6  }
```

36. Що буде виведено на консоль при виконання наступної програми:

```
1  void main () {
2
3      const name = "45";
4      name = "Tom";
5      print(name);
6  }
```

37. Які константи в наступному коді визначено неправильно?

```
1  void main () {
2
3      const tom = "Tom";
4      String const kate = "Kate";
5      const String sam = "Sam";
6      final String bob = "Bob";
7      final mike = "Mike";
8      String final alex = "Alex";
9  }
```

38. Якщо змінній не присвоєно ніякого значення, яке значення вона має за замовчуванням (наприклад, в наступному коді)?

```
1 void main () {  
2  
3     String name;  
4 }
```

39. Що з нижче перерахованого не є примітивним типом даних в мові Dart:

- **string;**
- **int;**
- **float;**
- **double;**
- **char;**
- **runes;**
- **boolean.**

40. Чи можна змінити тип даних у змінній в Dart?

41. Які з наступних змінних визначені некоректно і чому?

```
1 String id = 123;  
2 var isAlive = true;  
3 double height = 45;  
4 bool isValid = 1;  
5 var weight = 67.5;  
6 var sam = 'Sam';
```

42. Що виведе на консоль наступна програма і чому:

```
1 void main() {  
2     int a = 6;  
3     int b = 4;  
4     int c = a ~/ b;  
5     print(c);  
6 }
```

43.Що виведе на консоль наступна програма і чому:

```
1 void main() {
2     int a = 4;
3     int b = 5;
4     int c = a + 5 * b++;
5     print(b);
6     print(c);
7 }
```

44.Що виведе на консоль наступна програма:

```
1 void main() {
2     int a = 4;
3     int b = 4;
4
5     int c = ++a * 5 + a;
6     int d = b + 5 * ++b;
7
8     print(c);
9     print(d);
10 }
```

45.Чи обов'язково використовувати блок **default** в конструкції **switch..case** в мові Dart?

46.Що виведе на консоль наступна програма і чому:

```
1 void main() {
2     int num;
3     switch(num) {
4
5         case 0:
6             print("число дорівнює 0");
7             break;
8         case 1:
9             print("число дорівнює 1");
10            num++;
11     }
```

```

11         break;
12     case 2:
13         print("число дорівнює 2");
14         break;
15     }
16 }

```

47. Блок **case** в конструкції **switch...case** обов'язково повинен містити оператор **break**?

48. Який результат буде при виконанні наступної програми:

```

1 void main() {
2     int num1;
3     int num2 = 8;
4     if(num1 > num2){
5         print("Перше число більше за друге");
6     }
7     else if(num1 < num2){
8         print("Перше число менше за друге");
9     }
10    else if(num1 == num2){
11        print("Числа дорівнюють одне одному");
12    }
13    else{
14        print("Невизначений результат");
15    }
16 }

```

49. Що буде виведено на консоль при виконанні наступної програми:

```

1 void main() {
2     int num1 = 5;
3     int num2 = 8;
4     int num3 = num1 ?? num2;
5     print(num3);
6 }

```


50.Що буде виведено на консоль при виконанні наступної програми:

```
1 void main() {
2     int num1;
3     int num2;
4     int num3 = 8;
5
6     num3 = num1 ?? num2;
7     print(num3);
8 }
```

51.Скільки разів буде виконуватися наступний цикл?

```
1 for (int i = 0; i <= 9; i++){
2     print("Квадрат числа $i дорівнює ${i *i} \n");
3 }
```

52.Скільки разів буде виконуватися наступний цикл?

```
1 int counter = -1;
2 do{
3     print(counter);
4     counter++;
5 }
6 while(counter > 1);
```

53.Скільки разів буде виконуватися цикл в наступній програмі і що вона виведе на консоль?

```
1 void main() {
2     int counter = 0;
3
4     while(true) {
5
6         if(counter%2==0) {
7             print(counter);
8         }
9         counter++;
10    }
```

```
10
11     if(counter == 10) break;
12 }
13 }
```

4. ФУНКЦІОНАЛЬНЕ ПРОГРАМУВАННЯ

4.1. Визначення функцій

Функціями є деякі дії. Можна оформити набір операторів у вигляді однієї функції і багаторазово викликати її в програмі. Формальне визначення функції:

```
1  [тип] ім'я_функції (параметри) {  
2  
3      Виконувані_інструкції  
4  }
```

Спочатку вказується тип функції, який повертається. Якщо функція не повертає ніякого значення, то використовується тип **void**.

Потім йде ім'я функції, яке є довільною назвою, до якого застосовуються ті ж правила, що і до іменування змінних.

Після імені функції в дужках йде перерахування параметрів. Функція може не мати параметрів, в цьому випадку вказуються порожні дужки.

Далі в фігурних дужках йде тіло функції, яке містить виконувані інструкції.

Наприклад, визначимо функцію, яку назовемо **hello**:

```
1  void main() {  
2  }  
3  void hello() {  
4  
5      print("Hello!");  
6  }
```

Функція **hello** не повертає ніякого результату, а просто виконує деякі дії, тому функція має тип **void**. Все, що вона робить - це виводить рядок «Hello!».

Але сама по собі функція не виконуватиметься. За замовчуванням середовище виконує тільки функцію **main**. Тому, щоб виконати функцію **hello**, її необхідно викликати всередині функції **main**. Виклик функції здійснюється у формі:

```
1  ім'я_функції (аргументи) ;
```

Після імені функції вказуються дужки, в яких перераховуються аргументи - значення для параметрів функції. Наприклад, викликаємо функцію **hello** два рази:

```

1 void main() {
2
3     hello();
4     hello();
5 }
6 void hello() {
7
8     print("Hello!");
9 }

```

Якщо функція складається з одного виразу, то її можна скоротити в такий спосіб:

```

1 [тип] ім'я_функції(параметри) =>
   Виконувати_інструкції

```

В наведеному прикладі після списку параметрів в дужках йде =>, а потім виконуваний вираз. Наприклад, наведена раніше функція **hello** фактично виконувала тільки один вираз, тому ми можемо її скоротити:

```

1 void main() {
2
3     hello();
4     hello();
5 }
6 void hello() => print("Hello!");

```

4.2. Параметри функції

Функція може приймати параметри. Параметри перераховуються в дужках після імені функції та мають таке визначення:

```

1 тип назва_параметра

```

Наприклад, визначимо функцію, яка складає два числа:

```

1 void main() {
2
3     sum(2, 5); // 7
4 }
5 void sum(int a, int b) {
6     int c = a + b;
7     print(c);
8 }

```

Функція **sum** приймає два параметри, які мають імена **a** і **b** та мають тип **int**. У функції **main** викликаємо функцію **sum** і передаємо їй два аргументи. Аргументи - це значення для параметрів функції. Аргументи передаються параметрам за позицією. Тобто перший аргумент передається першому параметру, другий аргумент - другому параметру і так далі. В наведеному прикладі параметр **a** як значення отримає число 2, а параметр **b** - число 5.

Кількість переданих аргументів повинна відповідати кількості параметрів. Крім того між типами аргументів і типами параметрів має бути відповідність. Наприклад, визначимо ще одну функцію:

```
1 void main() {
2
3     showPerson("Tom", 35);
4     showPerson("Alice", 29);
5 }
6 void showPerson(String name, int age){
7     print("Name: $name");
8     print("Age: $age \n");
9 }
```

Функція **showPerson** приймає два параметри. Перший параметр має тип **String**, а другий має тип **int**. Тому при виконанні функції необхідно спочатку передати значення типу **String** (тобто рядок), а потім значення типу **int** (тобто ціле число).

При визначенні функції можна не вказувати тип параметрів. Тоді Dart сам визначає їх тип виходячи з переданих значень:

```
1 void main() {
2
3     showPerson("Tom", 35);
4     showPerson(29, "Alice");
5 }
6 void showPerson(name, age){
7     print("Name: $name");
8     print("Age: $age \n");
9 }
```

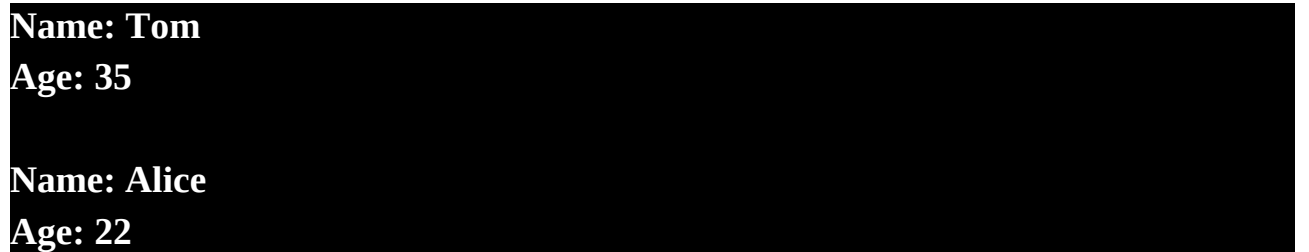
Тут варто звернути увагу, що тепер при визначенні функції невідомо, який саме тип мають параметри **name** і **age**. Тому в другому виклику функції **showPerson (29, "Alice")** Dart буде трактувати параметр **name** як параметр типу **int**, тому що йому передається ціле число. А параметр **age** буде трактуватися як рядок.

4.2.1. Необов'язкові параметри

Деякі параметри можна зробити необов'язковими, тобто можна не передавати для них ніяких значень. Для цього параметр розташовується в квадратні дужки. Але при цьому йому необхідно надати значення за замовчуванням, яке буде використовуватися, якщо параметру не передано жодного значення:

```
1 void main() {
2
3     showPerson("Tom", 35);
4     showPerson("Alice");
5 }
6 void showPerson(String name, [int age = 22]){
7     print("Name: $name");
8     print("Age: $age \n");
9 }
```

В наведеному прикладі параметр **age** є необов'язковим. Якщо не передавати йому ніякого значення, то він за замовчуванням буде мати значення 22. А на консоль буде виведено результат, який показано на рисунку 4.1.



```
Name: Tom
Age: 35

Name: Alice
Age: 22
```

Рисунок 4.1. Результат роботи програми

Однак якщо необов'язковий параметр має **nullable-тип**, тобто може приймати значення **null**, тоді можна не вказувати для нього значення за замовчуванням - в цьому випадку значення за замовчуванням буде **null**:

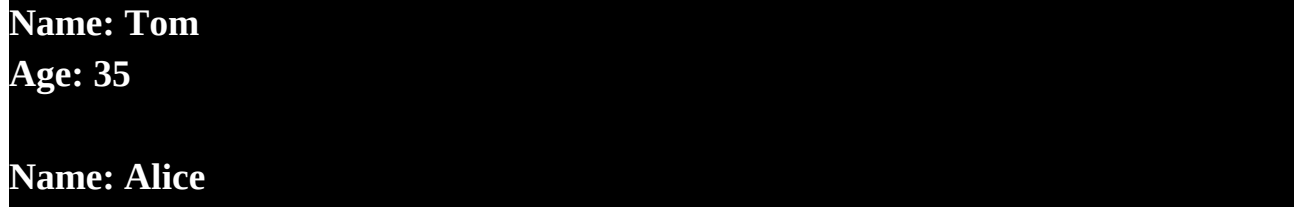
```
1 void main() {
2
3     showPerson("Tom", 35);
4     showPerson("Alice");
5 }
6 void showPerson(String name, [int? age]){
7     print("Name: $name");
```

```

8      if (age!=null) {
9          print("Age: $age \n");
10     }
11 }

```

Результат роботи програми показано на рисунку 4.2.



```

Name: Tom
Age: 35

Name: Alice

```

Рисунок 4.2. Результат роботи програми

4.2.2. Іменовані параметри

Dart підтримує передачу параметрів за іменем. Розглянемо наступний приклад:

```

1  void main() {
2
3      showPerson(name: "Tom", age: 35);
4      showPerson(age: 29, name: "Alice");
5      showPerson(name: "Kate");
6  }
7  void showPerson({String name = "undefined", int
   age=0}){
8      print("Name: $name");
9      print("Age: $age \n");
10 }

```

При визначенні функції список параметрів розташовується у фігурних дужках, при цьому для кожного параметра, якщо його тип не підтримує значення **null**, вказується значення за замовчуванням.

При виконанні функції вказується ім'я параметра і через двокрапку його значення (**name: "Tom"**), при цьому можна вказувати параметри в будь-якій послідовності. Наприклад, у другому виклику спочатку вказується значення для параметра **age**, а потім для параметра **name** (**showPerson (age: 29, name: "Alice")**). Більш того, можна взагалі не передавати параметрам значення, тоді такий параметр буде мати значення за замовчуванням, як у випадку з параметром **age** в третьому виклику (**showPerson (name: "Kate")**).

Результат роботи програми показано на рисунку 4.3.

Name: Tom

Age: 35

Name: Alice

Age: 29

Name: Kate

Age: 0

Рисунок 4.3. Результат роботи програми

Якщо параметр має **nullable-тип**, тобто підтримує значення **null**, то можна не вказувати значення за замовчуванням, тоді значенням за замовчуванням буде **null**:

```
1 void main() {
2
3     showPerson(age: 35);
4     showPerson(age: 29, name: "Alice");
5     showPerson(name: "Kate");
6 }
7 void showPerson({String? name = "undefined", int?
  age}) {
8     print("Name: $name");
9     if (age != null) {
10         print("Age: $age \n");
11     }
12 }
```

Результат роботи програми показано на рисунку 4.4.

Name: undefined

Age: 35

Name: Alice

Age: 29

Name: Kate

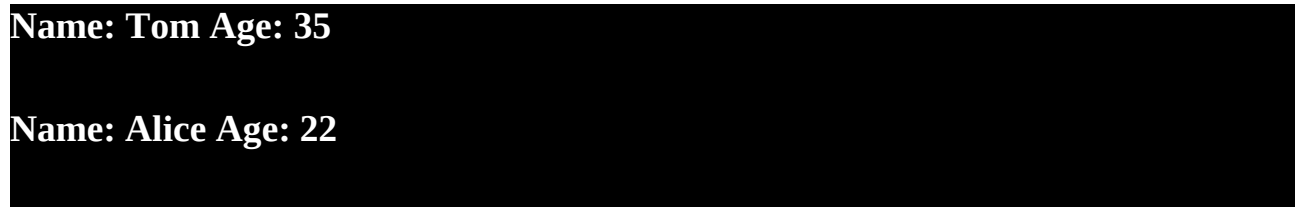
Рисунок 4.4. Результат роботи програми

4.2.3. Константи у функціях

Параметри дозволяють передати значення локальних змінних, які визначені у функціях. Але також можна передавати через параметри значення локальним **final-константам**, оскільки вони визначаються під час виконання. Але при цьому не можна передавати значення **const-констант**, так як їх значення повинні бути відомі вже під час компіляції. У зв'язку з чим, при різних викликах функції, можна передати **final-константам** різні значення:

```
1 void main() {
2
3     showPerson("Tom", 35);
4     showPerson("Alice", 22);
5 }
6 void showPerson(String userName, int userAge) {
7     // const int initialAge = userAge; - так неможна,
    константа повинна визначатися на етапі компіляції
8     final String name = userName;
9     final int age = userAge;
10    print("Name: $name    Age: $age \n");
11 }
```

Результат роботи програми показано на рисунку 4.5.



Name: Tom Age: 35

Name: Alice Age: 22

Рисунок 4.5. Результат роботи програми

4.3. Повернення результату з функції

Функція може повертати певне значення. Для цього використовується оператор **return**. Розглянемо наступний приклад, де з функції повертається сума двох чисел:

```
1 int sum(int a, int b) {
2     return a + b;
3 }
```

Тип значення функції, що повертається, зазначається перед ім'ям функції. Тобто, в наведеному прикладі функція повертає значення типу **int**. Після

оператора **return** в тілі функції вказується яке саме значення буде повернуто. В наведеному прикладі - це результат виразу **(a + b)**. Причому тип значення після оператора **return** повинен відповідати типу функції. В нашому прикладі функція має тип **int** і повертає суму **a + b**, яка теж має тип **int**.

Крім того, оператор **return** завершує виконання функції, тому всі інші інструкції у функції повинні розташовуватись до оператора **return**.

Результат функції можна присвоїти змінній і потім використовувати в програмі:

```
1 void main() {
2
3     int c = sum(2, 5);
4     print(c);    // 7
5 }
6 int sum(int a, int b){
7     return a + b;
8 }
```

Тут результат функції **sum**, тобто сума двох чисел присвоюється змінній **c**.

При визначенні функції можна не вказувати тип результату, що повертається. Тоді він буде з'ясований із значення, що отримується після оператора **return**:

```
1 void main() {
2
3     var n = sum(2, 5);
4     print(n);
5 }
6
7 // функція має тип int
8 sum(int a, int b){
9     return a + b;
10 }
```

Якщо функція не має оператора **return**, то в тіло функції неявно додається інструкція **return null**. Наприклад, функція:

```
1 sum(int a, int b){
2     print(a + b);
3 }
```

Фактично буде еквівалентна:

```

1  sum(int a, int b){
2      print(a + b);
3      return null;
4  }

```

Якщо функція не повинна повертати ніякого значення, то функція має тип **void**. Причому навіть можна визначити змінну типу **void** і присвоїти їй результат **void-функції**, але нічого більше зробити з **void-об'єктом** не можна:

```

1  void main() {
2
3      void n = hello();
4      // оскільки змінна n має тип void, то ні чого
5      // не можна з нею робити, навіть вивести на
6      // консоль print(n);
7  }
8
9  void hello(){
10     print("Hello!");
11     return null;
12 }

```

4.4. Функція як об'єкт

Будь-яка функція в мові Dart має тип **Function** і фактично може виступати в якості окремого об'єкта. Наприклад, можна визначити об'єкт функції, динамічно присвоїти йому посилання на якусь функцію і викликати її. Розглянемо наступний приклад:

```

1  void main() {
2
3      Function func = hello;
4      func();      // Hello!
5      func = bye;
6      func();      // Goodbye!
7  }
8  void hello(){
9      print("Hello!");
10 }
11 void bye(){
12

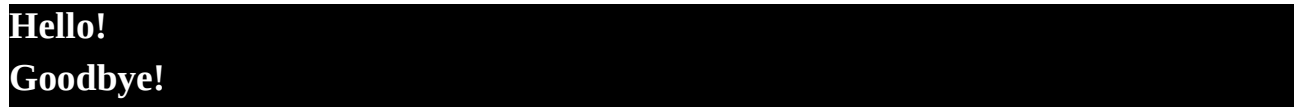
```

```

13     print("Goodbye!");
14 }

```

В наведеному прикладі визначається змінна типу **Function** - **func**, і їй присвоюється посилання на функцію **hello**. Потім можна використати змінну **func** як звичайну функцію, фактично викликаючи ту функцію, на яку ця змінна посилається. Крім того, можна динамічно змінити функцію, на яку посилається змінна. Результат роботи програми показано на рисунку 4.6.



```

Hello!
Goodbye!

```

Рисунок 4.6. Результат роботи програми

Також функції можуть виступати в якості параметрів інших функцій:

```

1  void main() {
2
3      showMessage(hello);
4      showMessage(bye);
5  }
6  void showMessage(Function func) {
7      func();          // викликаємо передану функцію
8  }
9  void hello() {
10     print("Hello!");
11 }
12 void bye() {
13
14     print("Goodbye!");
15 }

```

Тут у функцію **showMessage** як параметр передаються функції **hello** і **bye**. У самій **showMessage** потім можна викликати передані функції.

Також функція може виступати в якості значення, що повертається:

```

1  void main() {
2
3      Function message = getMessage(11);
4      message();
5      message = getMessage(15);

```

```

6      message();
7  }
8
9  Function getMessage(int hour){
10      if(hour < 12) return morning;
11      else return evening;
12 }
13 void morning(){
14     print("Good morning!");
15 }
16 void evening(){
17
18     print("Good evening!");
19 }

```

В наведеному прикладі функція **getMessage** в залежності від переданого числа повертає або функцію **morning**, або функцію **evening**.

Потім в програмі можна отримати функцію і викликати її:

```

1  Function message = getMessage(11);
2  message();

```

4.5. Анонімні і вкладені функції

4.5.1. Анонімні функції

Анонімні функції схожі на звичайні функції за тим винятком, що вони не мають назви. Розглянемо наступний приклад:

```

1  void main() {
2
3      Function operation = (a, b) => a + b;
4      print(operation(4, 5));
5  }

```

В цьому прикладі змінній **operation** присвоюється посилання на анонімну функцію **(a, b) => a + b**, яка приймає два параметри і повертає суму їх значень.

В наведеному прикладі анонімна функція має скорочений запис, але також можна його розгорнути:

```

1  void main() {
2
3      Function operation = (int a, int b)
4      {
5          print("Sum of $a and $b");
6          return a + b;
7      };
8      print(operation(4, 5));
9  }

```

Розглянемо, де має сенс використовувати анонімні функції. Якщо викликати функцію, яка приймає іншу функцію, то для функції-параметра може бути простіше використовувати анонімну функцію, ніж визначати іменовану, особливо якщо іменована функція далі більше ніде не знадобиться:

```

1  void main() {
2
3      int result = getResult(3, 5, (a,b)=>a+b);
4      print(result);           // 8
5      result = getResult(3, 5, (a,b)=>a-b);
6      print(result);           // -2
7  }
8  int getResult(int a, int b, Function operation){
9
10     return operation(a, b);
11 }

```

4.5.2. Вкладені функції

В мові Dart функції можуть бути вкладеними в інші функції:

```

1  void main() {
2
3      void hello(){
4          print("Hello!");
5      }
6
7      hello();
8      hello();
9  }

```

Причому визначення вкладеної функції повинно передувати її виклику. Вкладені функції мають сенс, якщо планується використовувати якісь повторювані дії тільки всередині певної функції.

Крім того, вкладені функції самі можуть містити інші вкладені функції:

```
1 void main() {
2
3     void showMessage() {
4         void hello() {
5             print("Hello!");
6         }
7
8         hello();
9         hello();
10    }
11
12    showMessage();
13 }
```

Варто відзначити, що вкладені функції утворюють власний контекст - змінні і константи, до яких зовнішня функція не може звернутися. Розглянемо наступний приклад:

```
1 void main() {
2
3     void welcome() {
4         var message = "Welcome!";
5         print(message);
6     }
7
8     print(message); // ! Помилка - зовнішня функція не
    може звертатися до змінних вкладених функцій.
9 }
```

В наведеному прикладі змінна **message** визначена всередині функції **welcome**, тому може викликатися тільки в рамках цієї функції, але не поза нею.

4.6. Замикання

Замикання (closure) є об'єктом функції, який запам'ятовує своє лексичне оточення навіть у тому випадку, коли вона виконується поза своєю областю видимості.

Технічно замикання включає три компоненти:

- зовнішня функція, яка визначає деяку область видимості і в якій визначені деякі змінні і параметри - лексичне оточення;
- змінні і параметри (лексичне оточення), які визначені у зовнішній функції;
- вкладена функція, яка використовує змінні і параметри зовнішньої функції.

```
1  Function outer(){           // зовнішня функція
2      var n;                  // деяка змінна - лексичне
    оточення
3      Function inner(){       // вкладена функція
4          // операції із змінною n
5      }
6      return inner;
7  }
```

Розглянемо замикання на простому прикладі:

```
1  Function outer(){           // зовнішня функція
2      var n = 5;              // деяка змінна - лексичне
    оточення функції inner
3      void inner(){           // вкладена функція
4          // операції із змінною n
5          n++;
6          print(n);
7      }
8      return inner;
9  }
10 void main() {
11
12     Function fn = outer();    // fn = inner, так як
    функція outer повертає функцію inner
13     // викликаємо внутрішню функцію inner
14     fn();                     // 6
15     fn();                     // 7
16     fn();                     // 8
17 }
```

Тут функція **outer** задає область видимості, в якій визначені внутрішня функція **inner** і змінна **n**. Змінна **n** відображає лексичне оточення для функції

inner. У самій функції **inner** інкрементуємо змінну **n** і виводимо її значення на консоль. В кінці функція **outer** повертає функцію **inner**.

Далі викликається функцію **outer**:

```
1  Function fn = outer();
```

Оскільки функція **outer** повертає функцію **inner**, то змінна **fn** буде зберігати посилання на функцію **inner**. При цьому ця функція запам'ятала своє оточення - тобто зовнішню змінну **n**.

Далі фактично три рази викликається функцію **inner**, і бачимо, що змінна **n**, яка визначена поза функцією **inner**, збільшується на одиницю:

```
1  fn();    // 6
2  fn();    // 7
3  fn();    // 8
```

Тобто незважаючи на те, що змінна **n** визначена поза функцією **inner**, ця функція запам'ятала своє оточення і може його використовувати, не дивлячись на те, що вона викликається поза функцією **outer**, в якій була визначена. В цьому і суть замикань.

До речі, можна скоротити визначення функції **outer**, використовуючи анонімну функцію:

```
1  Function outer() {
2      var n = 5;
3      return () {
4          n++;
5          print(n);
6      };
7  }
```

Розглянемо ще один приклад:

```
1  Function multiply(int n) {
2
3      return (int m) => n * m;
4  }
5  void main() {
6
7      Function func = multiply(5);
8      int result1 = func(6); // 30
9      print(result1);       // 30
10
12     print(result2);       // 25
13 }
```

Отже, тут виклик функції **multiply()** призводить до виклику іншої внутрішньої функції. Внутрішня ж функція:

```
1 (int m) => n * m;
```

запам'ятовує оточення, в якому вона була створена, зокрема, значення параметра **n**.

В результаті при виконанні функції **multiply** визначається змінна **func**, яка і являє собою замикання, тобто об'єднує дві речі: функцію і оточення, в якому функція була створена. Оточення складається з будь-якої локальної змінної або будь-якого параметра, які були визначені в області дії функції **multiply** під час створення замикання.

Тобто в цьому прикладі **result1** - це замикання, яке містить і внутрішню функцію **(int m) => n * m** ;, і параметр **n**, який існував під час створення замикання.

При цьому важливо не заплутатися у параметрах. При визначенні замикання:

```
1 Function func = multiply(5);
```

Число 5 передається для параметра **n** функції **multiply**.

При виклику внутрішньої функції:

```
1 int result1 = func(6); // 30
```

Число 6 передається для параметра **m** у внутрішню функцію **(int m) => n * m**;

Також можна використовувати інший спосіб для виклику замикання:

```
1 Function multiply(int n){
2     return (int m) => n * m;;
3 }
4 void main() {
5
6     int result1 = multiply(5)(6); // 30
7     print(result1);               // 30
8 }
```

Резюме

В цьому розділі було розглянуто питання використання можливостей Dart для функціонального програмування.

Розглянуто способи визначення та використання функцій, їх параметрів та отримання результатів, що повертаються функціями.

Показано яким чином визначаються та використовуються іменовані та необов'язкові параметри, анонімні та вкладені функції, а також замикання при написанні програм.

Контрольні запитання і завдання

1. Як у Dart формально визначається функція?
2. Що таке параметри функції? Як вони позначаються в Dart?
3. Чи підтримує Dart передачу параметрів за іменем?
4. Як у Dart використовуються константи у функціях?
5. Який оператор використовується для повернення результату функції?
6. Поясніть зміст поняття «функція як об'єкт».
7. Що таке анонімні і вкладені функції?
8. Що таке замикання? Для чого воно використовується?
9. Наведіть приклади передачі параметрів функції.
10. Наведіть приклади застосування анонімних та вкладених функцій
11. Наведіть приклади застосування анонімних та вкладених функцій.
12. Наведіть приклади застосування замикань.
13. Який буде результат виконання наступної програми і чому?

```
1 void main() {
2
3     void showMessage() {
4         String message = "Hello World!";
5         print(message);
6     }
7
8     showMessage();
9     print(message);
10 }
```

5. ОБ'ЄКТНО-ОРІЄНТОВАНЕ ПРОГРАМУВАННЯ

5.1. Класи і об'єкти

Dart є об'єктно-орієнтованою мовою, і кожне значення, над яким проводяться маніпуляції в програмі на Dart, є об'єктом.

Об'єкт є **екземпляром** деякого класу, а **клас** є шаблоном або описом об'єкта. Можна ще провести наступну аналогію. У всіх є деяке уявлення про людину - наявність двох рук, двох ніг, голови, тулуба і т.і. Є деякий шаблон - цей шаблон можна назвати класом. Реально ж існуюча людина (фактично екземпляр даного класу) є об'єктом цього класу.

Клас позначається за допомогою ключового слова **class**:

```
1  class Person{
2
3  }
```

Тут визначено клас **Person**. Після назви класу йдуть фігурні дужки, між якими міститься тіло класу - тобто його поля і методи.

Будь-який об'єкт може володіти двома основними характеристиками: стан - деякі дані, які зберігає об'єкт, і поведінка - дії, які може здійснювати об'єкт.

Для зберігання стану об'єкта в класі застосовуються поля або змінні класу. Для визначення поведінки об'єкта в класі застосовуються методи. Наприклад, клас **Person**, який відображає людину, міг би мати таке визначення:

```
1  class Person{
2
3      String name = "undefined";           // ім'я
4      int age = 0;                          // вік
5      void display() {
6          print("Name: $name Age: $age");
7      }
8  }
```

У класі **Person** визначені два поля: **name**, яке позначає ім'я людини, та **age** - її вік. І також визначено метод **display**, який нічого не повертає і просто виводить ці дані на консоль.

Варто враховувати, що оскільки змінні **name** і **age** мають типи **String** і **int**, які не допускають значення **null**, то перед використанням спочатку необхідно присвоїти цим змінним початкові значення. Або можна використовувати **nullable-типи**, тоді присвоєння початкових значень є необов'язковим:

```

1  class Person{
2
3      String? name;          // ім'я
4      int? age;              // вік
5      void display() {
6          print("Name: $name Age: $age");
7      }
8  }

```

Тепер використаємо створений вище клас. Для цього визначимо наступну програму:

```

1  void main () {
2
3      Person tom;
4  }
5
6  class Person{
7
8      String name = "undefined";    // ім'я
9      int age = 0;                  // вік
10     void display() {
11         print("Name: $name Age: $age");
12     }
13 }

```

Клас відображає новий тип, тому можна визначати змінні, які мають даний тип. Так, тут у функції **main** визначена змінна **tom**, яка є представником класу **Person**. Але поки ця змінна не вказує ні на який об'єкт і за замовчуванням вона має значення **null**. За великим рахунком цю змінну поки ще не можна використовувати, тому спочатку необхідно створити об'єкт класу **Person**.

5.1.1. Конструктори

Крім звичайних методів, класи можуть визначати спеціальні методи, які називаються конструкторами. Конструктори викликаються при створенні нового об'єкта даного класу. Конструктори виконують ініціалізацію об'єкта.

Якщо в класі не визначено жодного конструктора, то для цього класу автоматично створюється конструктор без параметрів.

Вище визначений клас **Person** не має ніяких конструкторів. Тому для нього автоматично створюється конструктор за замовчуванням, який можна використовувати для створення об'єкта **Person**. Зокрема, створимо один об'єкт:

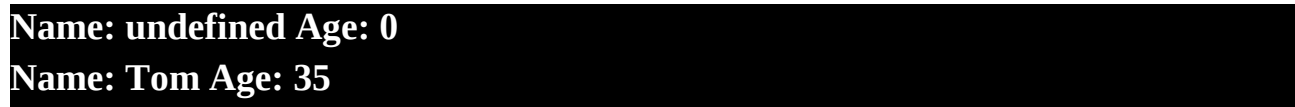
```
1 void main () {
2
3     Person tom = Person();
4     tom.display();
5     // змінюємо ім'я та вік
6     tom.name = "Tom";
7     tom.age = 35;
8     tom.display();
9 }
10
11 class Person{
12
13     String name = "undefined";
14     int age = 0;
15     void display(){
16         print("Name: $name Age: $age");
17     }
18 }
```

Для створення об'єкта **Person** використовується вираз **Person()**. Більш старі версії Dart для виклику конструктора також використовували оператор **new**: **Person tom = new Person();**. Але в останніх версіях Dart оператор **new** можна не використовувати.

Конструктор за замовчуванням не приймає ніяких параметрів. У підсумку після виконання цього виразу в пам'яті буде виділено ділянку, де будуть зберігатися всі дані об'єкта **Person**. А змінна **tom** отримає посилання на створений об'єкт.

Якщо конструктор не ініціює значення змінних об'єкта, то вони отримують значення за замовчуванням, тобто значення **null** (тобто фактично відсутність значення).

Після створення об'єкта можна звернутися до змінних і методів об'єкта **Person** через змінну **tom**. Для цього використовується оператор точка (.) - тобто через точку вказується назва поля або методу: **tom.name**. Наприклад, можна встановити або отримати значення полів: **tom.name = "Tom"**. У підсумку отримаємо результат роботи програми, як показано на рисунку 5.1.



```
Name: undefined Age: 0
Name: Tom Age: 35
```

Рисунок 5.1. Результат роботи програми

Якщо необхідно, щоб при створенні об'єкта виконувалася якась логіка, наприклад, щоб поля класу отримували якісь певні значення, то можна визначити в класі свої конструктори. Розглянемо приклад наведений нижче:

```
1  void main () {
2
3      Person sam = Person("Sam", 25);
4      sam.display();
5  }
6
7  class Person{
8
9      String name = "undefined";
10     int age = 0;
11     Person(String n, int a)
12     {
13         name = n;
14         age = a;
15     }
16     void display() {
17         print("Name: $name Age: $age");
18     }
19 }
```

Тепер в класі визначено конструктор, який приймає два параметри і з їх допомогою встановлює значення полів об'єкта. У підсумку отримаємо результат роботи програми, як показано на рисунку 5.2.



```
Name: Sam Age: 25
```

Рисунок 5.2. Результат роботи програми

Слід враховувати, що якщо визначати в класі особисті конструктори, як у випадку вище, то конструктор за замовчуванням вже використовувати не можна.

5.1.2. Значення **null** і класи

Як і всі інші вбудовані типи, класи за замовчуванням мають тип, що не допускає значення **null**. Якщо ж необхідно, щоб об'єкт класу міг зберігати значення **null**, можна використовувати **nullable-клас**, тобто додати до визначення типу оператор **?**:

```
1 void main () {
2
3     Person? sam;
4     print(sam); // null - sam допускає значення
    null
5     Person tom;
6     // print(tom); // ! Помилка, tom не може приймати
    значення null
7 }
```

У випадку з викликом **print(tom)**, як і в випадку із змінними інших типів, які не можуть приймати значення **null**, необхідно присвоїти змінній початкове значення перед її використанням.

У той же час при використанні **nullable-класів** можна потрапити в таку ситуацію. Як писалося вище, для звернення до полів і методів об'єкта використовується оператор крапка (**.**), після якої вказується ім'я поля / методу. Однак, як відомо, якщо змінним **nullable-класу** не присвоєно значення, то вони за умовчанням мають значення **null**. Це необхідно враховувати. Наприклад, звернемося до полів об'єкта через змінну, яку не ініціалізовано:

```
1 void main () {
2
3     Person? sam;
4     sam.age = 23; // Помилка, sam = null
5 }
6
7 class Person {
8
9     String name = "undefined";
10    int age = 0;
11    void display() {
12        print("Name: $name Age: $age");
13    }
14 }
```


При виконанні рядка коду **sam.age = 23** програма викине виняток і завершить своє виконання. Тому що ще не створений об'єкт класу **Person** за допомогою конструктора. Тому змінна **sam** має значення **null**, тобто фактично нічого, а сам об'єкт не існує. Відповідно звернутися до поля неіснуючого об'єкта не можливо. Щоб уникнути подібної проблеми для звернення до полів можна використовувати інший оператор **?** (символ знаку питання з крапкою). Для прикладу розглянемо наступну програму:

```
1  void main () {
2
3      Person? sam;
4      sam?.age = 19;
5      sam?.display();
6      if(sam == null){
7          print("Змінна sam невизначена");
8      }
9  }
10
11 class Person{
12
13     String name = "undefined";
14     int age = 0;
15     void display(){
16         print("Name: $name Age: $age");
17     }
18 }
```

Оператор **?.** перевіряє значення змінної, і якщо змінна не дорівнює **null**, то здійснюється звернення до її полів і методів. Якщо ж вона дорівнює **null**, то звернення до змінної ігнорується.

5.1.3. Іменовані конструктори

За замовчуванням можна визначити тільки один загальний конструктор. Якщо ж необхідно використовувати в класі відразу кілька конструкторів, то в цьому випадку потрібно застосовувати **іменовані конструктори (named constructors)**. Розглянемо наступний приклад:

```
1  void main () {
2
3      Person bob = Person.undefined();           // ВИКЛИК
```

```

першого конструктора без параметрів
4     bob.display();
5
6     Person tom = Person.fromName("Tom"); // виклик
другого конструктора з одним параметром
7     tom.display();
8
9     Person sam = Person("Sam", 25); // виклик
третього конструктора з двома параметрами
10    sam.display();
11 }
12
13 class Person{
14
15     String name = "";
16     int age = 0;
17
18     Person.undefined(){
19         name = "undefined";
20         age = 18;
21     }
22     Person.fromName(String n){
23         name = n;
24         age = 18;
25     }
26     Person(String n, int a)
27     {
28         name = n;
29         age = a;
30     }
31     void display(){
32         print("Name: $name Age: $age");
33     }
34 }

```

В наведеному прикладі визначено два додаткових конструктора: **Person.undefined** без параметрів і **Person.fromName** з одним параметром. При виклику подібних конструкторів необхідно вказувати їх повне ім'я: **Person tom**

= **Person.fromName("Tom")**. Результат роботи програми показано на рисунку 5.3.

```
Name: undefined Age: 18
Name: Tom Age: 18
Name: Sam Age: 25
```

Рисунок 5.3. Результат роботи програми

5.1.4. Ключове слово **this**

Ключове слово **this** є посиланням на поточний екземпляр класу. Через це ключове слово можна звертатися до змінних, методів об'єкта, а також викликати його конструктори. Розглянемо наступний приклад:

```
1  class Person{
2
3      String name = "";
4      int age = 0;
5
6      Person(String name, int age)
7      {
8          this.name = name;
9          this.age = age;
10     }
11     void display(){
12         print("Name: $name Age: $age");
13     }
14 }
```

У конструкторі параметри називаються так само, як і поля класу. І щоб розмежувати поля і параметри, застосовується ключове слово **this**:

```
1  this.name = name;
```

Так, в цьому прикладі вказується, що значення параметра **name** присвоюється полю **name**.

5.1.5. Скорочена версія конструктора

Також використовуючи ключове слово **this**, можна скоротити визначення конструктора:

```

1  class Person{
2
3      String name;
4      int age;
5
6      Person(this.name, this.age);
7
8      void display(){
9          print("Name: $name Age: $age");
10     }
11 }

```

Коли буде відбуватися виклик конструктора:

```
1  Person sam = Person("Sam", 25);
```

то рядок **"Sam"** буде передаватися **this.name**, а число 25 - **this.age**.

Варто відзначити, що при такому визначенні конструктора необов'язково встановлювати для полів класу початкові значення.

5.1.6. Виклик ланцюжка конструкторів

У прикладі вище з іменованими конструкторами застосовувалося три конструктора, які виконують ідентичні дії: встановлюють поля **name** і **age**. Щоб уникнути повторів, за допомогою **this** можна викликати один з конструкторів класу і передати для його параметрів необхідні значення. У підсумку можна визначити кілька конструкторів, які приймають різну кількість параметрів і по ланцюжку викликають один одного:

```

1  void main (){
2
3      Person bob = Person.undefined();           // ВИКЛИК
першого конструктора без параметрів
4      bob.display();
5
6      Person tom = Person.fromName("Tom"); // ВИКЛИК
другого конструктора з одним параметром
7      tom.display();
8
9      Person sam = Person("Sam", 25); // ВИКЛИК
третього конструктора з двома параметрами
10     sam.display();

```

```

11 }
12
13 class Person{
14
15     String name = "";
16     int age = 0;
17     Person.undefined(): this("undefined", 18);
18
19     Person.fromName(String name): this(name, 18);
20
21     Person(this.name, this.age);
22
23     void display(){
24         print("Name: $name Age: $age");
25     }
26 }

```

В наведеному прикладі, перший і другий конструктор викликають третій, в якому власне будуть встановлюватися поля **name** та **age**.

5.1.7. Ініціалізатор

Ініціалізатор - це спосіб ініціалізації полів класу:

```

1 void main (){
2
3     Person tom = Person("Tom", 23);
4     tom.display();
5 }
6
7 class Person{
8
9     String name = "";
10    int age = 0;
11
12    Person(userName, userAge) : name=userName, age =
userAge
13    {
14        print("Person done!");
15    }

```

```

16
17     void display() {
18         print("Name: $name Age: $age");
19     }
20 }

```

Список ініціалізації зазначається після параметрів конструктора через двокрапку до фігурної дужки, яка відкривається:

```

1  Person (userName, userAge): name = userName, age =
    userAge {

```

Зазвичай списки ініціалізації використовують параметри конструктора для встановлення значень полів. При цьому, конструктор може виконувати якусь іншу роботу.

Окрім цього, при ініціалізації полів можна задати якусь динамічну логіку:

```

1  Person(userName, userAge) : name=userName, age =
    userAge + 10 {
2      print("Person done!");
3  }

```

Варто враховувати, що при використанні ініціалізаторів спочатку виконується ініціалізація полів в списках ініціалізації і тільки потім виконується логіка в конструкторі:

```

1  Person(userName, userAge) : name=userName, age =
    userAge + 10 {
2      age = 8;
3  }

```

В наведеному прикладі значення поля **age** дорівнюватиме 8, так як конструктор перезапише значення, встановлене ініціалізатором.

5.1.8. Каскадна нотація

Каскадна нотація – це операція `..`, яка дозволяє виконати послідовність операцій над одним об'єктом. Розглянемо наступний приклад:

```

1  void main () {
2
3      Person tom = Person()
4          ..name = "Tom"
5          ..age = 36
6          ..display();    // Name: Tom Age: 36
7  }

```

```

8
9  class Person{
10
11      String name = "";
12      int age = 0;
13
14      void display(){
15          print("Name: $name Age: $age");
16      }
17 }

```

В цьому разі код у функції **main** буде тотожний наступному:

```

1  void main () {
2
3      Person tom = Person();
4      tom.name = "Tom"
5      tom.age = 36
6      tom.display();  // Name: Tom  Age: 36
7  }

```

Подібним чином можна звертатися до більш складних об'єктів:

```

1  void main () {
2
3      Person tom = Person()
4          ..name = "Tom"
5          ..age = 36
6          ..company = (Company("Huawei"))
7          ..country = "Китай"
8          ..payTaxes()
9      )
10     ..display();
11 }
12
13 class Person{
14
15     String name = "";
16     int age = 0;
17     Company company = new Company("");
18
19     void display(){

```

```

20         print("Ім'я: $name Вік: $age");
21         print("працює в ${company.title}");
22     }
23 }
24 class Company{
25
26     String title = "";
27     String country = "";
28     Company(String companyTitle){
29
30         title = companyTitle;
31     }
32     payTaxes(){
33         print("Компанія $title сплачує податки \n");
34     }
35 }

```

Результат роботи програми показано на рисунку 5.4.

Компанія Huawei сплачує податки

**Ім'я: Tom Вік: 36
працює в Huawei**

Рисунок 5.4. Результат роботи програми

Якщо потрібно було б викликати конструктор, то можна було б написати так:

```

1  void main () {
2
3      Person tom = Person()
4          ..name = "Tom"
5          ..age = 36
6          ..company.title = "Huawei"
7          ..company.country = "Китай"
8          ..company.payTaxes()
9          ..display();
10 }

```


5.2. Константи і константні конструктори класів

5.2.1. Константи класів

Крім звичайних полів-змінних, класи можуть визначати константи. Для визначення констант можна використовувати ключове слово **final**. Розглянемо наступний приклад:

```
1  void main () {
2
3      Person tom = Person("Tom", 23);
4      print(tom.minAge);
5  }
6
7  class Person {
8
9      String name = "";
10     int age = 1;
11     final minAge = 1;
12
13     Person(n, a) {
14
15         this.name = n;
16         this.age = a;
17     }
18 }
```

Значення констант класів повинні бути встановлені до виклику конструктора. Це може відбуватися при визначенні константи, як в прикладі вище:

```
1  final minAge = 1;
```

Іншим варіантом є установка значення константи через параметр конструктора:

```
1  void main () {
2
3      Person tom = Person(4, "Tom", 23);
4      print(tom.minAge); // 4
5  }
6
7  class Person {
```

```

8
9     String name = "";
10    int age = 1;
11    final minAge;
12
13    Person(this.minAge, n, a){
14
15        this.name = n;
16        this.age = a;
17    }
18 }

```

Третім способом - є установка значень констант за допомогою ініціалізатора:

```

1  void main (){
2
3      Person tom = Person(6, "Tom", 23);
4      print(tom.minAge);    // 7
5  }
6
7  class Person{
8
9      String name = "";
10     int age = 1;
11     final minAge;
12
13     Person(mAge, name, age): minAge= mAge + 1{
14
15         this.name = name;
16         this.age = age;
17     }
18 }

```

В наведеному прикладі константа отримує значення параметра **minAge + 1**, тобто значення констант можуть відображати динамічно обчислювані значення.

5.2.2. Константні конструктори

Класи можуть містити константні конструктори. Такі конструктори покликані створювати об'єкти, які не повинні змінюватися. Константні

конструктори позначаються ключовим словом **const**. Клас, який визначає подібний конструктор, не повинен містити змінних, але може визначати константи. Крім того, константні конструктори не мають тіла. Розглянемо наступний приклад:

```
1  void main () {
2
3      Person tom = const Person("Tom", 23);
4      print(tom.name);      // Tom
5  }
6
7  class Person{
8
9      final String name;
10     final int age;
11
12     // константний конструктор
13     const Person(this.name, this.age);
14 }
```

При виклику константного конструктору перед його викликом ставиться ключове слово **const**: **Person tom = const Person ("Tom", 23);**

5.3. Успадковування

Успадковування є одним з ключових моментів об'єктно-орієнтованого програмування, дозволяючи передавати одним класам функціонал інших. У мові Dart успадкування реалізується за допомогою ключового слова **extends** (як у Java). В якості прикладу розглянемо наступний програмний код:

```
1  void main () {
2
3      Employee bob = Employee();
4      bob.name = "Bob";
5      bob.display();
6  }
7
8  class Person{
9
10     String name = "";
11 }
```

```

12     void display() {
13         print("Name: $name");
14     }
15 }
16 class Employee extends Person{
17
18 }

```

Клас **Employee**, який позначає працівника, є підкласом або успадковується від класу **Person**. А клас **Person** називається батьківським або базовим класом. При успадковуванні клас **Employee** переймає весь функціонал класу **Person** - всі його поля і методи та може їх використовувати. Також можна визначити в підкласі нові поля і методи, які відсутні в класі **Person**.

У підсумку можна створити об'єкт **Employee** і звертатися до його поля **name** і методу **display**, хоча вони визначені в батьківському класі **Person**.

5.3.1. Конструктори і ключове слово **super**

На відміну від полів і методів, конструктори базового класу не успадковуються. Якщо базовий клас явно визначає конструктор (конструктор за замовчуванням не враховується), то його необхідно викликати в класі-спадкоємця при визначенні конструктора:

```

1  void main () {
2
3      Employee bob = Employee("Bob");
4      bob.display();
5  }
6
7  class Person{
8
9      String name = "";
10     Person(this.name);
11
12     void display() {
13         print("Name: $name");
14     }
15 }
16 class Employee extends Person{
17

```

```

18     Employee(name) : super(name);
19 }

```

Тут базовий клас **Person** визначає конструктор, який приймає один параметр. У похідному класі **Employee** також визначається конструктор, який викликає конструктор базового класу, передаючи йому значення параметра **name**. Для звернення до функціональності базового класу з похідного застосовується ключове слово **super**. Зокрема, виклик **super (name)** фактично буде означати звернення до конструктору базового класу **Person (name)**.

Якщо не викликати конструктор базового класу при визначенні конструктора в похідному класі, то зіткнемося з помилкою.

Подібним чином можна перевизначати і звертатися до іменованих конструкторів базового класу. Приклад коду програми наведено нижче:

```

1  void main () {
2
3      Employee bob =
4      Employee.byAgeNameConstructor("Bob", 45);
5      bob.display();
6  }
7
8  class Person{
9
10     String name = "";
11     int age = 0;
12     Person(this.name) {
13         age = 18;
14     }
15     Person.byAgeNameConstructor(this.name, this.age);
16     void display() {
17         print("Name: $name  Age: $age");
18     }
19 }
20 class Employee extends Person{
21
22     Employee(String name) : super(name);
23     Employee.byAgeNameConstructor(String name, int
24     age) : super.byAgeNameConstructor(name, age);

```

5.3.2. Перевизначення методів

Похідні класи можуть визначати свої поля і методи, але також можуть перевизначати, змінювати поведінку методів базового класу. Для цього застосовується анотація **@override**:

```
1  void main () {
2
3      Employee bob = Employee("Bob", "Google");
4      bob.display();
5  }
6
7  class Person{
8
9      String name = "";
10     Person(this.name);
11
12     void display(){
13         print("Name: $name");
14     }
15 }
16 class Employee extends Person{
17
18     String company = "";
19
20     Employee(name, this.company) : super(name);
21
22     @override
23     void display(){
24         print("Name: $name");
25         print("Company: $company");
26     }
27 }
```

Тут в клас **Employee** додається ще одне поле - **company** для зберігання місця роботи співробітника. Щоб цю інформацію вивести на консоль, перевизначається метод **display**. В результаті, при виконанні програми отримаємо вигляд консолі, як показано на рисунку 5.5.

Name: Bob
Company: Google

Рисунок 5.5. Результат роботи програми

Однак можна побачити, що частина коду методу **display** в **Employee** повторює код методу **display** з **Person**. Щоб не повторюватися, за допомогою ключового слова **super** можна просто викликати реалізацію методу **display** з базового класу:

```
1  @override
2  void display() {
3      super.display();    // Виклик реалізації з
                           класу Person
4      print("Company: $company");
5  }
```

5.4. Абстрактні класи та методи

5.4.1. Абстрактні класи

Абстрактні класи є класами, які визначені з ключовим словом **abstract**:

```
1  abstract class Figure {
2  }
```

Абстрактні класи схожі на звичайні класи (також можуть визначати поля, методи, конструктори) за тим винятком, що не можна створити безпосередньо об'єкт абстрактного класу, використовуючи його конструктор:

```
1  void main () {
2
3      // Figure ellipse = Figure();    // так
                                         зробити не можна - клас Figure - абстрактний
4  }
```

Як правило, абстрактні класи описують сутності, які в реальності не мають конкретного втілення. Наприклад, геометрична фігура може позначати коло, квадрат, трикутник, але як такої геометричної фігури самої по собі не існує. Є конкретні фігури, з якими і працюємо. У той же час всі фігури можуть мати якийсь загальний функціонал, наприклад, методи обчислення периметра, площі і т.і. Розглянемо наступний приклад:

```
1  void main () {
2
```

```

3      Figure rect = Rectangle(20, 30);
4      rect.calculateArea();
5  }
6  abstract class Figure {
7      void calculateArea(){
8          print("Not Implemented");
9      }
10 }
11 class Rectangle extends Figure{
12
13     int width;
14     int height;
15     Rectangle(this.width, this.height);
16
17     void calculateArea(){
18         int area = width * height;
19         print("area = $area");
20     }
21 }

```

Тут абстрактний клас геометричної фігури визначає метод **calculateArea()**, який виводить на консоль площу фігури. Клас прямокутника визначає свою реалізацію для цього методу.

5.4.2. Абстрактні методи

В наведеному вище прикладі метод **calculateArea** в базовому класі **Figure** не виконує ніякої корисної роботи, так як у абстрактної фігури не може бути площі. І в цьому випадку подібний метод краще визначити як абстрактний. Розглянемо наступний програмний код:

```

1  void main () {
2
3      Figure rect = Rectangle(20, 30);
4      rect.calculateArea();
5  }
6  abstract class Figure {
7      void calculateArea();
8  }
9  class Rectangle extends Figure{

```



```

10
11     int width;
12     int height;
13     Rectangle(this.width, this.height);
14
15     void calculateArea() {
16         int area = width * height;
17         print("area = $area");
18     }
19 }

```

Абстрактний метод визначається також, як і звичайний, тільки замість тіла методу після списку параметрів йде крапка з комою: **void calculateArea();**.

Важливо відзначити, що абстрактні методи можуть бути визначені тільки в абстрактних класах. Крім того, якщо базовий клас визначає абстрактний метод, то клас-спадкоємець обов'язково повинен його реалізувати, тобто визначити тіло методу.

5.5. Реалізація інтерфейсів

Успадковування в мові Dart має важливе обмеження: не можна успадковувати клас відразу від декількох класів, наприклад, наступним чином:

```

1  class Person{
2
3      String name = "";
4      void display(){
5          print("Name: $name");
6      }
7  }
8  class Worker{
9      String company = "";
10     void work(){
11         print("Work in $company");
12     }
13 }
14 class Employee extends Person, Worker{ // Помилка,
15     множинне успадковування не дозволяється

```

Для вирішення цієї проблеми в Dart застосовується реалізація інтерфейсів. При цьому важливо розуміти, що інтерфейс - це не окрема сутність, як в деяких мовах програмування (наприклад, тип `interface` в C # або Java), а той же клас. Тобто клас в Dart одночасно виступає в ролі інтерфейсу, і інший клас може реалізувати даний інтерфейс.

Інтерфейс є синтаксичним контрактом (угодою), якому повинні відповідати класи, які реалізують цей інтерфейс. Тобто, якщо **клас-інтерфейс** визначає якісь поля і методи, то клас, який реалізує даний інтерфейс, повинен також визначити ці поля і методи.

Для реалізації інтерфейсів застосовується оператор **Implements**. Розглянемо наступний приклад:

```
1  void main () {
2
3      Person bob = Employee();
4      bob.name = "Bob";
5      bob.display(); // Employee name: Bob
6  }
7
8  class Person{
9
10     String name;
11     Person(this.name);
12
13     void display() {
14         print("Name: $name");
15     }
16 }
17
18 class Employee implements Person{
19
20     String name = ""; // реалізація поля name
21     // реалізація метода display
22     void display() {
23         print("Employee name: $name");
24     }
25 }
```

В наведеному прикладі клас **Employee** застосовує інтерфейс класу **Person** (**Employee implements Person**). Застосування інтерфейсу класу **Person** означає, що клас **Employee** повинен реалізувати всі поля та методи, які визначені в класі **Person**. Тому клас **Employee** повинен визначити поле **name** і метод **display**. По суті інтерфейс - це контракт (домовленість), що клас повинен містити певний функціонал (в прикладі, наведеному вище, поле **name** і метод **display**). Якщо клас **Employee** не визначив би всі поля та методи, які є в класі **Person**, то згенерувалася б помилка.

У той же час, з наведеного вище прикладу, видно, що клас **Employee** не зобов'язаний реалізувати конструктор і може взагалі не визначати ніякого конструктора.

Подібним чином можна реалізувати відразу кілька інтерфейсів:

```
1  void main () {
2
3      Employee bob = Employee("Bob", "Google");
4      bob.display(); // Employee name: Bob
5      bob.work();    // Employee works in Google
6  }
7
8  class Person {
9
10     String name;
11     Person(this.name);
12
13     void display() {
14         print("Name: $name");
15     }
16 }
17 class Worker {
18     String company = "";
19     void work() {
20         print("Work in $company");
21     }
22 }
23 class Employee implements Person, Worker {
24
25     String name; // реалізація поля name із Person
26     String company; // реалізація поля company із
```

```

Worker
27     // реалізація метода display Person
28     void display() {
29         print("Employee name: $name");
30     }
31     // реалізація метода work із Worker
32     void work() {
33         print("Employee works in $company");
34     }
35
36     Employee(this.name, this.company);
37 }

```

В цьому прикладі клас **Employee** реалізує інтерфейс класів **Person** і **Worker**.

5.5.1. Успадковування класів та реалізація інтерфейсів

При успадковуванні похідний клас не зобов'язаний визначати ті ж поля і методи, які є в базовому класі (за винятком абстрактних методів). Якщо у базовому класі визначається конструктор, то похідний клас зобов'язаний визначити свій конструктор, при якому викликається конструктор базового класу. У похідному класі можна звертатися до реалізації базового класу за допомогою ключового слова **super**. Множинне успадковування не підтримується.

При реалізації інтерфейсу похідний клас повинен визначити всі поля та методи, які визначені в класі інтерфейсу. Якщо в базовому є конструктор, то похідний клас НЕ зобов'язаний визначити свій конструктор. У похідному класі НЕ можна звертатися до методів реалізованого інтерфейсу за допомогою ключового слова **super**. При цьому підтримується множинна реалізація інтерфейсів.

5.6. Міксини

Ще одним рішенням проблеми множинного успадковування в Dart є **міксини**. По суті міксин є звичайним класом за тим винятком, що він не повинен містити конструктор, інакше він не може бути використаний в якості міксина. Для застосування міксина використовується оператор **with**. Розглянемо наступний приклад:

```

1  void main () {
2
3      Employee bob = Employee("Bob", "Google");
4      bob.display();    // Name: Bob
5      bob.work();       // Works in Google
6  }
7
8  class Person {
9
10     String name;
11     Person(this.name);
12     void display() {
13         print("Name: $name");
14     }
15 }
16 class Worker {
17     String company = "";
18     void work() {
19         print("Work in $company");
20     }
21 }
22 class Employee extends Person with Worker {
23
24     Employee(name, comp) : super(name) {
25
26         company = comp; // звертаємося до поля
27         company, яке визначено в міксині Worker
28     }

```

В наведеному прикладі клас **Worker** виступає в ролі міксина. Для використання його функціоналу при визначенні класу **Employee** застосовується оператор **with**:

```

1  class Employee extends Person with Worker {

```

Після цього клас **Employee** може використовувати поля і методи з класу **Worker**. При цьому клас **Employee** може успадковувати і від інших класів, як в зазначеному випадку від класу **Person**.

Також, оскільки тип **Worker** визначено як звичайний клас, можна використовувати його незалежно:

```

1  Worker tom = Worker();
2  tom.company = "Apple";
3  tom.work();

```

У той же час можна вказати, що даний тип буде використовуватися тільки як міксин (якщо не планується використовувати його в якості окремого класу).

В цьому випадку для визначення типу застосовується ключове слово **mixin**:

```

1  mixin Worker{
2      String company = "";
3      void work(){
4          print("Work in $company");
5      }
6  }

```

5.7. Статичні члени класів

Крім звичайних методів і полів, клас може мати статичні поля, методи, константи. Значення таких полів, методів і констант відносяться в цілому до всього класу, а не до окремих об'єктах. Для визначення статичних змінних, констант і методів перед їх оголошенням вказується ключове слово **static**.

5.7.1. Статичні поля

Статичні поля зберігають стан всього класу. Статичне поле визначається як і звичайне, тільки попереду ставиться ключове слово **static**. Наприклад, розглянемо клас **Employee**, який позначає працівника:

```

1  void main (){
2
3      Employee bob = Employee("Bob", 55);
4      bob.checkAge();
5      Employee.retirementAge = 65;
6      bob.checkAge();
7  }
8  class Employee{
9
10     String name;
11     int age;
12     static int retirementAge = 60;
13
14     Employee(this.name, this.age);

```

```

15
16     checkAge() {
17         if (age >= retirementAge) {
18             print("Пора на пенсію");
19         }
20         else {
21             print("До пенсії лишилось
22             ${retirementAge - age} роки");
23         }
24     }

```

У класі працівника визначені поля для зберігання імені та віку - у кожного окремого працівника може бути своє ім'я і свій вік. Крім того, визначено статичне поле для зберігання пенсійного віку **retirementAge**. Воно стосується не до конкретного працівника, а до всіх працівників, тобто до всього класу **Employee** в цілому, так як не можна для кожного окремого працівника встановити свій пенсійний вік. Потім в самому класі можна використовувати це поле як і будь-які інші.

Якщо необхідно звернутися до цього поля поза класом, наприклад, змінити його значення, то для звернення застосовується ім'я класу:

```

1  Employee.retirementAge = 65;

```

5.7.2. Статичні константи

Якщо пенсійний вік міняти не планується, то можна визначити його як статичну константу:

```

1  void main () {
2
3      Employee bob = Employee("Bob", 55);
4      bob.checkAge();
5      print("Retirement age:
6      ${Employee.retirementAge}");
7  }
8
9  class Employee {
10
11      String name;
12      int age;
13      static const int retirementAge = 65;

```

```

12
13     Employee(this.name, this.age);
14
15     checkAge() {
16         if(age >= retirenmentAge){
17             print("Пора на пенсію");
18         }
19         else{
20             print("До пенсі лишилось ${retirenmentAge
- age} років");
21         }
22     }
23 }

```

Для визначення статичної константи можна використовувати або вираз **static const**, або **static final**:

```

1  static const int retirenmentAge = 65;
2  static final int retirenmentAge = 65;

```

5.7.3. Статичні методи

Статичні методи також відносяться до всього класу і позначаються ключовим словом **static**. Як правило, статичні методи виконують такі обчислення, які не зачіпають стан або поведінку об'єкта:

```

1  void main () {
2
3      int a = Operation.sum(3, 5);
4      int b = Operation.subtract(15, 7);
5      int c = Operation.multiply(3, 5);
6      print("a=$a b=$b c=$c"); // a=8 b=8 c=15
7  }
8  class Operation{
9
10     static int sum(int x, int y) => x + y;
11     static int subtract(int x, int y) => x - y;
12     static int multiply(int x, int y) => x * y;
13 }

```

В наведеному прикладі для методів **sum**, **subtract**, **multiply** не має значення, який саме екземпляр класу **Operation** використовується. Ці методи

працюють тільки з параметрами, не зачіпаючи стан класу. Тому їх можна визначити як статичні.

Варто відзначити, що Google не рекомендує створювати класи, які містять тільки статичні методи і змінні/константи, як вище визначений клас **Operation**. Так як в цьому випадку просто можна визначити звичайні функції і також їх використовувати:

```
1 void main () {
2
3     int a = sum(3, 5);
4     int b = subtract(15, 7);
5     int c = multiply(3, 5);
6     print("a=$a b=$b c=$c"); // a=8 b=8 c=15
7 }
8 int sum(int x, int y) => x + y;
9 int subtract(int x, int y) => x - y;
10 int multiply(int x, int y) => x * y;
```

Статичні методи можуть використовуватися для управління статичними полями, тобто станом, який відноситься до всього класу. У той же час в статичних методах не можна використовувати нестатичні методи і поля класу:

```
1 void main () {
2
3     Employee bob = Employee("Bob", 55);
4     Employee.setRetirenmentAge(63);
5     print(Employee.retirenmentAge);
6 }
7 class Employee{
8
9     String name;
10    int age;
11    static int retirenmentAge = 65;
12
13    Employee(this.name, this.age);
14
15    checkAge() {
16        if(age >= retirenmentAge) {
17            print("Пора на пенсію");
18        }
19        else{
```

```

20         print("До пенсії лишилось ${retirenmentAge
- age} років");
21     }
22 }
23 static void setRetirenmentAge(int value){
24     if(value > 50 && value < 70){
25         retirenmentAge = value;
26     }
27     else{
28         print("Некоректне значення");
29     }
30 }
31 }

```

В наведеному прикладі статичний метод **setRetirenmentAge** дозволяє управляти статичною змінною **retirenmentAge**.

Розглянемо у чому ж переваги статичних полів і методів перед нестатичними. Статичні поля і методи споживають менше пам'яті, ніж нестатичні. Нестатична змінна після ініціалізації (при першому присвоєнні значення) відразу починає споживати пам'ять незалежно від того, використовується вона чи ні. А статичні поля і методи не ініціалізуються до того моменту, поки вони не почнуть використовуватися в програмі. Відповідно споживати пам'ять вони починають тільки тоді, коли безпосередньо починають використовуватися.

5.8. Generics

Generics або узагальнення дозволяють додати програмі гнучкості і відійти від жорсткої прив'язки до певних типів. Іноді виникає необхідність, визначити функціонал таким чином, щоб він міг би використовувати дані будь-яких типів.

Наприклад, визначимо клас користувача, який містить **id** (ідентифікатор користувача):

```

1  class Person{
2      int id; // ідентифікатор користувача
3      String name; // ім'я користувача
4      Person(this.id, this.name);
5  }

```

Тут **id** заданий як числове значення, тобто воно може дорівнювати 2, 5, 7 і так далі. Але можливо, згодом знадобиться використовувати в якості типу **id**

рядок або якісь інші класи. Для додавання гнучкості можна використовувати оператор **dynamic**, щоб уникнути жорсткої прив'язки до типу:

```
1  class Person{
2      dynamic id; // ідентифікатор користувача
3      String name; // ім'я користувача
4      Person(this.id, this.name);
5  }
6  void main () {
7
8      Person tom = Person(134, "Tom");
9      print(tom.id);
10     Person bob = Person("324", "Bob");
11     print(bob.id);
12 }
```

Однак може бути невідомо, який саме об'єкт позначає поле **id**, особливо якщо клас **Person** визначено у зовнішній бібліотеці, яка написана сторонніми розробниками. І при спробі отримати число в даному випадку можна зіткнутися з виключенням:

```
1  Person bob = Person("324", "Bob");
2  int id = bob.id; // Помилка
```

Як вихід з такої ситуації, можна передбачити два типи **Person** для роботи з різними типами:

```
1  class PersonInt{
2      int id;
3      String name;
4      PersonInt(this.id, this.name);
5  }
6  class PersonString{
7      string id;
8      String name;
9      PersonString(this.id, this.name);
10 }
```

Але в цьому разі можна стикнутись з іншою проблемою - дублюванням коду.

Generics або узагальнення дозволяють забезпечити більшу безпеку типів і допомагають уникнути дублювання коду. Перепишемо код класу **Person** з використанням **generics**:

```

1  void main () {
2
3      Person bob = Person("324", "Bob");
4      print(bob.id.runtimeType); // String
5      Person sam = Person(123, "Sam");
6      print(sam.id.runtimeType); // int
7  }
8
9  class Person<T>{
10     T id;    // ідентифікатор користувача
11     String name; // ім'я користувача
12     Person(this.id, this.name);
13 }

```

За допомогою виразу `<T>` вказується, що клас **Person** типізовано певним типом **T**. **T** ще називається універсальним параметром. Причому назва параметра може бути довільною, але зазвичай використовуються великі літери, часто літера **T**. Після цього можна використовувати **T** як звичайний тип, наприклад, визначати змінні цього типу: **T id**;

При виконанні програми замість **T** буде підставлятися конкретний тип. Причому тип буде визначатися динамічно на основі переданих значень. За допомогою поля **runtimeType** можна отримати конкретний тип даних змінної. Також можна явним чином позначити, які типи будуть використовуватися в об'єктах:

```

1  Person<String> bob = Person<String>("324", "Bob");
2  print(bob.id.runtimeType);
3  Person<int> sam = Person<int>(123, "Sam");
4  print(sam.id.runtimeType);

```

Тип, яким типізується клас, вказується в кутових дужках після назви класу (**Person <String>**).

Подібним чином можна визначати **generic**-методи і функції. Наприклад, визначимо і використаємо невелику функцію логарифмування:

```

1  void main () {
2
3      int x = 20;

```

```

4      log(x);
5      x = 34;
6      log(x);
7      String name = "Tom";
8      log(name);
9  }
10 void log<T>(T a) {
11
12     // DateTime.now() - отримує поточну дату та час
13     print("${DateTime.now()} a=$a");
14 }

```

Для створення узагальненого методу після його імені вказується в кутових дужках назва універсального параметра. Після цього універсальний тип **T** може використовуватися всередині методу - як тип параметрів або змінних.

5.8.1. Обмеження узагальнень

Іноді необхідно використовувати узагальнення, однак приймати будь-який тип у функцію або клас замість параметра **T** небажано. Наприклад, є наступний клас **Account**, який позначає банківський рахунок:

```

1  class Account{
2
3      int id;        // номер рахунку
4      int sum;       // сума на рахунку
5      Account(this.id, this.sum);
6  }

```

Для переказу коштів з одного рахунку на інший можна визначити клас **Transaction**, який для виконання всіх операцій буде використовувати об'єкти класу **Account**.

Але у класі **Account** може бути багато спадкоємців: **DepositAccount** (депозитний рахунок), **DemandAccount** (рахунок до запитання) і т.і. І не відомо заздалегідь, які саме типи рахунків будуть використовуватися в класі **Transaction**. Можливо, транзакції будуть проводитися тільки між рахунками до запитання. І в цьому разі в якості універсального параметра можна встановити тип **Account**:

```

1  class Transaction<T extends Account>{
2

```

```

3      T fromAccount;    // з якого рахунку перевід
4      T toAccount;      // на який рахунок перевід
5      int sum;          // сума переводу
6      Transaction(this.fromAccount, this.toAccount,
this.sum);
7      void execute() {
8
9          if (fromAccount.sum > sum) {
10             fromAccount.sum -= sum;
11             toAccount.sum += sum;
12             print("Рахунок ${fromAccount.id}:
${fromAccount.sum}\$ \nРахунок ${toAccount.id}:
${toAccount.sum}\$");
13         }
14         else
15         {
16             print("Недостатньо грошей на рахунку
${fromAccount.id}");
17         }
18     }
19 }

```

За допомогою виразу **<T extends Account>** вказуємо, що використовуваний тип **T** обов'язково повинен бути класом **Account** або його спадкоємцем. Завдяки подібному обмеженню можна використовувати всередині класу **Transaction** всі об'єкти типу **T** саме як об'єкти **Account** і відповідно звертатися до їх полів і методів.

Тепер застосуємо клас **Transaction**:

```

1  void main () {
2
3      Account acc1 = Account(1857, 4500); // sum = 4500;
4      Account acc2 = Account(3453, 5000); // sum = 5000;
5      Transaction transaction =
Transaction<Account>(acc1, acc2, 1900);
6      transaction.execute();
7  }

```

Результат роботи програми показано на рисунку 5.6.

Рахунок 1857 2600 \$

Рахунок 3453 6900 \$

Рисунок 5.6. Результат роботи програми

5.9. Фабричний конструктор

За замовчуванням конструктори класу створюють і повертають новий об'єкт цього класу. Але, можливо, не завжди потрібно створювати новий об'єкт класу. Можливо, потрібно тільки використовувати і повертати з конструктора вже наявний об'єкт. Для цієї мети в мові Dart застосовуються фабричні конструктори або конструктори, які позначаються ключовим словом **factory**.

Наприклад, розглянемо наступну програму:

```
1  void main () {
2
3      Application myApp1 = Application("Internet
   Browser");
4      Application myApp2 = Application("Internet Browser
   2");
5      myApp2.about();
6  }
7
8  class Application{
9
10     String name;
11     static Application app = Application.fromName("");
12
13     Application.fromName(this.name);
14
15     factory Application(String name) {
16         if(app.name == ""){
17             app = Application.fromName(name);
18             print("Застосунок $name запущено");
19         }
20         else{
21             // у фабричних конструкторах не можна
   звертатися до this
22             // print("В застосунку ${this.name}
```

```

        відкрита нова вкладка");
23         print("В застосунку ${app.name} відкрита
        нова вкладка");
24     }
25     return app;
26 }
27 void about(){
28     print("Застосунок $name");
29 }
30 }

```

В даному випадку визначено клас **Application**, який позначає застосунок. Припустимо, стоїть задача, щоб одночасно було запущено тільки один застосунок. Для цієї мети визначаємо в класі статичну змінну **app**, яка позначає тип **Application**. І також визначаємо змінну **name** для зберігання умовної назви програми.

У класі **Application** визначено іменований конструктор **Application.fromName**, який встановлює **name**.

І також визначено фабричний конструктор:

```

1  factory Application(String name) {
2      if(app.name == ""){
3          app = Application.fromName(name);
4          print("Застосунок $name запущено");
5      }
6      else{
7          // у фабричних конструкторах не можна
          звертатися до this
8          // print("В застосунку ${this.name}
          відкрита нова вкладка");
9          print("В застосунку ${app.name} відкрита
          нова вкладка");
10     }
11     return app;
12 }

```

Фабричний конструктор, як і звичайні конструктори, може приймати параметри. В наведеному прикладі в конструктор для установки передається **name**.

У самому конструкторі перевіряється, якщо змінна **name** дорівнює порожньому рядку, то створюється новий об'єкт **Application** за допомогою

конструктора **Application.fromName** і присвоюється його змінній **app**. Якщо об'єкт вже був створений, то просто виводиться деяка інформація. І в кінці повертається створений об'єкт.

Тобто, фабричний конструктор створює новий об'єкт **Application**, якщо його немає (змінна **name** є порожнім рядком), або повертає вже наявний. Тобто отримуємо об'єкт, який існує в одному примірнику.

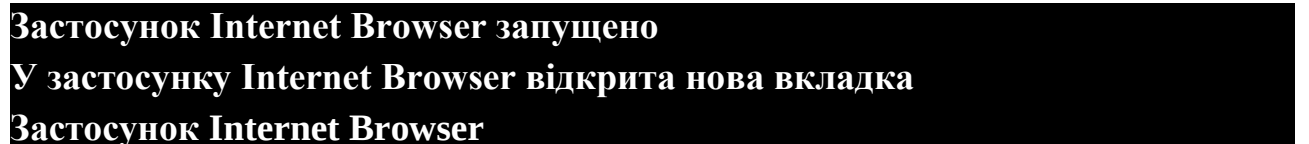
Також варто звернути увагу на те, що у фабричних конструкторах можна звертатися до поточного об'єкту через ключове слово **this**:

```
1 //print("В застосунку ${this.name} відкрита нова  
   вкладка");
```

Для прикладу, у функції **main** визначаємо дві змінні цього класу:

```
1 void main () {  
2  
3     Application myApp1 = Application("Internet  
   Browser");  
4     Application myApp2 = Application("Internet  
   Browser 2");  
5     myApp2.about();  
6 }
```

Результат роботи програми показано на рисунку 5.7.



```
Застосунок Internet Browser запущено  
У застосунку Internet Browser відкрита нова вкладка  
Застосунок Internet Browser
```

Рисунок 5.7. Результат роботи програми

З консольного виведення видно, що конструктор **Application.fromName**, який створює новий об'єкт **Application**, спрацьовує тільки один раз. Також бачимо, що при установці **name** у другій змінній, змінюється **name** і у першій змінній. Тому що в наведеному прикладі існує тільки один об'єкт **Application**, а обидві змінні - **myApp1** і **myApp2** вказують на один і той самий об'єкт.

Розглянемо де може застосовуватися подібна техніка. Перш за все це різні варіанти кешування, коли замість того, щоб витрачати додаткові ресурси на створення нового об'єкту, можна взяти з кешу вже існуючий. А якщо він відсутній, то створити його.

5.10. Перевизначення операторів

Dart дозволяє змінити поведінку деяких вбудованих операторів, що дозволяє скористатися додатковими можливостями по роботі з об'єктами. Наприклад, нехай є наступний клас **Counter**:

```
1  class Counter{
2
3      int value;
4      Counter(this.value);
5  }
```

Наведений клас відображає певний лічильник, значення якого зберігається в полі **value**.

Тепер припустимо, що є два об'єкти класу **Counter** - два лічильника, які потрібно порівнювати або складати на підставі їх поля **value**, використовуючи стандартні операції порівняння і складання:

```
1  Counter counter1 = Counter(5);
2  Counter counter2 = Counter(15);
3  Counter counter3 = counter1 + counter2;
4
5  bool isFirstGreater = counter1 > counter2;
```

Але за замовчуванням ні операція порівняння, ні операція складання для об'єктів **Counter** не доступні. Ці операції можна використовувати з об'єктами примітивних типів, наприклад, **int**. Але як складати об'єкти класів, визначених розробником, Dart не знає. І для цього потрібно перевизначити відповідні оператори.

Перевизначення операторів полягає у визначенні в класі, для об'єктів якого необхідно визначити оператор, спеціального методу:

```
1  тип_який_повертається operator оператор(параметр) { }
```

Тип, який повертається, є тим типом, об'єкти якого необхідно отримати. Наприклад, в результаті складання двох об'єктів **Counter** очікується отримати новий об'єкт **Counter**. А в результаті порівняння двох об'єктів **Counter** потрібно отримати об'єкт типу **bool**, який вказує істинність умовного виразу або помилковість. Але в залежності від завдання типи, що повертаються, можуть бути будь-якими. Потім замість назви методу іде ключове слово **operator** і власне сам оператор. І далі в дужках вказується параметр.

Наприклад, перевизначимо оператор додавання для класу **Counter**:

```
1  void main () {
2
```

```

3     Counter counter1 = Counter(5);
4     Counter counter2 = Counter(15);
5     Counter counter3 = counter1 + counter2;
6     print(counter3.value); // 20
7 }
8
9 class Counter{
10
11     int value;
12     Counter(this.value);
13
14     Counter operator +(Counter otherCounter) {
15
16         return Counter(this.value +
17             otherCounter.value);
18     }
19 }

```

У випадку з операцією додавання потрібно скласти два об'єкти класу **Counter**, тобто оператор приймає як параметр об'єкт цього класу. І так як потрібно в результаті складання отримати новий об'єкт **Counter**, то даний клас також використовується в якості типу, що повертається. Всі дії цього оператора зводяться до створення, нового об'єкта, поле **value** якого об'єднує значення поля **value** обох об'єктів.

Варто зазначити, що не можна перевизначити абсолютно всі наявні у мові Dart оператори, а тільки деякі з них, а саме: `< + | [] > / ^ [] = <= ~ / & ~ >= * << == - % >>`

Розглянемо наприклад перевизначення для класу **Counter** ще й операторів `>` та `<`:

```

1 void main () {
2
3     Counter counter1 = Counter(5);
4     Counter counter2 = Counter(15);
5
6     bool isFirstGreater = counter1 > counter2;
7     print(isFirstGreater); // false
8
9     bool isFirstLess = counter1 < counter2;
10    print(isFirstLess); // true

```

```

11 }
12
13 class Counter{
14
15     int value;
16     Counter(this.value);
17
18     Counter operator +(Counter otherCounter) {
19
20         return Counter(this.value +
otherCounter.value);
21     }
22
23     bool operator >(Counter otherCounter) => this.valu
> otherCounter.value;
24     bool operator <(Counter otherCounter) => this.valu
< otherCounter.value;
25 }

```

Оскільки операції `<` та `>` повертають значення типу **bool**, то і тут при визначенні оператора повертається тип **bool**, а фактична дія оператора зводиться до порівняння значень полів **value** обох об'єктів.

Хоча в прикладах вище в якості параметра передавався об'єкт того ж типу - **Counter**, це необов'язково, в залежності від конкретної задачі параметр може мати будь-який тип. Наприклад, змінимо код наступним чином:

```

1 void main () {
2
3     Counter counter1 = Counter(15);
4
5     Counter counter2 = counter1 + 25;
6     print(counter2.value); // 40
7
8     int diff = counter1 - 7;
9     print(diff); // 8
10 }
11
12 class Counter{
13
14     int value;

```

```

15     Counter(this.value);
16
17     Counter operator +(int number){
18
19         return Counter(this.value + number);
20     }
21
22     int operator -(int number){
23
24         return this.value - number;
25     }
26 }

```

Тепер оператор додавання дозволяє скласти об'єкт **Counter** і звичайне число та повернути новий об'єкт **Counter**. А оператор віднімання дозволяє віднімати з **Counter** число і повернути інше - різницю поля **value** і числа від'ємника.

5.11. Перерахування

Перерахування в Dart або **enum** є особливим типом класів, який позначає фіксований набір константних значень. Для визначення перерахування застосовується ключове слово **enum**. Розглянемо приклад, де визначається найпростіше перерахування:

```

1  enum Operation{
2
3      add,
4      subtract,
5      multiply
6  }
7  void main (){
8
9      print(Operation.multiply); // Operation.multiply
10     print(Operation.multiply.index); // 2
11 }

```

В наведеному прикладі визначено перерахування **Operation**. У середині блоку перерахування перераховуються константи перерахування. В нашому випадку перерахування **Operation** має три константи: **add**, **subtract** і **multiply**.

Для звернення до констант перерахування вказується назва перерахування і через точку константа:

```
1 print(Operation.multiply);
```

Якщо спробувати вивести на консоль значення константи, то побачимо фактично її назву - **"Operation.multiply"**. Однак з кожною константою також зіставляється числовий індекс, який можна отримати через поле **index** (**Operation.multiply.index**). Так, в цьому разі константа **Operation.multiply** має індекс 2, тому що вона визначена третьої по порядку в перерахуванні. Перша константа (в наведеному прикладі - **add**) має індекс 0, друга - індекс 1 і так далі.

Кожне перерахування фактично визначає новий тип даних. Потім в програмі можна визначити змінну цього типу і використовувати її:

```
1 enum Operation{
2
3     add,
4     subtract,
5     multiply
6 }
7 void main (){
8
9     // створюємо змінну перерахування Operation
10    Operation op;
11    // присвоюємо значення
12    op = Operation.add;
13    print(op);           // Operation.add
14    print(op.index);     // 0
15 }
```

Значенням такої змінної є одна з констант цього перерахування.

Зазвичай змінна перерахування виступає в якості сховища стану, в залежності від якого здійснюються деякі дії. Так, розглянемо застосування перерахування на більш реальному прикладі:

```
1 enum Operation{
2
3     add,
4     subtract,
5     multiply
6 }
7 void main (){
```

```

8
9      // Тип операції задаємо за допомогою константи
      Operation.add, яка дорівнює 0
10     runOperation(10, 5, Operation.add);
11
12     // Тип операції, яка дорівнює 2
13     runOperation(11, 5, Operation.multiply);
14 }
15
16 void runOperation(int x, int y, Operation op)
17 {
18     int result = 0;
19
20     switch (op){
21         case Operation.add:
22             result = x + y;
23             break;
24         case Operation.subtract:
25             result = x - y;
26             break;
27         case Operation.multiply:
28             result = x * y;
29             break;
30     }
31
32     print("Результат операції дорівнює $result");
33 }

```

Тут функція **runOperation** в якості параметрів приймає два числа і тип операції. У функції **main** два рази викликається **runOperation**, отримуючи два числа і тип операції.

Результат роботи програми показано на рисунку 5.8.

Результат операції дорівнює 15
Результат операції дорівнює 55

Рисунок 5.8. Результат роботи програми

Резюме

В цьому розділі було розглянуто питання використання можливостей Dart для об'єктно-орієнтованого програмування.

Розглянуто способи визначення та використання класів, об'єктів, інтерфейсів, міксинів та узагальнень.

Показано яким чином визначаються та використовуються конструктори, абстрактні класи та методи, статичні члени класів, фабричні конструктори та перевизначення операторів при написанні програм.

Контрольні запитання і завдання

1. Що таке об'єкт в Dart?
2. Що таке клас в Dart? Як він позначається?
3. Що таке конструктор класу в Dart? Як він позначається?
4. Що таке конструктор класу за замовчуванням? Який тип має клас за замовчуванням?
5. Як позначаються та застосовуються **nullable-класи**?
6. Що таке іменовані конструктори? Як вони позначаються та використовуються?
7. Що таке ключове слово **this**? Як воно використовується?
8. Що таке скорочена версія конструктора?
9. Що таке ланцюжок конструкторів?
10. Що таке ініціалізатор?
11. Що таке каскадна нотація? Для чого вона застосовується?
12. Як використовуються константи класів?
13. Як використовуються константні конструктори?
14. Що таке успадковування? Як воно реалізується у Dart?
15. Який зміст і правила використання ключового слова **super**?
16. Що таке і як позначаються абстрактні класи?
17. Що таке і як позначаються абстрактні методи?
18. Що таке і як реалізуються інтерфейси?
19. Що таке і як реалізуються міксини?
20. Що таке статичні члени класу?
21. Як в Dart реалізуються статичні поля, константи, методи?
22. Що таке узагальнення (**Generics**)? Для чого і як вони використовуються?
23. Для чого потрібні фабричні конструктори? Як вони позначаються та використовуються?

24. Що таке перевизначення операторів та як воно реалізується?
25. Що таке перерахування (**enum**) в Dart? Як воно визначається та використовується?
26. Створіть який-небудь клас.
27. Додайте конструктор класу.
28. Створіть іменованний конструктор у класі.
29. Додайте приклад використання ключового слова **this**.
30. Додайте приклад використання ланцюжка конструкторів.
31. Наведіть приклад успадковування та використання ключового слова **super**.
32. Створіть абстрактний клас та абстрактні методи.
33. Наведіть приклад реалізації інтерфейсів.
34. Наведіть приклад реалізації міксина.
35. Наведіть приклад реалізації узагальнення (**Generics**).
36. Наведіть приклад реалізації фабричних конструкторів.
37. Наведіть приклад реалізації перевизначення.
38. Для успадковування одного класу від іншого застосовується який оператор?
- ✓ **extends;**
 - ✓ **implements;**
 - ✓ **inherits;**
 - ✓ **extending.**
39. Скільки параметрів приймає конструктор класу **Employee** в наступному коді?

```
1 class Person{
2
3     String name = "";
4     Person(this.name);
5
6     void display(){
7         print("Name: $name");
8     }
9 }
10 class Employee extends Person{
11 }
```

40. Що буде виведено на консоль при виконанні наступної програми і чому?

```

1  void main () {
2
3      Employee bob = Employee("Bob");
4      bob.display();
5  }
6
7  class Person {
8
9      String name = "";
10     Person(this.name) {
11
12         print("Створено користувача $name");
13     }
14
15     void display() {
16         print("Name: $name");
17     }
18 }
19 class Employee extends Person {
20 }

```

41.Що буде виведено на консоль при виконанні наступної програми і чому?

```

1  void main () {
2
3      Employee bob = Employee("Bob");
4      bob.display();
5  }
6
7  class Person {
8
9      String name = "";
10     Person(this.name) {
11
12         print("Створено користувача $name");
13     }
14
15     void display() {
16         print("Name: $name");

```

```

17     }
18 }
19 class Employee extends Person{
20
21     Employee(String name){
22         this.name = name;
23     }
24 }

```

42.Що буде виведено на консоль при виконанні наступної програми і чому?

```

1 void main () {
2
3     Employee bob = Employee("Bob");
4     bob.display();
5 }
6
7 class Person{
8
9     String name = "";
10    void display(){
11        print("Name: $name");
12    }
13 }
14 class Employee extends Person{
15
16     Employee(String name){
17         this.name = name;
18     }
19 }

```

6. ЗАСТОСУВАННЯ КОЛЕКЦІЙ

6.1. List

List <T> або список позначає набір значень. В інших мовах програмування йому відповідають масиви. Визначимо список чисел:

```
1 var list = [1, 2, 3];
```

Також можна явно вказати тип:

```
1 List<int> list = [1, 2, 3];
```

При визначенні списку йому присвоюється перелік елементів в квадратних дужках.

Для звернення до елементів списку застосовуються індекси. Індексом є номер елемента в списку, при цьому нумерація починається з нуля, тому індекс першого елемента буде дорівнювати 0. А щоб звернутися до четвертого елемента у списку, необхідно використовувати індекс 3, наприклад: **list [3]**. Застосуємо індекси для отримання і встановлення значень елементів списку:

```
1 void main () {
2
3     var list = [2, 4, 6];
4     list[1] = 5;           // установка 2-го елемента
5     int d = list[2];       // отримання 3-го елемента
6     print(d);             // 6
7 }
```

Якщо спробувати звернутися до неіснуючого індексу, то зіткнемося з помилкою:

```
1 var list = [2, 4, 6];
2 list[6] = 5;
```

В наведеному прикладі в списку тільки 3 елементи, тому елемента з індексом 6 в списку не існує.

Для перебору елементів списку можна використовувати вбудовані цикли **for / while**:

```
1 void main () {
2
3     var list = [2, 4, 6];
4     for(int i=0; i < list.length; i++){
5         print(list[i]);
6     }
7 }
```

Тут змінна-лічильник **i** виконує роль індексу. Збільшення індексу буде відбуватися, поки він не досягне значення **list.length**, тобто довжини списку.

Але також можна використовувати спеціальну форму циклу для колекцій:

```
1 void main () {
2
3     var list = [2, 4, 6];
4     for (var n in list) {
5         print(n);
6     }
7 }
```

6.1.1. Фіксовані і нефіксовані списки

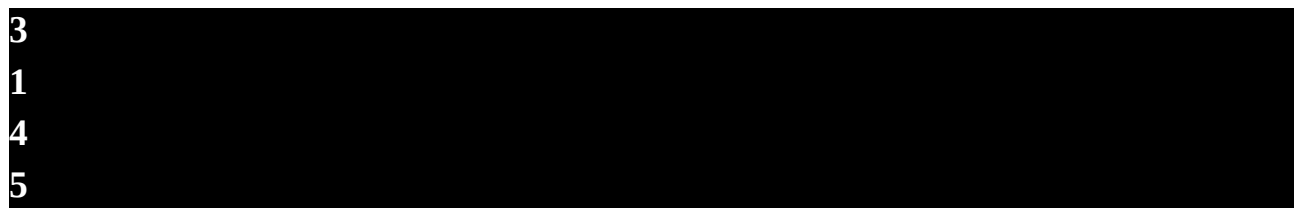
Списки можуть бути фіксованими (з жорстко визначеним розміром) і нефіксовані (можуть збільшуватися в розмірах). Приклади створення нефіксованих списків:

```
1 var list1 = [];
2 var list2 = [2, 4, 6];
```

Відмінною особливістю нефіксованих списків є те, що можна динамічно змінювати їх розмір, наприклад, за допомогою методу **add ()** додати нові елементи:

```
1 void main () {
2
3     var list1 = [3];
4     list1.add(1);
5     list1.add(4);
6     list1.add(5);
7     for (var n in list1) {
8         print(n);
9     }
10 }
```

Результат роботи програми показано на рисунку 6.1.



```
3
1
4
5
```

Рисунок 6.1. Результат роботи програми

Для створення фіксованого списку можна використовувати один з іменованих конструкторів класу **List**. Наприклад, конструктор **List.filled** (кількість_елементів, значення_за_замовчуванням) приймає два параметри. Перший параметр - кількість елементів у фіксованому списку. Другий параметр вказує на значення, яке матимуть всі елементи при створенні списку:

```
1 void main () {
2
3     var list1 = List.filled(4, 2); // фіксований список
    [2, 2, 2, 2]
4     list1[0] = 3;
5     list1[1] = 7;
6     list1[2] = 11;
7     list1[3] = 15;
8     //list1.add(1); // неможливо додати новий елемент
9
10    for (var n in list1) {
11        print(n);
12    }
13 }
```

Хоча список і фіксований, проте дозволено змінювати значення окремих елементів.

Ще один спосіб створення фіксованого списку забезпечує конструктор **List.generate()**:

```
1 List.generate(кількість_елементів, функція_наповнення,
    можливість_додавання_елементів)
```

Перший параметр вказує на кількість елементів. Другий параметр позначає функцію заповнення списку, в яку як параметр передається індекс елемента. Третій параметр - логічне значення - якщо **true**, то список нефіксованим (розширюваний), якщо **false**, то список фіксований.

Застосування конструктора **List.generate**:

```
1 void main () {
2     var list1 = List.generate(4, (int index) => index
    1, growable: false); // фіксований список [1, 2, 3, 4]
3
4     list1[0] = 3;
5     list1[1] = 7;
```

```

6      list1[2] = 11;
7      list1[3] = 15;
8      //list1.add(1); // неможливо додати новий елемент
9
10     for (var n in list1){
11         print(n);
12     }
13
14 }

```

6.1.2. Властивості і методи списків

Розглянемо основні властивості та методи списків в Dart.

Основні властивості списків:

- **first**: повертає перший елемент;
- **last**: повертає останній елемент;
- **length**: повертає довжину списку;
- **reversed**: повертає список, в якому всі елементи розташовані в протилежному порядку;
- **isEmpty**: повертає **true**, якщо список порожній.

Основні методи списків:

- **add (E value)**: додає елемент в кінець списку;
- **addAll (Iterable <E> iterable)**: додає в кінець списку інший список;
- **clear ()**: видаляє всі елементи зі списку;
- **indexOf (E element)**: повертає перший індекс елемента;
- **insert (int index, E element)**: вставляє елемент на певну позицію;
- **remove (Object value)**: видаляє об'єкт зі списку (видаляється тільки перше входження елемента до списку);
- **removeAt (int index)**: видаляє елементи за індексом;
- **removeLast ()**: видаляє останній елемент списку;
- **forEach (void f (E element))**: здійснює над елементами списку деяку дію, яка задається функцією-параметром, аналогічно циклу **for..in**;
- **sort ()**: сортує список;
- **sublist (int start, [int end])**: повертає частину списку від індексу **start** до індексу **end**;
- **contains (Object element)**: повертає **true**, якщо елемент міститься в списку;

- **join** ([String separator = ""]): об'єднує всі елементи списку в рядок. Можна вказати необов'язковий параметр **separator**, який буде розділяти елементи в рядку;

- **skip** (int count): повертає колекцію, в якій відсутні перші **count** елементів;

- **take** (int count): повертає колекцію, яка містить перші **count** елементів;

- **where** (bool test (E element)): повертає колекцію, елементи якої відповідають деякій умові, яка передається у вигляді функції.

Це тільки короткий перелік основних функціональних можливостей класу **List**. Повний список полів / методів / конструкторів наводиться в Документації¹⁾ [31].

Приклад застосування деяких полів і методів:

```
1 void main () {
2
3     var list = [-3, -1, 2, 4, 5, 7];
4     // отримаємо перший елемент
5     print(list.first); // -3
6     // отримаємо останній елемент
7     print(list.last); // 7
8     // отримаємо довжину списку
9     print(list.length); // 6
10
11     // додамо список
12     list.addAll([4, 6]);
13
14     // об'єднаємо елементи описка в рядок, розділювач
    кома
15     print(list.join(", ")); // -3, -1, 2, 4, 5, 7, 4,
    6
16
17     // відсортуємо список
18     list.sort();
19     print(list.join(", ")); // -3, -1, 2, 4, 4, 5, 6,
    7
```

¹⁾ <https://api.dart.dev/stable/2.4.0/dart-core/List-class.html>


```

20
21     // видалимо число 4
22     list.remove(4);
23     // видалимо другий елемент
24     list.removeAt(1);
25     print(list.join(", ")); // -3, 2, 4, 5, 6, 7
26
27     // отримаємо елементи більше за 2
28     list = list.where((element) => element >
29         2).toList();
30     print(list.join(", ")); // 4, 5, 6, 7
31 }

```

6.2. Set

Клас **Set** позначає невпорядкований набір унікальних об'єктів. Для створення **Set** застосовуються фігурні дужки {}:

```

1  var set = {1, 2, 3, 5};
2  // еквівалентні визначення Set
3  Set<int> set1 = {1, 2, 3, 5};
4  var set2 = <int> {1, 2, 3, 5};
5  Set<int> set3 = <int> {1, 2, 3, 5};

```

Для перебору набору можна використовувати цикл **for..in**:

```

1  void main () {
2
3      var intSet = {1, 2, 3, 5};
4
5      for (var n in intSet) {
6          print(n);
7      }
8
9      var stringSet = {"Tom", "Alice", "Kate"};
10     for (var s in stringSet) {
11         print(s);
12     }
13 }

```

Відмінною особливістю наборів є те, що вони мають містити тільки унікальні значення, тобто не можна додати одні й ті ж значення в набір кілька разів:

```
1 void main () {
2
3     var set = {1, 2, 3, 5};
4     set.add(3);           // додаємо один елемент - 3
5     set.addAll({4, 5});   // додаємо другий набір -
6                             {4, 5}
7     for (var n in set) {
8         print(n);         // 1 2 3 5 4
9     }
10 }
```

6.2.1. Властивості і методи класу Set

Розглянемо основні властивості та методи класу **Set**.

Основні властивості наборів:

- **first**: повертає перший елемент;
- **last**: повертає останній елемент;
- **length**: повертає довжину набору;
- **isEmpty**: повертає **true**, якщо набір порожній.

Основні методи наборів:

- **add (E value)**: додає елемент в набір;
- **addAll (Iterable <E> iterable)**: додає в набір іншу колекцію;
- **clear ()**: видаляє всі елементи з набору;
- **difference (Set <Object> other)**: повертає різницю поточного набору і набору **other** у вигляді іншого набору;
- **intersection (Set <Object> other)**: повертає перетин поточного набору і набору **other** у вигляді іншого набору;
- **remove (Object value)**: видаляє об'єкт з набору;
- **removeAll (Iterable <Object> elements)**: видаляє з набору всі елементи колекції **elements**;
- **union (Set <E & g> other)**: повертає об'єднання двох наборів - поточного і набору **other**;
- **contains (Object element)**: повертає **true**, якщо елемент міститься в наборі;

- **join** ([String separator = ""]): об'єднує всі елементи набору в рядок. Можна вказати необов'язковий параметр **separator**, який буде розділяти елементи в рядку;

- **skip** (int count): повертає колекцію, в якій відсутні перші **count** елементів;
- **take** (int count): повертає колекцію, яка містить перші **count** елементів;
- **where** (bool test (E element)): повертає колекцію, елементи якої відповідають деякій умові, яка передається у вигляді функції.

Повний список полів / методів / конструкторів класу **Set** наводиться в Документації¹⁾ [32].

Приклади застосування деяких полів і методів:

```
1 void main () {
2
3     var set = {-3, -1, 2, 4, 5, 7};
4     // отримуємо перший елемент
5     print(set.first);    // -3
6     // отримуємо останній елемент
7     print(set.last);    // 7
8     // отримуємо довжину набору
9     print(set.length);  // 6
10
11    // додаємо набір
12    set.addAll({4, 2, 6});
13
14    // об'єднуємо елементи набору в рядок, розділювач
15    print(set.join(", ")); // -3, -1, 2, 4, 5, 7, 6
16
17    // видаляємо число 4
18    set.remove(4);
19
20    print(set.join(", ")); // -3, -1, 2, 5, 7, 6
21
22    // отримуємо елементи більше за 2
```

¹⁾ <https://api.dart.dev/stable/2.4.0/dart-core/Set-class.html>

```

23     set = set.where((element) => element > 2).toSet();
24     print(set.join(", ")); // 5, 6, 7
25 }

```

6.3. Map

Клас **Map** позначає колекцію елементів, де кожен елемент має ключ і значення. Ключ і значення елемента можуть бути різних типів.

Для створення об'єкта **Map** можуть використовуватися фігурні дужки {}, всередині яких містяться пари ключ-значення, або конструктор класу **Map**. Способи створення **Map** наступні:

```

1  var map = {
2      1: "Tom",
3      2: "Bob",
4      3: "Sam"
5  };
6  // еквівалентне визначення Map
7  Map<int, String> map2 = {
8      1: "Tom",
9      2: "Bob",
10     3: "Sam"
11 };

```

В наведеному прикладі обидва об'єкти **Map** мають тип **Map <int, String>** - як тип ключів використовується тип **int**, а в якості типу значень - тип **String**.

У фігурних дужках визначаються три елементи в форматі **ключ: значення**. Тобто в наведеному прикладі є три ключа - 1, 2, 3, з якими зіставляються три значення - рядки "Tom", "Bob", "Sam".

Використовуючи ключі, можна отримати або змінити значення елементів:

```

1  void main () {
2
3      var map = {
4          1: "Tom",
5          2: "Bob",
6          3: "Sam"
7      };
8      map[2] = "Alice"; // встановлюємо значення
                           елемента з ключем 2

```

```

9      print(map[2]);      // отримуємо значення елемента з
    ключем 2
10 }

```

Подібним чином можна створювати об'єкти **Map** з іншими типами ключів і значень. Розглянемо як приклад об'єкт **Map <String, String>**, у якого ключі і значення мають тип **String**:

```

1  void main () {
2
3      var countries = {
4          "Spain" : "Madrid",
5          "France" : "Paris",
6          "Germany": "Berlin"
7      };
8      print(countries["Spain"]);      // Madrid
9      print(countries["Germany"]);    // Berlin
10     print(countries["France"]);     // Paris
11 }

```

За допомогою конструктора **Map** можна створити порожній об'єкт **Map** і потім додавати в нього об'єкти:

```

1  void main () {
2
3      var map = Map();      // порожній Map
4      map[1] = "Alice";
5      map[44] = "Bob";
6      map[6] = "Sam";
7
8  }

```

Додавання нового елемента здійснюється наступним чином: в квадратних дужках вказується ключ і присвоюється йому якесь значення (**map [6] = "Sam";**)

6.3.1. Властивості і методи Map

Розглянемо основні властивості та методи класу **Map**.

Основні властивості:

- **entries**: повертає об'єкт **Iterable <MapEntry <K, V >>**, який позначає всі елементи **Map**;
- **keys**: повертає об'єкт **Iterable <K>**, який позначає всі ключі **Map**;
- **values**: повертає об'єкт **Iterable <V>**, який позначає всі значення **Map**;

- **length**: повертає кількість елементів в **Map**;
- **isEmpty**: повертає **true**, якщо **Map** порожній.

Кожен елемент в **Map** фактично є об'єктом **MapEntry** <K, V>, де **K** - тип ключів, а **V** - тип значень. У цього типу є дві властивості: **key** (ключ елемента) і **value** (зберігає значення елемента). Використовуючи ці властивості, можна отримати вміст елементів у **Map**. Розглянемо приклад, де перебираються всі елементи і окремо всі ключі і значення **Map**:

```

1  void main () {
2
3      var map = { 5 : "Tom" };
4      map[1] = "Alice";
5      map[44] = "Bob";
6      map[6] = "Sam";
7
8      // перебір всіх елементів
9      print("All items");
10     for(var item in map.entries){
11         // item позначає MapEntry<K, V>
12         print("${item.key} - ${item.value}");
13     }
14
15     // перебір ключів
16     print("Keys");
17     for(var key in map.keys) {
18
19         print(key);
20     }
21
22     // перебір значень
23     print("Values");
24     for(var value in map.values) {
25
26         print(value);
27     }
28 }

```

Результат роботи програми показано на рисунку 6.2.

All items

5 - Tom

1 - Alice

44 - Bob

6 - Sam

Keys

5

1

44

6

Values

Tom

Alice

Bob

Sam

Рисунок 6.2. Результат роботи програми

Основні методи Map:

- **addAll (Map <K, V> other):** додає в **Map** інший об'єкт **Map**;
- **addEntries (Iterable <MapEntry <K, V >> newEntries):** додає в **Map** колекцію **Iterable <MapEntry <K, V >>**;
- **clear ():** видаляє всі елементи з **Map**;
- **containsKey (Object key):** повертає **true**, якщо **Map** містить ключ **key**;
- **containsValue (Object value):** повертає **true**, якщо **Map** містить значення **value**;
- **remove (Object key):** видаляє з **Map** елемент з ключем **key**.

Приклади застосування методів:

```
1 void main () {  
2  
3     var map = { "Germany" : "Berlin"};  
4     map.addAll ({ "France" : "Paris", "Spain":  
        "Madrid"});    // додаємо Map
```

```

5
6     print(map.containsKey("Poland"));    // false
7     print(map.containsKey("France"));    // true
8
9     print(map.containsValue("Madrid"));  // true
10
11     // видаляємо елемент з ключем "Spain"
12     map.remove("Spain");
13     print(map.containsKey("Spain"));    // false
14     print(map.containsValue("Madrid")); // false
15 }

```

Резюме

В цьому розділі було розглянуто питання використання вбудованих в мову Dart колекцій.

Розглянуто способи визначення та використання таких колекцій як List, Set та Map, їх основних властивостей та методів.

Показано яким чином визначаються та використовуються такі колекції як List, Set та Map при написанні програм.

Контрольні запитання і завдання

1. Що таке клас **List** у Dart? Які його аналоги в інших мовах програмування?
2. Як здійснюється звернення до елементів списку при операціях читання та запису?
3. Який зміст мають фіксовані та нефіксовані списки?
4. Чи можна у фіксованому списку змінювати значення окремих елементів?
5. Які основні властивості списків?
6. Які основні методи списків?
7. Що таке клас **Set** у Dart? Як здійснюється звернення до елементів набору при операціях читання та запису?
8. Які відмінні особливості притаманні наборам?
9. Які основні властивості наборів?
10. Які основні методи наборів?
11. Що таке клас **Map** у Dart?

12. Як здійснюється звернення до елементів **Map** при операціях читання та запису?
13. Які основні властивості **Map**?
14. Які основні методи **Map**?
15. Наведіть приклади створення списку та звернення до його елементів.
16. Наведіть приклади створення нефіксованого списку та звернення до його елементів.
17. Наведіть приклади створення фіксованого списку та звернення до його елементів.
18. Наведіть приклади використання основних властивостей списків.
19. Наведіть приклади використання основних методів списків.
20. Наведіть приклади створення набору та звернення до його елементів.
21. Наведіть приклади використання основних властивостей наборів.
22. Наведіть приклади використання основних методів наборів.
23. Наведіть приклади створення **Map** та звернення до його елементів.
24. Наведіть приклади використання основних властивостей **Map**.
25. Наведіть приклади використання основних методів **Map**.

7. ОБРОБКА ВИНЯТКІВ

7.1. Блок `try..catch`

В процесі роботи програм можуть виникати різні помилки, які порушують звичний хід програми і навіть змушують її перервати виконання. Такі помилки називаються винятками. Мова Dart, як і багато мов програмування, підтримує обробку винятків, що дозволяє уникати аварійного переривання роботи програми.

Розглянемо найпростішу ситуацію:

```
1 void main () {  
2  
3     int x = 9;  
4     int y = 0;  
5     int z = x ~/ y;  
6  
7     print(z);  
8 }
```

В наведеному прикладі відбувається цілочисельне ділення (операція `~/`) на 0. У результаті програма згенерує виняток і завершить своє виконання. Ситуація може здатися штучною, так як тут вочевидь видно, що одна із змінних дорівнює 0, і природно що програма завершиться з помилкою. Однак в реальності можна отримувати дані із зовнішнього джерела і точно не знати, яке там значення 0 або не 0. Крім того, діленням на нуль, винятки в Dart не обмежуються, є багато ситуацій, де можлива подібна аварійна зупинка роботи програми.

Розглянемо, що пропонує Dart для обробки подібних ситуацій. Перш за все це конструкція **`try ... on ... catch ... finally`**:

```
1 try{  
2     // Код, який може призвести до генерації винятку  
3 }  
4 on Тип_Винятка{  
5     // Обробка отриманого винятку  
6 }  
7 catch (e){  
8     // Обробка отриманого винятку  
9 }  
10 finally{  
11     //  
12 }
```

При використанні блоку **`try..on..catch..finally`** спочатку виконуються всі інструкції в блоці **`try`**. Якщо в цьому блоці не виникло винятків, то після його

виконання починає виконуватися блок **finally**. І потім конструкція **try..on..catch..finally** завершує свою роботу.

Якщо ж в блоці **try** раптом виникає виняток, то звичайний порядок виконання зупиняється, а виконання програми переходить до блоку **on** або **catch**, який може обробити отриманий виняток. Якщо потрібний блок **on** або блок **catch** знайдений, то він виконується, і після його завершення виконується блок **finally**.

Різниця між операторами **on** і **catch** полягає в тому, що **on** обробляє винятки певного типу (докладніше оператор **on** розглядається в наступній темі). А **catch** обробляє всі винятки.

Якщо потрібний блок **on** або блок **catch** не знайдений, то при отриманні винятку знову ж виконується блок **finally**, а потім програма аварійно завершує своє виконання.

Застосуємо конструкцію **try..on..catch..finally** для обробки ділення на 0:

```
1 void main () {
2
3     try{
4         int x = 9;
5         int y = 0;
6         int z = x ~/ y;
7
8         print(z);
9     }
10    catch(e) {
11        print("Отримано виняток $e");
12    }
13    print("Завершення програми");
14 }
```

Тут код, який може згенерувати виняток, розташовано в блок **try**. Блок **catch** в дужках містить список параметрів, які передаються при генерації винятку. В наведеному прикладі це один параметр - **(e)**, який позначає інформацію про виняток.

І при виконанні цієї програми буде отримано результат, як показано на рисунку 7.1.

Отримано виняток IntegerDivisionByZeroException
Завершення програми

Рисунок 7.1. Результат роботи програми

Тут видно, що незважаючи на генерацію винятку, програма не завершилася аварійно, а після виконання блоку **catch** продовжила свою роботу.

При визначенні конструкції можна опускати деякі блоки, якщо вони не потрібні. Тільки блок **try** є обов'язковим. Однак разом з ним повинен також використовуватися, як мінімум, або блок **on i** (або) **catch**, або блок **finally**. Так, в прикладі вище використовувались тільки блоки **try** і **catch**. Додамо блок **finally**:

```
1  void main () {
2
3      try{
4          int x = 9;
5          int y = 0;
6          int z = x ~/ y;
7
8          print(z);
9      }
10     catch(e) {
11         print("Отримано виняток $e");
12     }
13     finally{
14         print("Виконання блоку finally");
15     }
16     print("Завершення програми");
17
18 }
```

Результат роботи програми показано на рисунку 7.2.



Отримано виняток IntegerDivisionByZeroException
Завершення програми
Виконання блоку finally

Рисунок 7.2. Результат роботи програми

В наведеному прикладі можна опустити блок **catch**:

```

1  void main () {
2
3      try{
4          int x = 9;
5          int y = 0;
6          int z = x ~/ y;
7
8          print(z);
9      }
10     finally{
11         print("Виконання блоку finally");
12     }
13     print("Завершення програми");
14
15 }

```

Після генерації винятку в блоці **try** виконується блок **finally**, після чого програма аварійно зупиняє свою роботу.

7.1.1. Параметри блоку **catch**

У блок **catch** передаються деякі параметри, які несуть інформацію про виняток і можна за необхідності отримати і використовувати цю інформацію. Розглянемо наступний приклад:

```

1  try{
2      int x = 9;
3      int y = 0;
4      int z = x ~/ y;
5
6      print(z);
7  }
8  catch(e, s){
9      print("Обробка винятку $e");
10     print("Стек $s");
11 }

```

Перший параметр блоку **catch** - **e** позначає об'єкт винятку, а другий - **s** - стек винятків, який дозволяє зрозуміти, де відбувся виняток.

7.2. Класи винятків

Ключовий клас для обробки винятків у мові Dart - це клас **Exception**, який є базовим класом для всіх інших типів винятків. За замовчуванням стандартна бібліотека надає ряд вбудованих типів винятків. Наприклад, в прикладі, наведеному вище, виконувалося ділення на нуль. Це призводило до генерації винятку типу **IntegerDivisionByZeroException**, вбудованого класу, який успадковується від **Exception**.

У конструкції **try..catch** можна обробляти винятки певного типу. Для цього застосовується оператор **on**:

```
1  void main () {
2
3      try{
4          int x = 9;
5          int y = 0;
6          int z = x ~/ y;
7
8          print(z);
9      }
10     on Exception{
11         print("Обробка винятку у блоці on");
12     }
13     print("Завершення програми");
14
15 }
```

Після оператора **on** вказується тип винятку, який потрібно обробити. В наведеному прикладі вираз **on Exception** позначає, що обробляється виняток типу **Exception**. А оскільки це базовий тип для всіх винятків, то фактично будуть оброблюватися всі винятки.

Але також можна конкретизувати тип винятку. Розглянемо наступний приклад:

```
1  void main () {
2
3      try{
4          int x = 9;
5          int y = 0;
6          int z = x ~/ y;
7
```

```

8         print(z);
9     }
10     on IntegerDivisionByZeroException{
11         print("Обробка винятку
IntegerDivisionByZeroException");
12     }
13     print("Завершення програми");
14
15 }

```

Тут виконується обробка винятку тільки типу **IntegerDivisionByZeroException**. Якщо в блоці **try** раптом виникнуть винятки інших типів, то вони не будуть оброблені.

Обробку конкретних типів винятків можна здійснювати разом з блоком **catch**:

```

1 void main () {
2
3     try{
4         int x = 9;
5         int y = 0;
6         int z = x ~/ y;
7
8         print(z);
9     }
10    on IntegerDivisionByZeroException{
11        print("Обробка винятку
IntegerDivizionByZeroException");
12    }
13    catch(e) {
14        print("Отримано виняток: $e");
15    }
16    print("Завершення програми");
17
18 }

```

В цьому прикладі, якщо виникне виняток типу **IntegerDivisionByZeroException**, то його буде обробляти блок **on**, якщо виняток іншого типу - то блок **catch**.

Також можна об'єднувати блоки **on** і **catch**:

```

1  void main () {
2
3      try {
4          int x = 9;
5          int y = 0;
6          int z = x ~/ y;
7
8          print(z);
9      }
10     on IntegerDivisionByZeroException catch (e, s) {
11         print("Обробка винятку $e");
12         print("Стек $s");
13     }
14     print("Завершення програми");
15 }

```

7.2.1. Вбудовані типи винятків

Розглянемо вбудовані в Dart типи винятків. Станом на зараз в Dart є такі типи винятків:

1. DeferredLoadException;
2. FormatException;
3. IntegerDivisionByZeroException;
4. IOException;
5. FileSystemException;
6. HttpException;
7. ProcessException;
8. SignalException;
9. SocketException;
10. StdinException;
11. StdoutException;
12. TlsException;
13. WebSocketException;
14. IsolateSpawnException;
15. TimeoutException.

Для кожного типу винятку, який необхідно обробити, можна додати свій блок **on**:


```

1  void main () {
2
3      try {
4          int x = 9;
5          int y = 0;
6          int z = x ~/ y;
7          print(z);
8      }
9      on IntegerDivisionByZeroException {
10         print("Обробка винятку
IntegerDivisionByZeroException");
11     }
12     on FormatException {
13         print("Обробка винятку FormatException");
14     }
15     print("Завершення програми");
16
17 }

```

7.3. Оператор throw і створення своїх типів винятків

7.3.1. Оператор throw

Оператор **throw** дозволяє створювати і генерувати власні винятки в необхідному місці. В якості прикладу розглянемо наступний код:

```

1  void main () {
2
3      try {
4          Person tom = Person("Tom", -45);
5          print(tom.age);
6      }
7      on Exception catch (e) {
8          print(e);
9      }
10     print("Завершення програми");
11

```

```

12 }
13
14 class Person{
15     String name;
16     int age = 1;
17
18     Person(this.name, age){
19         if(age < 1 || age > 110) {
20             throw Exception("Неприпустимий вік");
21         }
22         else{
23             this.age = age;
24         }
25     }
26 }

```

Тут, при створенні нового користувача - об'єкта класу **Person** необхідно попередити ситуацію, коли в конструктор для віку передається неприпустиме (наприклад, негативне значення). У цьому випадку можна за допомогою оператора **throw** згенерувати виняток:

```

1 throw Exception("Неприпустимий вік");

```

У конструктор **Exception** передається повідомлення, яке можна отримати при обробці винятку.

Таким чином, при виконанні наступного рядка в блоці **try** виникатиме виняток:

```

1 Person tom = Person("Tom", -45);

```

А в блоці **on..catch** можна обробити цей виняток:

```

1 on Exception catch (e) {
2     print(e);
3 }

```

За допомогою аргументу **e** в блоці **catch** можна отримати повідомлення про помилку. А результат роботи програми показано на рисунку 7.2.

Exception: Неприпустимий вік
Завершення програми

Рисунок 7.3. Результат роботи програми

7.3.2. Створення власних винятків

Вбудована бібліотека Dart забезпечує великою кількістю різних типів винятків. Проте для якихось окремих ситуацій, можливо, знадобиться створення власних типів винятків. Наприклад, в прикладі вище з віком користувача можна всі дії оформити у вигляді окремого винятку. Наприклад:

```
1  void main () {
2
3      try{
4          Person tom = Person("Tom", -45);
5          print(tom.age);
6      }
7      on AgeException catch(e) {
8          print(e.getErrorMessage());
9      }
10     print("Завершення програми");
11
12 }
13
14 class Person{
15     String name;
16     int age = 1;
17
18     Person(this.name, age){
19         if(age < 1 || age > 110) {
20             throw AgeException();
21         }
22         else{
23             this.age = age;
24         }
25     }
26 }
27 class AgeException implements Exception{
28     String getErrorMessage() {
29         return "Неприпустимий вік";
30     }
31 }
```

Результат роботи програми показано на рисунку 7.4.

Неприпустимий вік Завершення програми

Рисунок 7.4. Результат роботи програми

Тут визначено клас винятку **AgeException**, який реалізує інтерфейс типу **Exception**. (Фактично можна визначити клас без реалізації **Exception**). І в цьому класі визначений власний метод **getErrorMessage**, який повертає повідомлення про помилку.

При обробці винятку можна використовувати цей метод для виведення повідомлення про помилку:

```
1  on AgeException catch(e) {  
2      print(e.getErrorMessage());  
3  }
```

В наведеному прикладі параметр **e** в **catch** позначає об'єкт **AgeException**, у якого можна викликати метод **getErrorMessage**.

Резюме

В цьому розділі було розглянуто питання обробки вбудованих в мову Dart винятків.

Розглянуто способи визначення та використання власних винятків.

Показано яким чином визначаються та використовуються вбудовані та власні винятки при написанні програм.

Контрольні запитання і завдання

1. Що таке винятки у мові Dart?
2. Які механізми є у Dart для обробки аварійних ситуацій?
3. В чому полягає різниця між операторами **on** і **catch**?
4. Чи дозволено у Dart при визначенні конструкції обробки винятків опускати деякі блоки? Який блок є обов'язковим?
5. Що таке клас винятків? Які вбудовані типи винятків є у Dart?
6. Чи можна у Dart створювати власні винятки?
7. Який оператор застосовується для створення власного винятку?
8. Наведіть приклад обробки винятку.
9. Наведіть приклад де проявляється різниця між операторами **on** і **catch**.
10. Наведіть приклад створення власного винятку.

8. АСИНХРОННІСТЬ

8.1. Вступ до асинхронності і Future

Мова Dart, як і багато сучасних мов програмування, має підтримку асинхронності. Розглянемо навіщо потрібна іще асинхронність. Візьмемо, наприклад, графічний застосунок на Android. При запуску програми на Android вона запускається в основному потоці призначеного для користувача інтерфейсу. Цей основний потік обробляє всі події призначеного для користувача інтерфейсу, зокрема, натискання кнопок, дотику екрану і т.і. Користувач може натискати на кнопки, вводити текст в текстові поля, і в основному потоці все буде оброблятися. Однак робота застосунків, як правило, цим не обмежується. Наприклад, застосунок для електронної пошти може одночасно відправляти запити по мережі - відправляти раніше написані листи або звертатися до сервера для перевірки наявності нових листів. Запит по мережі може займати багато часу, особливо в умовах нестабільної роботи Інтернету. Однак подібні мережеві запити зазвичай відправляються таким чином, щоб не блокувати основний потік, призначеного для користувача інтерфейсу. Якби мережеві запити проводилися б в основному потоці, то користувачеві довелося б чекати, поки відправиться мережевий запит і буде отримано його результат, а під час запиту застосунок блокувався б і чекав би закінчення запиту. Однак у багатьох нормальних застосунках так не відбувається - користувач може відправити лист і відразу почати писати новий лист без сильних зависань застосунку - тому що мережевий запит здійснюється асинхронно. А коли він буде виконаний, користувач зможе просто побачити якесь повідомлення. В цьому і полягає перевага застосування асинхронності. Причому це стосується не тільки мережевих запитів, а взагалі всіх великовагових завдань в застосунку, які можуть займати тривалий час.

Що стосується безпосередньо мови Dart, то це однопотокова мова - вона може виконувати тільки одну задачу в один момент часу. Проте завдяки реалізації циклу подій (**event loop**) і двох черг подій (**event queue** і **microTask queue**) вона дозволяє асинхронно виконувати різні завдання. Зокрема, коли запускається будь-який застосунок на Dart, починає виконуватися єдиний потік застосунку.

Він ініціалізує дві черги - **MicroTask** і **Event**, які будуть містити завдання, які необхідно буде виконати.

Далі потік запускає функцію **main ()**, а після її завершення - цикл подій або **Event Loop**. У цьому циклі послідовно витягуються і виконуються всі завдання з черг **MicroTask** і **Event**.

Черга **MicroTask** призначена для зберігання невеликих внутрішніх завдань, які повинні виконуватися асинхронно.

Основна частина завдань розташовується у черзі **Event** - це зовнішні події, наприклад, події графічного інтерфейсу (наприклад, натискання кнопки в мобільних застосунках), таймери, читання-запис файлів, робота з потоками і т.д., а також об'єкти **Future**.

8.1.1. Клас **Future**

Ключовим класом є клас **Future**. Клас **Future** позначає результат відкладеної операції, яка завершить своє виконання в майбутньому. Результатом операції може бути деяке значення або помилка.

Об'єкт **Future** може перебувати в двох станах: незавершеному (**Uncompleted**) і завершеному (**Completed**). У незавершеному стані операція, яку позначає об'єкт **Future** визначена, можливо, вже почалася виконуватися, але результат ще не отримано. У завершеному стані операція вже завершила своє виконання, її результат - якесь значення або помилка - отримано.

Розглянемо найпростіший приклад:

```
1 Future getMessage() {
2     // для емуляції довготривалої операції встановлюємо
   затримку у 3 секунди
3     return Future.delayed(Duration(seconds: 3), () =>
   print("Прийшло нове повідомлення від Тома"));
4 }
5
6 void main() {
7     getMessage();
8     print("Перевірка повідомлень...");
9 }
```

Отже, тут визначена функція **getMessage**, яка повертає об'єкт **Future**. У цій функції викликається один з іменованих конструкторів класу **Future** - **Future.delayed()**. Цей конструктор приймає два параметри. Перший параметр - це об'єкт **Duration**, який встановлює час затримки (**Duration(seconds: 3)**) в наведеному прикладі становить три секунди. Другий параметр - це деяка функція, яка виконує будь-які дії - в наведеному прикладі це просто друкування повідомлення на консоль.

```
1 () => print("Прийшло нове повідомлення від Тома ")
```

Таким чином, через три секунди у консолі буде надруковано рядок «Прийшло нове повідомлення від Тома». Це по суті і є відкладеною операцією, яку відображає об'єкт **Future**.

У функції **main** викликається функція **getMessage** і потім виводиться на консоль деяке повідомлення. Звернемо увагу, що функція **getMessage** викликається першою. А результат роботи програми показано на рисунку 8.1.

Перевірка повідомлень ...

Прийшло нове повідомлення від Тома

Рисунок 8.1. Результат роботи програми

Фактично можна побачити, що функція **getMessage** завершується тільки після завершення всіх інших дій у функції **main**. Тобто функція **getMessage**, яка виконується 3 секунди, виконується асинхронно. При виклику, функція **getMessage** повертає об'єкт **Future**, який знаходиться в незавершеному стані. І тільки через три секунди після виконання функції з конструктора **Future.delayed** він перейде в завершений стан.

Механіка роботи **Future** така, що при визначенні нового об'єкта **Future**, операція, яку він позначає (наприклад, в наведеному прикладі отримання повідомлення), потрапляє в **event queue**. Потім після виконання функції **main** в циклі **Event Loop** ця операція витягується з **Event Queue** і виконується.

8.2. Робота з класом Future

Ключовим класом для написання асинхронного коду в Dart є клас **Future**. Розглянемо, які можливості він надає.

8.2.1. Конструктори Future

Для створення об'єкта **Future** можна використовувати один з його конструкторів:

1. **Future (FutureOr <T> computation ())**: створює об'єкт **future**, який за допомогою методу **Timer.run** запускає функцію **computation** асинхронно і повертає її результат;

2. Тип **FutureOr <T>** вказує, що функція **computation** повинна повертати або об'єкт **Future <T>** або об'єкт типу **T**. Наприклад, щоб отримати об'єкт **Future <int>**, функція **computation** повинна повертати або об'єкт **Future <int>**, або об'єкт **int**;

3. **Future.delayed (Duration duration, [FutureOr <T> computation ()])**: створює об'єкт **Future**, який запускається після певної часової затримки,

зазначеної через перший параметр **Duration**. Другий необов'язковий параметр вказує на функцію, яка запускається після цієї затримки;

4. **Future.error (Object error, [StackTrace stackTrace])**: створює об'єкт **Future**, який містить інформацію про помилку, яка виникла;

5. **Future.microtask (FutureOr <T> computation ())**: створює об'єкт **Future**, який за допомогою функції **scheduleMicrotask** запускає функцію **computation** асинхронно і повертає її результат;

6. **Future.sync (FutureOr <T> computation ())**: створює об'єкт **Future**, який містить результатах функції **computation**, яка викликається негайно;

7. **Future.value ([FutureOr <T> value])**: створює об'єкт **Future**, який містить значення **value**.

Розглянемо застосування деяких конструкторів. В якості прикладу розглянемо використання першого конструктора:

```
1 void main() {
2     Future myFuture = Future(() {
3         print("Hello Future");
4     });
5
6     print("Main ends");
7 }
```

Тут в конструктор **Future** передається анонімна функція, яка не приймає параметрів і нічого не повертає, а просто виводить деяке повідомлення. Хоча тут застосовується анонімна функція, але також можна зробити її повноцінною функцією. Результат роботи програми показано на рисунку 8.2.



```
Main ends
Hello Future
```

Рисунок 8.2. Результат роботи програми

Застосування іменованого конструктора **Future.delayed()** аналогічно за тим винятком, що він очікує певний час (заданий першим параметром) перед тим, як перейти до виконання функції з другого параметра:


```

1  void main() {
2      Future future = Future.delayed(Duration(seconds: 3),
        () => print("Hello Future"));
3      print("Main ends");
4  }

```

Розглянемо застосування конструктора **Future.value ()**, який може використовуватися, якщо вже відомо значення, яке буде містити об'єкт **Future**. Цей конструктор може бути корисний, зокрема, при створенні вебсервісів, які використовують кешування.

```

1  Future future = Future.value(35);

```

В наведеному прикладі **Future** отримує число 35 (хоча це може бути будь-який об'єкт). І потім в програмі можна отримати це значення з **future**.

Розглянемо приклад застосування конструктора **Future.error ()**, який приймає об'єкт помилки і необов'язковий параметр - стек трасування:

```

1  Future future =
    Future.error(ArgumentError.notNull("input"));

```

В цьому прикладі в якості об'єкта помилки використовується вираз **ArgumentError.notNull("input")**, який повідомляє, що аргумент **input** не повинен бути рівний **null**.

8.2.2. Отримання значення

В реальності тип **Future** - це **generic-тип** або узагальнений тип, який описується певним типом - **Future<T>**; **T** в кутових дужках якраз і позначає тип значення, яке містить **Future**.

Розглянемо повторно один з вище наведених прикладів:

```

1  void main() {
2      Future future = Future(() {
3          print("Hello Future");
4      });
5
6      print("Main ends");
7  }

```

Тут функція, яка передається в конструктор, нічого не повертає, а просто виводить деяке повідомлення на консоль. Тому в реальності тут отримується не просто об'єкт **Future**, а **Future<void>**. Тобто можна також написати наступним чином:

```
1 Future<void> future = Future(() {  
2     print("Hello Future");  
3 });
```

Змінімо функцію. Нехай тепер вона повертає рядок:

```
1 Future<String> myFuture = Future(() {  
2     return "Hello Future";  
3 });
```

Оскільки функція повертає рядок, то об'єкт **Future** буде містити об'єкт **String** і буде позначати тип **Future <String>**.

Те ж саме стосується і інших конструкторів, наприклад:

```
1 void main() {  
2     Future<double> future1 = Future(() {  
3         return 23.5;    // повертає число double  
4     });  
5     Future<String> future2 =  
6     Future.delayed(Duration(seconds: 3),  
7         () => "Hello  
8     Future");    // повертає рядок  
9     Future<int> future3 =  
10    Future.value(35);    // зберігає число int  
11    Future<String> future4 = Future.value("Hello  
12    Dart");    // зберігає рядок  
13  
14    print("Main ends");  
15 }
```

8.2.3. Отримання значення і використання Future

Вище було зазначено кілька об'єктів **Future**. При їх визначенні всі вони знаходяться в незавершеному стані, їм ще тільки належить перейти до виконання. Розглянемо як же відстежити той момент, коли вони вже виконані і

перейшли в завершений стан. Для цього у **Future** визначено метод **then**. Цей метод приймає функцію зворотного виклику, яка буде спрацьовувати при завершенні **Future**. В якості єдиного параметра функція приймає його значення.

Розглянемо наступний приклад:

```
1 void main() {
2     Future<String> future = Future(() {
3         print("Виконання Future");
4         return "Hello Future"; // повертаємо рядок
5     });
6
7     future.then((value) {
8         print("Із Future отримано значення: $value");
9     });
10
11     print("Main ends");
12 }
```

В наведеному прикладі функція, яка передається в конструктор **Future**, повертає рядок, тому **future** позначає тип **Future <String>**.

У метод **then()** передається функція зворотного виклику, яка приймає параметр **value** - це і буде той рядок, який повертається при виконанні конструктора. Сама функція в **then** просто виводить це отримане значення на консоль. У підсумку отримаємо результат роботи програми, як показано на рисунку 8.3.

Main ends

Виконання Future

Із Future отримано значення: Hello Future

Рисунок 8.3. Результат роботи програми

Подібним чином можна отримувати значення інших типів - **int**, **double** і т.і.

Може виникнути питання, як бути з типом **void**. В цьому випадку замість параметра вказується нижнє підкреслення (**_**):

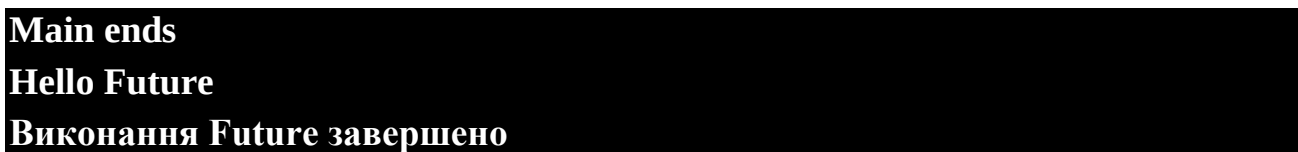
```
1 void main() {
2     Future<void> future =
3     Future.delayed(Duration(seconds: 3),
4         () => print("Hello
5     Future")); // тип, який повертається - void
```

```

4
5     future.then((_) {    // нижнє підкреслення
    замість параметра для типу void
6         print("Виконання Future завершено");
7     }
8 );
9
10    print("Main ends");
11 }

```

Результат роботи програми показано на рисунку 8.4.



```

Main ends
Hello Future
Виконання Future завершено

```

Рисунок 8.4. Результат роботи програми

За необхідності, можна створювати ланцюжок методів **then**, які будуть виконуватися один за одним:

```

1  void main() {
2
3      Future<String> future = Future(() {
4          print("Виконується Future");
5          return "Hello Future";    // повертаємо рядок
6      });
7
8      future.then((value) {
9          print("Із Future отримано значення: $value");
10         return 22;
11     })
12     .then((value) => print("Виконання Future заверше
13 Отримано: $value") );
14
15    print("Main ends");
16 }

```

Функція з першого методу **then** отримує з **Future** результат - рядок і виводить його на консоль. Крім того, вона повертає новий результат - для наведеного прикладі це число 22.

Із функції з другого методу **then** отримується саме це число, а не початкове значення з **Future**. Крім того, параметр функції з кожного наступного методу **then** - це результат функції з попереднього методу **then**. Результат роботи програми показано на рисунку 8.5.

Main ends

Виконується Future

Із Future отримано значення: Hello Future

Виконання Future завершено. Отримано: 22

Рисунок 8.5. Результат роботи програми

8.2.4. Обробка помилок

При виконанні операції, яку позначає **Future**, може виникнути помилка. В цьому випадку при переході в завершений стан **Future** замість конкретного значення буде зберігати інформацію про помилку. Для отримання інформації про помилку можна використовувати метод **catchError()**, який працює аналогічно методу **then** - також приймає функцію зворотного виклику, в яку передається інформація про помилку. Розглянемо наступний приклад:

```
1  void main() {
2
3      Future<String> future = Future(() {
4          print("Виконується Future");
5          throw "Непередбачена помилка";
6      });
7
8      future.then((value) {
9          print("Із Future отримано значення: $value");
10     })
11     .catchError((err) => print(err) );
12
13     print("Чекаємо отримання значення із Future");
14 }
```

В наведеному прикладі в конструкторі **Future** емулюємо генерацію помилки за допомогою оператора **throw** (**throw "Непередбачена помилка";**). Як об'єкт помилки тут застосовується звичайний рядок ("Непередбачена помилка") - це і є те, що передається у функцію в **catchError** через параметр **err**.

У підсумку, в цьому разі функція з методу **then** не спрацює, а спрацює функція з **catchError**, оскільки в процесі виконання **Future** виникла помилка. Результат роботи програми показано на рисунку 8.6.

Чекаємо отримання значення із Future

Виконується Future

Непередбачена помилка

Рисунок 8.6. Результат роботи програми

В якості другого і необов'язкового параметра метод **catchError** приймає функцію, яка перевіряє відповідність об'єкта помилки деякій умові і, якщо об'єкт помилки відповідає цій умові, то повертається **true**, інакше повертається **false**. Якщо повертається **false**, тобто об'єкт помилки НЕ відповідає умові, то функція з першого параметра НЕ виконується.

Розглянемо приклад:

```
1 void main() {
2
3     Future(() {
4         print("Виконується Future");
5         throw 404;
6     })
7     .then((value) {
8         print("Із Future отримано значення: $value");
9     })
10    .catchError((err) {
11        print("Код помилки: $err");
12    },
13    test: (err) {
14        return err is int;
15    });
16
17    print("Чекаємо отримання значення із Future");
18 }
```

Вираз **throw 404** вказує, що об'єкт помилки є число 404, тобто тип **int**.

У методі **catchError** другий параметр - функція **(err) {return err is int; }** Через параметр **err** отримує об'єкт помилки і за допомогою виразу **err is int** перевіряє, чи є об'єкт помилки тип **int**. Якщо є, то повертається **true** і виконується функція **(err) {print ("Код помилки: \$ err");}**. В прикладі отримаємо результат роботи програми, як показано на рисунку 8.7.

Чекаємо отримання значення із Future

Виконується Future

Код помилки: 404

Рисунок 8.7. Результат роботи програми

Проте якщо прописати генерацію помилки наступним чином:

```
1  throw "404";
```

То тепер буде передаватися рядок, а не число, відповідно вираз **return err is int** для такої помилки поверне **false**, і вона не буде оброблятися.

8.2.5. Метод `whenComplete`

Ще один метод - **whenComplete** виконує деяку функцію при завершенні **Future**. Ця функція виконується незалежно від того, чи виникла у **Future** помилка чи ні - після всіх функцій із **then** та **catchError**. В якості прикладу розглянемо наступний код програми:

```
1  void main() {
2
3      Future(() {
4          print("Виконується Future");
5          return "Hello Dart";
6      })
7      .then((value) {
8          print("Із Future отримано значення: $value");
9      })
10     .catchError((err) {
11         print("Помилка: $err");
12     })
13     .whenComplete(() { print("Future завершив свою
роботу"); });
14
15     print("Чекаємо отримання значення із Future");
16 }
```

Результат роботи програми показано на рисунку 8.8.

Чекаємо отримання значення з Future

Виконується Future

Із Future отримано значення: Hello Dart

Future завершив свою роботу

Рисунок 8.8. Результат роботи програми

8.3. Асинхронні функції і оператори `async` і `await`

В новіших підверсіях в мову Dart були додані оператори `async` і `await`, які полегшують написання асинхронного коду. Застосування `async-await` дозволяє відійти від прямого використання **Future API**, зокрема, його методів `then` і `catchError`.

Спочатку розглянемо, як був би написаний код за допомогою **Future API** без `async` і `await`:

```
1  void doWork() {
2      print("Початок функції doWork");
3      Future<String> messageFuture = getMessage();
4      messageFuture.then((message) {
5          print("Отримано повідомлення: $message");
6      });
7      print("Завершення функції doWork");
8  }
9
10 Future<String> getMessage() {
11     // імітація довготривалої роботи за допомогою
12     // затримки у 3 секунди
13     return Future.delayed(Duration(seconds: 3), () =>
14         "Hello Dart");
15 }
16
17 void main () {
18     doWork();
19     print("Виконання функції main");
20 }
```

В цьому прикладі визначена функція `getMessage()`, яка повинна отримувати деякі повідомлення. Для імітації довготривалої роботи встановлюємо в ній затримку у 3 секунди. Але в реальності це може бути функція, яка виконує мережеві запити, звернення до бази даних, читання-запис файлів і іншу важку роботу, яка може зайняти тривалий час.

Припустимо, у функції **doWork()** викликається функція **getMessage**, яка повертає об'єкт **Future**. Через її метод **then** отримуємо потрібне повідомлення і виводимо його на консоль. У функції **main** викликається **doWork**. Результат роботи програми показано на рисунку 8.9.

Початок функції **doWork**
Завершення функції **doWork**
Виконання функції **main**
Отримано повідомлення: **Hello Dart**

Рисунок 8.9. Результат роботи програми

Звернемо увагу, що функція **doWork** завершує своє виконання до завершення **Future**.

Тепер перепишемо той же код з використанням **async-await**:

```
1  void doWork() {
2      print("Початок функції doWork");
3      Future<String> messageFuture = getMessage();
4      messageFuture.then((message) {
5          print("Отримано повідомлення: $message");
6      });
7      print("Завершення функції doWork");
8  }
9
10 Future<String> getMessage() {
11     // імітація довготривалої роботи за допомогою
12     // затримки у 3 секунди
13     return Future.delayed(Duration(seconds: 3), () =>
14         "Hello Dart");
15 }
16
17 void main () {
18     doWork();
19     print("Виконання функції main");
20 }
```

Функція **getMessage**, яка виконує якусь довготривалу роботу (в нашому прикладі отримує повідомлення), залишається такою ж самою, як і була. Основні зміни стосуються функції **doWork**, яка звертається до **getMessage** для отримання повідомлення. Тепер вона визначена як асинхронна. Для цього після списку параметрів функції вказується оператор **async**:

```
1 Future<void> doWork() async {
```

Асинхронна функція - це така функція, яка містить як мінімум одну асинхронну операцію (хоча також разом з цим може містити та виконувати і синхронні операції). Асинхронна функція виглядає як синхронна за тим винятком, що вона використовує оператори **async** і **await**.

Асинхронна функція виконує синхронно весь код, який йде до першого виклику виразу **await**. В наведеному випадку синхронно виконується наступний рядок:

```
1 print("Початок функції doWork");
```

Вираз **await** позначає асинхронну операцію. **Асинхронна операція** - це така операція, яка не блокує виконання інших операцій і дозволяє їм виконуватися до свого завершення. В наведеному прикладі це:

```
1 String message = await getMessage();
```

Вираз **await** зазвичай повертає об'єкт **Future**. Якщо повертається об'єкт іншого типу, то він автоматично обертається під **Future**. І цей об'єкт, який зберігається у **Future**, можна отримати простою операцією присвоєння. Так, функція **getMessage()** повертає **Future <String>**, тобто **Future** зберігає деякий рядок. І шляхом операції присвоєння можна передати цей рядок в змінну **message**. Далі можна робити з цим рядком, все що заманеться, наприклад, вивести на консоль.

Вираз **await** призупиняє виконання, поки потрібний об'єкт (в наведеному прикладі рядок) не доступний. А асинхронна функція **getMessage** повертає об'єкт **Future**, який починає своє виконання.

І вже після того, як з виразу **await** отримано рядок, функція **doWork** продовжує своє виконання і переходить до виконання коду, що залишився:

```
1 print("Отримано повідомлення: $message");  
2 print("Завершення функції doWork");
```

У підсумку, в цьому прикладі вигляд консолі буде виглядати, як показано на рисунку 8.10.

```
Початок функції doWork  
Виконання функції main  
Отримано повідомлення: Hello Dart
```

Завершення функції doWork

Рисунок 8.10. Результат роботи програми

Варто відзначити, що якщо функція використовує **await**, то вона обов'язково повинна бути визначена з ключовим словом **async**. Також варто відзначити, що асинхронна функція може містити кілька виразів **await**. У цьому випадку вони виконуються послідовно: коли завершиться попередній, починає виконуватися наступний вираз **await**.

8.3.1. Обробка помилок

Якщо в процесі виконання довготривалої операції сталася помилка, то об'єкт **Future** замість конкретних даних буде містити інформацію про помилку. Для обробки подібної ситуації досить розташувати вираз **await** в блок **try..catch**:

```
1 Future<void> doWork() async {
2     print("Початок функції doWork");
3     try{
4         String message = await getMessage();
5         print("Отримано повідомлення: $message");
6     }
7     catch(e) {
8         print("Сталася помилка: $e");
9     }
10    print("Завершення функції doWork");
11 }
12
13 Future<String> getMessage() {
14     // за допомогою оператора throw імітуємо виникнення
    помилки
15     return Future.delayed(Duration(seconds: 3), () =>
        throw "Сервер не відповідає");
16 }
17
18 void main () {
19     doWork();
20     print("Виконання функції main");
21 }
```

Для імітації помилки викликаємо оператор **throw**, після якого вказуємо текст помилки. У асинхронній функції в конструкції **try..catch** можна перехопити виникнення помилки і обробити її. Результат роботи програми показано на рисунку 8.11.

Початок функції **doWork**
Виконання функції **main**
Сталася помилка: Сервер не відповідає
Завершення функції **doWork**

Рисунок 8.11. Результат роботи програми

Резюме

В цьому розділі було розглянуто питання призначення та використання асинхронності у мові Dart, призначенню та реалізації циклу подій (**event loop**) і двох черг подій (**event queue** і **microTask queue**), що дозволяє асинхронно виконувати різні завдання.

Розглянуто способи визначення і використання класу **Future**, його основних конструкторів та способів обробки помилок.

Показано способи визначення і використання асинхронних функцій і операторів **async** і **await**, в тому числі способи обробки помилок у них.

Показано яким чином визначаються та використовуються механізми асинхронності та обробка винятків у них при написанні програм.

Контрольні запитання і завдання

1. Який зміст вкладається в поняття асинхронність?
2. Які приклади корисності застосування асинхронності можна навести?
3. Мова Dart однопотокова чи багатопотокова?
4. Завдяки яким механізмам мова Dart дозволяє асинхронно виконувати різні завдання?
5. Для чого призначена черга **MicroTask**?
6. Для чого призначена черга **Event**?
7. Який клас позначає результат відкладеної операції, яка завершить своє виконання в майбутньому?
8. Які стани має об'єкт класу **Future**?
9. Яка механіка роботи **Future**?
10. Які є конструктори для створення об'єкта **Future**?
11. Як можна відстежити той момент, коли об'єкт **Future** вже виконався і перейшов в завершений стан?

12. Як при виконанні операції, яку позначає **Future**, оброблюються помилки?
13. Для чого використовується метод **whenComplete**?
14. Що таке асинхронна функція?
15. Що таке асинхронна операція?
16. Яке призначення операторів **async** і **await**?
17. Як оброблюються помилки при застосування оператора **await**?
18. Наведіть приклади отримання значення з **Future**.
19. Наведіть приклади застосування методу **then** об'єкту **Future**.
20. Наведіть приклади обробки помилок в процесі виконання **Future**.
21. Наведіть приклади застосування операторів **async** і **await**.
22. Наведіть приклади обробки помилок при застосування оператора **await**.

9. ПРИКЛАДИ ТА ВПРАВИ

Мова Dart розроблена так, щоб її легко було вивчити програмістам, що переходять з інших мов, але вона має кілька унікальних особливостей. Приклади і вправи в цьому розділі базується на мовній таблиці Dart (Dart language cheatsheet¹⁾ [33]), написаній інженерами Google, і дозволять опанувати найважливіші з цих мовних функцій.

Для написання та тестування коду можна використовувати онлайн ресурс DartPad²⁾ [34]. Для зручності, в деяких завданнях наводиться вже частково заповнений фрагмент коду. Отже, можна використовувати редактор, що знаходиться за посиланням <https://dart.dev/#try-dart> , щоб перевірити отримані знання, заповнивши код і натиснувши кнопку **Виконати (Run)**. Якщо знадобиться допомога, можна натиснути кнопку **Підказка (Hint)**. Щоб запустити програму форматування коду (**dart format**³⁾ [35]), потрібно натиснути кнопку **Формат (Format)**. Кнопка **Скинути (Reset)** стирає написану роботу та відновлює редактор до початкового стану.

9.1. Інтерполяція рядків

Щоб помістити значення виразу всередину рядка, використовується конструкція **`$ {expression}`**. Якщо вираз є ідентифікатором, то можна пропустити **`{}`**.

Кілька прикладів використання інтерполяції рядків наведені в таблиці 9.1.

Таблиця 9.1

Використання інтерполяції рядків

Рядок	Результат
<code>'\${3 + 2}'</code>	<code>'5'</code>
<code>'\${"word".toUpperCase()}'</code>	<code>'WORD'</code>
<code>'\$myObject'</code>	<code>The value of myObject.toString()</code>

¹⁾ <https://dart.dev/guides/language/cheatsheet>

²⁾ <https://dart.dev/#try-dart>

³⁾ <https://dart.dev/tools/dart-format>

Завдання 9.1

Наступна функція приймає два цілі числа як параметри. Необхідно зробити так, щоб вона повернула рядок, що містить обидва цілих числа, розділені пробілом. Наприклад, **stringify (2, 3)** повинен повернути **'2 3'**.

```
1 String stringify(int x, int y) {  
2     TODO('Повернути форматований рядок тут');  
3 }
```

9.2. Nullable змінні

Відомо, що Dart 2.12 оголосив нульову безпеку (**null safety**), що означає, що (коли вмикається нуль-безпека (**enable null safety**¹⁾ [36])) значення не можуть бути нульовими, якщо не буде додатково вказано, що вони можуть бути **null**. Іншими словами, типи за замовчуванням не допускають нульового значення (**non-nullable types**).

Наприклад, розглянемо такий код, який є **недійсним**, оскільки (із нуль-безпекою) змінна типу **int** не може мати значення **null**:

```
1 int a = null; // Недійсний код у null-safe Dart.
```

Створюючи змінну в Dart 2.12 або вище, можна додати символ **?** до типу, щоб вказати, що змінна може мати нульове значення:

```
1 int? a = null; // Дійсний код у null-safe Dart
```

Можна трохи спростити цей код, оскільки у всіх версіях Dart **null** є значенням за замовчуванням для неініціалізованих змінних:

```
1 int? a; // Початкове значення a дорівнює null
```

Для отримання додаткової інформації про **null safety** в Dart доцільно ознайомитись з посібником **Sound null safety guid**²⁾ [37].

Завдання 9.2

Необхідно оголосити дві змінні, що наведені нижче, ігноруючи всі початкові помилки в DartPad.

- **Nullable** рядок **String**, з іменем **name** зі значенням **'Jane'** .

- **Nullable** рядок **String** з іменем **address** зі значенням **null**.

```
1 // Оголосити дві змінні тут
```

¹⁾ <https://dart.dev/null-safety#enable-null-safety>

²⁾ <https://dart.dev/null-safety>

9.3. Null-aware оператори

Dart пропонує кілька зручних операторів для роботи зі значеннями, які можуть мати нульове значення (**null**). Одним із них є оператор присвоєння **??=**, який присвоює значення змінній лише в тому випадку, якщо ця змінна в даний час має значення **null**:

```
1  int? a; // = null
2  a ??= 3;
3  print(a); // <-- Друкує 3.
4
5  a ??= 5;
6  print(a); // <-- Продовжує друкувати 3.
```

Іншим **null-aware** оператором є **??**, який повертає вираз зліва, якщо значення цього виразу не має значення **null**, і в цьому випадку він обчислює та повертає вираз праворуч:

```
1  print(1 ?? 3); // <-- Друкує 1.
2  print(null ?? 12); // <-- Друкує 12.
```

Завдання 9.3

Необхідно використати оператори **??=** та **??** в прикладі, наведеному нижче, ігноруючи всі початкові помилки в **DartPad**.

```
1  String? foo = 'a string';
2  String? bar; // = null
3
4  // Замініть оператор аби 'a string' присвоїти bar.
5  String? baz = foo /* TODO */ bar;
6
7  void updateSomeVars() {
8      // Замініть оператор аби 'a string' присвоїти bar.
9      bar /* TODO */ 'a string';
10 }
```

9.4. Умовний доступ до властивостей

Щоб захистити доступ до властивості або методу об'єкта, який може бути нульовим (**null**), використовується знак питання (?) перед крапкою (.):

```
1  myObject?.someProperty
```


Попередній код еквівалентний наступному:

```
1 (myObject != null) ? myObject.someProperty : null
```

Можна застосувати ланцюжок із декількох символів (?) разом в одному виразі:

```
1 myObject?.someProperty?.someMethod()
```

Попередній код повертає **null** (і ніколи не викликає **someMethod ()**), якщо **myObject** або **myObject.someProperty** є **null**.

Завдання 9.4

Необхідно використати умовний доступ до властивостей, щоб закінчити фрагмент коду наведений нижче.

```
1 // Цей метод має повернути версію `str` з усіма
  великими літерами
2 // або null, якщо `str` є null.
3 String? upperCaseIt(String? str) {
4     // Спробуйте умовний доступ у методі `toUpperCase`
  тут.
5 }
```

9.5. Колекції

Dart має вбудовану підтримку **lists**, **maps**, and **sets**. Можна створити їх за допомогою наступних літералів:

```
1 final aListOfStrings = ['one', 'two', 'three'];
2 final aSetOfStrings = {'one', 'two', 'three'};
3 final aMapOfStringsToInts = {
4     'one': 1,
5     'two': 2,
6     'three': 3,
7 };
```

Dart може самостійно визначити типи для цих змінних на основі аналізу змісту даних. В наведеному прикладі, Dart визначить наступні типи **List <String>**, **Set <String>** та **Map <String, int>**.

Або ж можна вказати тип самостійно:

```

1  final aListOfInts = <int>[];
2  final aSetOfInts = <int>{};
3  final aMapOfIntToDouble = <int, double>{};

```

Явне вказування типу зручно, коли ініціалізується список із вмістом, що є підтипом, але все одно хочемо, щоб список мав базовий тип, тобто **List** **<BaseType>**:

```

1  final aListOfBaseType = <BaseType>[SubType(),
    SubType()];

```

Завдання 9.5

Необхідно ініціалізувати наступні змінні вказаними значення. Необхідно замінити існуючі нульові значення.

```

1  // Ініціалізуйте цей list, аби він містив 'a', 'b',
    and 'c' саме в цьому порядку:
2  final aListOfStrings = null;
3
4  // Ініціалізуйте цей set, аби він містив 3, 4, та 5:
5  final aSetOfInts = null;
6
7  // Ініціалізуйте цей map зі String з int, аби
    aMapOfStringsToInts['myKey'] повертав 12:
8  final aMapOfStringsToInts = null;
9
10 // Ініціалізуйте цей порожній List<double>:
11 final anEmptyListOfDouble = null;
12
13 // Ініціалізуйте цей порожній Set<String>:
14 final anEmptySetOfString = null;
15
16 // Ініціалізуйте цей порожній Map із double з int:
17 final anEmptyMapOfDoublesToInts = null;

```

9.6. Синтаксис стрілки (=>)

У Dart та деяких інших мовах використовується символ `=>`. Синтаксис стрілки (`=>`) - це спосіб визначення функції, яка виконує вираз праворуч і повертає його значення.

Наприклад, розглянемо наступний виклик методу **any ()** класу **List**:

```
1 bool hasEmpty = aListOfStrings.any((s) {
2     return s.isEmpty;
3 });
```

Також існує простіший спосіб написання цього коду:

```
1 bool hasEmpty = aListOfStrings.any((s) => s.isEmpty)
```

Завдання 9.6

Необхідно закінчити наступні вирази, використовуючи синтаксис стрілки (`=>`).

```
1 class MyClass {
2     int value1 = 2;
3     int value2 = 3;
4     int value3 = 5;
5
6     // Метод, який повертає добуток, зазначених вище
7     // значень:
8     int get product => TODO();
9
10    // Додає 1 до значення value1:
11    void incrementValue1()=> TODO();
12
13    // Метод, який повертає рядок із значень
14    // list, що розділені символом кома (тобто
15    'a,b,c'):
16    String joinWithCommas(List<String> strings) =>
17    TODO();
18 }
```

9.7. Каскади

Щоб виконати послідовність операцій над тим самим об'єктом, використовують, так звані, каскади (`..`). Розглянемо відомий вираз:

```
1 myObject.someMethod()
```

Він викликає метод **someMethod ()** об'єкту **myObject**, а результатом виразу є значення, яке метод **someMethod ()** повертає.

А ось цей самий вираз з використанням каскадів:

```
1 myObject..someMethod()
```

Хоча він все ще викликає метод **someMethod ()** об'єкту **myObject**, результат виразу не є значення, яке метод **someMethod ()** повертає – в цьому разі це посилання на **myObject**! Використовуючи каскади, можна об'єднувати операції, які в іншому випадку вимагали б окремих операторів. Наприклад, розглянемо наступний код:

```
1 var button = querySelector('# підтвердити');
2 button.text = 'Підтвердити';
3 button.classes.add('важливо');
4 button.onClick.listen((e) =>
  window.alert('Підтверджено!'));
```

У каскадах код стає набагато коротшим, і нам не потрібна змінна **button**:

```
1 querySelector('# підтвердити')
2 ..text = 'Підтвердити'
3 ..classes.add('важливо')
4 ..onClick.listen((e) =>
  window.alert('Підтверджено!'));
```

Завдання 9.7

Використайте каскади, щоб створити єдиний вираз, який встановлює властивості **anInt**, **aString** та **aList** об'єкту **BigObject** значення **1**, **'String!'** та **[3.0]** (відповідно), а потім викликає метод **allDone ()**.

```
1 class BigObject {
2   int anInt = 0;
3   String aString = '';
```

```

4     List<double> aList = [];
5     bool _done = false;
6
7     void allDone() {
8         _done = true;
9     }
10 }
11
12 BigObject fillBigObject(BigObject obj) {
13     // Створіть код, який оновить та поверне obj:
14     return TODO('obj..');
15 }

```

9.8. Гетери та сетери

Можна визначати гетери та сетери, коли потрібно більше контролю над властивостями, ніж це дозволяє просту поле.

Наприклад, можна перевірити, що значення властивості є допустимим:

```

1 class MyClass {
2     int _aProperty = 0;
3
4     int get aProperty => _aProperty;
5
6     set aProperty(int value) {
7         if (value >= 0) {
8             _aProperty = value;
9         }
10    }
11 }

```

Також можна використовувати гетер для визначення якихось дій або обчислень:

```

1 class MyClass {
2     List<int> _values = [];
3
4     void addValue(int value) {
5         _values.add(value);
6     }
7 }

```

```

6      }
7
8      // Виконання обчислень.
9      int get count {
10         return _values.length;
11     }
12 }

```

Завдання 9.8

Уявімо, є клас кошика для покупок, який зберігає приватний список **List<double>** цін. Необхідно додати наступне:

- ✓ гетер, який називається **total**, що повертає суму цін;
- ✓ сетер, який замінює список новим, якщо новий список не містить жодних від'ємних цін (у цьому випадку сетер повинен згенерувати виняток **InvalidPriceException**).

В цьому прикладі необхідно буде ігнорувати всі початкові помилки в **DartPad**.

```

1  class InvalidPriceException {}
2
3  class ShoppingCart {
4      List<double> _prices = [];
5
6      // Додайте гетер "total" тут:
7
8      // Додайте сетер "prices" тут:
9  }

```

9.9. Необов'язкові позиційні параметри

Dart має два типи передачі та доступу до параметрів функцій: позиційний та іменований. Позиційні параметри - це ті, з які, мабуть, вже знайомі з інших мов програмування:

```

1  int sumUp(int a, int b, int c) {
2      return a + b + c;
3  }
4  // ...
5  int total = sumUp(1, 2, 3);

```

За допомогою Dart можна зробити ці позиційні параметри необов'язковими, обернувши їх у дужки `()`:

```
1  int sumUpToFive(int a, [int? b, int? c, int? d, int?
   e]) {
2    int sum = a;
3    if (b != null) sum += b;
4    if (c != null) sum += c;
5    if (d != null) sum += d;
6    if (e != null) sum += e;
7    return sum;
8  }
9  // ...
10 int total = sumUpToFive(1, 2);
11 int otherTotal = sumUpToFive(1, 2, 3, 4, 5);
```

Необов'язкові позиційні параметри завжди є останніми у списку параметрів функції. Їх значення за замовчуванням є нульовим (**null**), якщо не вказано інше значення за замовчуванням:

```
1  int sumUpToFive(int a, [int b = 2, int c = 3, int d
   = 4, int e = 5]) {
2    // ...
3  }
4  // ...
5  int newTotal = sumUpToFive(1);
6  print(newTotal); // <-- Виводить на консоль число
   15
```

Завдання 9.9

Необхідно реалізувати функцію з назвою **joinWithCommas ()**, яка приймає від одного до п'яти цілих чисел, а потім повертає рядок із цих чисел, розділених комами.

```
1  String joinWithCommas(int a, [int? b, int? c, int?
   d, int? e]) {
2    return TODO();
3  }
```

Декілька прикладів викликів функцій та повернутих значень наведені в таблиці 9.2.

Таблиця 9.2

Приклади викликів функцій та повернутих значень

Виклик функції	Повернуте значення
<code>joinWithCommas (1)</code>	<code>'1'</code>
<code>joinWithCommas (1, 2, 3)</code>	<code>'1,2,3'</code>
<code>joinWithCommas (1, 1, 1, 1, 1)</code>	<code>'1,1,1,1,1'</code>

9.10. Необов'язкові іменовані параметри

Використовуючи синтаксис фігурних дужок {}, можна визначити необов'язкові параметри, що мають імена.

```
1 void printName(String firstName, String lastName,  
  {String? suffix}) {  
2   print('$firstName $lastName ${suffix ?? ''}');  
3 }  
4 // ...  
5 printName('Winny', 'Pooh');  
6 printName('R2', 'D2', suffix: 'v_VI');
```

Як і слід було очікувати, значення за замовчуванням необов'язкового іменованого параметра є нульовим (**null**), але можна вказати значення за замовчуванням.

Якщо тип параметра є **non-nullable**, тоді потрібно або вказати значення за замовчуванням (як показано в наступному прикладі коду), або позначити параметр як необхідний (**required**), як показано далі в розділі «Конструкторів».

```
1 void printName(String firstName, String lastName,  
  {String suffix = ''}) {  
2   print('$firstName $lastName $suffix');  
3 }
```

Функція не може одночасно мати як необов'язкові позиційні, так і необов'язкові іменовані параметри.

Завдання 9.10

Необхідно додати наступний метод екземпляра **copyWith ()** до класу **MyDataObject**. Він повинен приймати три іменовані параметри, що допускають нульове значення (тобто **nullable**):

```
1  int? newInt;  
2  String? newString;  
3  double? newDouble.
```

Метод **copyWith ()** повинен повернути новий **MyDataObject** на основі поточного екземпляра, із даними попередніх параметрів (якщо такі є) скопійованими у властивості об'єкта. Наприклад, якщо **newInt** не є нульовим (**non-null**), скопіюйте його значення в **anInt**. В цьому прикладі слід ігнорувати всі початкові помилки в **DartPad**.

```
1  class MyDataObject {  
2      final int anInt;  
3      final String aString;  
4      final double aDouble;  
5  
6      MyDataObject({  
7          this.anInt = 1,  
8          this.aString = 'Old!',  
9          this.aDouble = 2.0,  
10     });  
11  
12     // Додайте свій метод copyWith тут:  
13 }
```

9.11. Винятки

Dart може генерувати та перехоплювати винятки. На відміну від Java, усі винятки у Dart не перевіряються (**unchecked exceptions**). Методи не оголошують, які винятки вони можуть згенерувати, і програміст не зобов'язаний перехоплювати будь-які винятки.

Dart забезпечує два вбудовані типи винятків: **Exception** та **Error**, але дозволено генерувати власні будь-які ненульові (**non-null**) об'єкти:

```
1  throw Exception('Щось сталося небезпечно');  
2  throw 'Aaaaaaaa!';
```

Для обробки винятків використовують ключові слова **try**, **on** та **catch**:

```
1  try {
2      breedMoreLlamas();
3  } on OutOfLlamasException {
4      // Конкретний виняток
5      buyMoreLlamas();
6  } on Exception catch (e) {
7      // Що-небудь інше, що є винятком
8      print('Невідомий виняток: $e');
9  } catch (e) {
10     // Не вказаний тип, обробляються всі
11     print('Якийсь справді невідомий виняток: $e');
12 }
```

Ключове слово **try** працює як у більшості інших мов. Використовуючи ключове слово **on**, можна фільтрувати конкретні винятки за типом, а використовуючи ключове слово **catch**, можна отримати посилання на об'єкт винятку.

Якщо не вдається повністю обробити виняток, то необхідно використовувати ключове слово **rethrow** для передачі винятку вгору:

```
1  try {
2      breedMoreLlamas();
3  } catch (e) {
4      print('Я просто намагався розводити лам!');
5      rethrow;
6  }
```

Щоб виконати код, незалежно від того, чи виник виняток, можна зрештою використовувати конструкцію **finally**:

```
1  try {
2      breedMoreLlamas();
3  } catch (e) {
4      // ... обробка винятків ...
5  } finally {
6      // Завжди прибирайте за собою, навіть якщо
7      // згенеровано виняток.
8      cleanLlamaStalls();
9  }
```

Завдання 9.11

Необхідно застосувати функцію **tryFunction ()**, що наведена нижче. Вона повинна виконати ненадійний метод, а потім виконати наступне:

- якщо ненадійний метод **untrustworthy()** генерує **ExceptionWithMessage**, необхідно виконати виклик **logger.logException** із типом винятку та повідомленням (спробуйте використовувати **on** та **catch**);

- якщо ненадійний метод **untrustworthy()** генерує **Exception**, необхідно виконати виклик **logger.logException** із типом винятку (спробуйте використати **on** для цього);

- якщо ненадійний метод **untrustworthy()** генерує будь-який інший об'єкт, не перехоплюйте та не оброблюйте виняток;

- після того, як все буде перехоплено та оброблено, необхідно виконати виклик **logger.doneLogging** (спробуйте використати **finally**).

```
1  typedef VoidFunction = void Function();
2
3  class ExceptionWithMessage {
4      final String message;
5      const ExceptionWithMessage(this.message);
6  }
7
8  // Виконайте виклик logException для реєстрації
   // винятків та doneLogging після закінчення їх обробки.
9  abstract class Logger {
10     void logException(Type t, [String? msg]);
11     void doneLogging();
12 }
13
14 void tryFunction(VoidFunction untrustworthy, Logger
   logger) {
15     // Виклик цього методу може спричинити виняток.
16     // Перехопіть та виконайте обробку цього винятку за
   // допомогою try-on-catch-finally.
17     untrustworthy();
18 }
```

9.12. Використання **this** в конструкторах

Dart надає зручний спосіб посилання для присвоєння значень властивостям у конструкторі: так, можна використовувати конструкцію **this.propertyName** при оголошенні конструктора:

```
1  class MyColor {
2      int red;
3      int green;
4      int blue;
5
6      MyColor(this.red, this.green, this.blue);
7  }
8
9  final color = MyColor(80, 80, 128);
```

Цей прийом працює і для іменованих параметрів. Назви властивостей стають іменами параметрів:

```
1  class MyColor {
2      ...
3
4      MyColor({required this.red, required this.green,
5               required this.blue});
6  }
7  final color = MyColor(red: 80, green: 80, blue: 80);
```

У попередньому коді **red**, **green**, та **blue** позначені як обов'язкові параметри, оскільки ці значення **int** не можуть бути нульовими (**null**). Якщо додати значення за замовчуванням, то можна пропустити обов'язкові параметри:

```
1  MyColor([this.red = 0, this.green = 0, this.blue = 0]
2  // або
3  MyColor({this.red = 0, this.green = 0, this.blue = 0
```

Завдання 9.12

Необхідно додати однорядковий конструктор до класу **MyClass**, який використовує синтаксис **this**. для отримання та призначення значень для всіх

трьох властивостей класу. При виконанні цієї вправи доцільно ігнорувати всі початкові попередження про помилки в **DartPad**.

```
1  class MyClass {
2      final int anInt;
3      final String aString;
4      final double aDouble;
5
6      // Конструктор створюйте тут.
7  }
```

9.13. Ініціалізація списків (list)

Іноді, коли необхідно реалізувати конструктор, потрібно виконати деякі налаштування перед тим, як виконати тіло конструктора. Наприклад, кінцеві поля повинні мати значення перед виконанням тіла конструктора. Виконуються ці дії в ініціалізаторі списку (**list**), який знаходиться між оголошенням конструктора та його тілом:

```
1  Point.fromJson(Map<String, num> json)
2      : x = json['x'],
3        y = json['y'] {
4      print('In Point.fromJson(): ($x, $y)');
5  }
```

Ініціалізатор списку (**list**) також є зручним місцем для розміщення оголошень, які запускаються лише під час розробки:

```
1  NonNegativePoint(this.x, this.y)
2      : assert(x >= 0),
3        assert(y >= 0) {
4      print('Ми щойно створили NonNegativePoin: ($x, $y)');
5  }
```

Завдання 9.13

Необхідно заповнити конструктор **FirstTwoLetters**, наведений нижче. Для цього слід використовувати ініціалізатор списку (**list**), щоб призначити перші два символи в **word** властивостям **letterOne** та **LetterTwo**. В якості ускладнення

завдання, необхідно додати оголошення (**assert**), щоб перехопити випадки, коли слова (**word**) мають довжину менше двох символів.

```
1  class FirstTwoLetters {
2      final String letterOne;
3      final String letterTwo;
4
5      // Створіть конструктор з ініціалізатором списку тї
6      FirstTwoLetters(String word)
7          ...
```

9.14. Іменовані конструктори

Щоб дозволити класам мати кілька конструкторів, Dart підтримує створення іменованих конструкторів:

```
1  class Point {
2      double x, y;
3
4      Point(this.x, this.y);
5
6      Point.origin()
7          : x = 0,
8            y = 0;
9  }
```

Щоб використовувати іменований конструктор, його викликають, використовуючи повне ім'я:

```
1  final myPoint = Point.origin();
```

Завдання 9.14

Необхідно надати класу **Color** конструктор на ім'я **Color.black**, який встановлює всі три властивості на нуль. При виконанні цієї вправи доцільно ігнорувати всі початкові попередження про помилки в **DartPad**.

```

1  class Color {
2      int red;
3      int green;
4      int blue;
5
6      Color(this.red, this.green, this.blue);
7
8      // Створіть іменований конструктор з назвою
      "Color.black" тут:
9  }

```

9.15. Фабричні конструктори

Dart підтримує фабричні конструктори, які можуть повертати підтипи або навіть **null**. Щоб створити фабричний конструктор, використовують ключове слово **factory**:

```

1  class Square extends Shape {}
2
3  class Circle extends Shape {}
4
5  class Shape {
6      Shape();
7
8      factory Shape.fromTypeName(String typeName) {
9          if (typeName == 'square') return Square();
10         if (typeName == 'circle') return Circle();
11
12         print('Тип $typeName не розпізнано');
13         throw Error();
14     }
15 }

```

Завдання 9.15

Необхідно заповнити фабричний конструктор з назвою **IntegerHolder.fromList**, реалізувавши наступне:

- якщо список (**list**) має одне значення, то необхідно створити **IntegerSingle** з цим значенням;

- якщо список (**list**) має два значення, то необхідно створити **IntegerDouble** із відповідними значеннями;
- якщо список (**list**) має три значення, то необхідно створити **IntegerTriple** із відповідними значеннями;
- в інших випадках необхідно згенерувати помилку **Error**.

```

1  class IntegerHolder {
2      IntegerHolder();
3
4      // Необхідно впровадити цей фабричний конструктор.
5      factory IntegerHolder.fromList(List<int> list) {
6          TODO();
7      }
8  }
9
10 class IntegerSingle extends IntegerHolder {
11     final int a;
12     IntegerSingle(this.a);
13 }
14
15 class IntegerDouble extends IntegerHolder {
16     final int a;
17     final int b;
18     IntegerDouble(this.a, this.b);
19 }
20
21 class IntegerTriple extends IntegerHolder {
22     final int a;
23     final int b;
24     final int c;
25     IntegerTriple(this.a, this.b, this.c);
26 }

```

9.16. Перенаправлення конструкторів

Іноді єдиною метою конструктора є перенаправлення на інший конструктор того самого класу. Тіло конструктора, який перенаправляє, порожнє, виклик конструктора відображається після двокрапки (:).


```

1  class Automobile {
2      String make;
3      String model;
4      int mpg;
5
6      // Основний конструктор для цього класу.
7      Automobile(this.make, this.model, this.mpg);
8
9      // Делегування до головного конструктора.
10     Automobile.hybrid(String make, String model) :
        this(make, model, 60);
11
12     // Делегування до іменованого конструктора
13     Automobile.fancyHybrid() : this.hybrid('Futurecar',
        'Mark 2');
14 }

```

Завдання 9.16

Розглянемо клас **Color**, з попереднього матеріалу. Необхідно створити іменований конструктор, який називається **black**, але замість того, щоб вручну призначати властивості, необхідно перенаправити його до конструктора за замовчуванням із нулями в якості аргументів. При виконанні цієї вправи доцільно ігнорувати всі початкові попередження про помилки в **DartPad**.

```

1  class Color {
2      int red;
3      int green;
4      int blue;
5
6      Color(this.red, this.green, this.blue);
7
8      // Створіть тут іменований конструктор із назвою
        "black"
9      // та перенаправте його на виклик існуючого
        конструктора
10 }

```

9.17. Константні конструктори

Якщо конкретний клас створює об'єкти, які ніколи не змінюються, то можна зробити ці об'єкти константами часу компіляції. Для цього визначають конструктор за допомогою модифікатора **const**, і всі змінні екземпляра **final**.

```
1  class ImmutablePoint {
2      const ImmutablePoint(this.x, this.y);
3
4      final int x;
5      final int y;
6
7      static const ImmutablePoint origin =
        ImmutablePoint(0, 0);
8  }
```

Завдання 9.17

Необхідно змінити клас **Recipe**, щоб його екземпляри могли бути константними, та створити константний конструктор, який реалізовує наступне:

- містить три параметри: **ingredients**, **calories**, та **milligramsOfSodium** (саме у такому порядку);
- використовує синтаксис **this.** для автоматичного присвоєння значень параметрів властивостям об'єкта з однойменною назвою;
- є константним, з ключовим словом **const** безпосередньо перед класом **Recipe** в оголошенні конструктора.

При виконанні цієї вправи доцільно ігнорувати всі початкові попередження про помилки в **DartPad**.

```
1  class Recipe {
2      List<String> ingredients;
3      int calories;
4      double milligramsOfSodium;
5  }
```

Резюме

Матеріал в цьому розділі було спрямовано на закріплення отриманих знань та здобутих навичок, а також ознайомлення з деякими особливостями мови Dart.

Основне призначення розділу полягає окрім формування практичних навичок при розв'язанні завдань також у формуванні здатності формулюванні та постановки задач, що підтверджує творчий компонент при розробці на написанні програм.

Контрольні запитання і завдання

1. Що таке інтерполяція рядків?
2. Що таке **null safety**?
3. Які є **null-aware** оператори у Dart?
4. Який знак використовується, аби захистити доступ до властивості або методу об'єкта, який може бути нульовим (**null**)?
5. Які вбудовані колекції підтримує Dart?
6. Для чого у Dart використовується символ **=>**?
7. Для чого призначені каскади (**..**)?
8. Для чого створювати гетери та сетери?
9. Що таке необов'язкові позиційні параметри?
10. Що таке необов'язкові іменовані параметри?
11. Що таке винятки? Яка різниця тлумачення цього поняття в Dart та Java?
12. Який спосіб посилання для присвоєння значень властивостям у конструкторі пропонує Dart?
13. Що таке ініціалізація списку?
14. Навіщо потрібні іменовані конструктори?
15. Навіщо потрібні фабричні конструктори?
16. Навіщо потрібно перенаправлення конструкторів?
17. Навіщо потрібні константні конструктори?
18. Придумайте завдання з інтерполяції рядків.
19. Придумайте завдання використання **nullable** та **non-nullable** типами.
20. Придумайте завдання з використання операторів **??=** та **??**.
21. Придумайте завдання з використанням умовного доступу до властивостей.
22. Придумайте завдання з використанням колекцій **List**, **Set** та **Map**.
23. Придумайте завдання з використанням символу **=>**.
24. Придумайте завдання з використанням каскадів (**..**).
25. Придумайте завдання з використанням гетерів та сетерів.
26. Придумайте завдання з використанням необов'язкових позиційних параметрів.

27. Придумайте завдання з використанням необов'язкових іменованих параметрів.
28. Придумайте завдання з генерацією та обробкою винятків.
29. Придумайте завдання з використанням синтаксису **this**.
30. Придумайте завдання з використанням ініціалізації списку.
31. Придумайте завдання з використанням іменованих конструкторів.
32. Придумайте завдання з використанням фабричних конструкторів.
33. Придумайте завдання з використанням перенаправлення конструкторів.
34. Придумайте завдання з використанням константних конструкторів.

ПЕРЕЛІК ПОСИЛАНЬ

1. DartPad _ Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/tools/dartpad>. - Назва з екрана.
2. Dart - Visual Studio Marketplace [Електронний ресурс]. - Режим доступу: <https://marketplace.visualstudio.com/items?itemName=Dart-Code.dart-code>. - Назва з екрана.
3. Declaring Member Variables (The Java™ Tutorials _ Learning the Java Language _ Classes and Objects) [Електронний ресурс]. - Режим доступу: <https://docs.oracle.com/javase/tutorial/java/javaOO/variables.html>. - Назва з екрана.
4. Dart programming language _ Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/#try-dart>. - Назва з екрана.
5. Dart format _ Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/tools/dartfmt>. - Назва з екрана.
6. Effective Dart _ Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/guides/language/effective-dart>. - Назва з екрана.
7. Java-to-dart codelab: Basic bicycle example [Електронний ресурс]. - Режим доступу: <https://dartpad.dev/f176b4c31b4be8a454107d0946fe2fa3>. - Назва з екрана.
8. Declaring Member Variables (The Java™ Tutorials _ Learning the Java Language _ Classes and Objects) [Електронний ресурс]. - Режим доступу: <https://docs.oracle.com/javase/tutorial/java/javaOO/variables.html>. - Назва з екрана.
9. Creating Objects (The Java™ Tutorials _ Learning the Java Language _ Classes and Objects) [Електронний ресурс]. - Режим доступу: <https://docs.oracle.com/javase/tutorial/java/javaOO/objectcreation.html>. - Назва з екрана.
10. Java-to-dart codelab: Starting Rectangle example [Електронний ресурс]. - Режим доступу: <https://dartpad.dev/68fb4923eee0e9e3d3af42730c1d3503>. - Назва з екрана.
11. Java-to-dart codelab: Starting Shapes example [Електронний ресурс]. - Режим доступу: <https://dartpad.dev/a98247f43448f5e0eac4c7f483c173a5>. - Назва з екрана.
12. Effective Dart _ Style _ Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/guides/language/effective-dart/style#prefer-using-lowercamelcase-for-constant-names>. - Назва з екрана.

13. Java-to-dart codelab: Using a try-catch statement [Электронный ресурс]. - Режим доступа: <https://dartpad.dev/63e040a3fd682e191af40f6427eaf9ef>. - Назва з екрана.
14. Java-to-dart codelab: Factory constructor [Электронный ресурс]. - Режим доступа: <https://dartpad.dev/8e2ad295a1b7c8254414afb43fc7d280>. - Назва з екрана.
15. What's New in JDK 8 [Электронный ресурс]. - Режим доступа: <http://www.oracle.com/technetwork/java/javase/8-whats-new-2157071.html>. - Назва з екрана.
16. Java-to-dart codelab: Scream example [Электронный ресурс]. - Режим доступа: <https://dartpad.dev/9bcb7dba6dca46e2a9dbfb300298461e>. - Назва з екрана.
17. Commonly used packages _ Dart [Электронный ресурс]. - Режим доступа: <https://dart.dev/guides/libraries/useful-libraries>. - Назва з екрана.
18. Dart packages [Электронный ресурс]. - Режим доступа: <https://pub.dev/>. - Назва з екрана.
19. Pi-monte-carlo _web version) [Электронный ресурс]. - Режим доступа: <https://dartpad.dev/fae22cffa7b184b4d27cd96dd633a5af>. - Назва з екрана.
20. Method Cascades in Dart [Электронный ресурс]. - Режим доступа: <http://news.dartlang.org/2012/02/method-cascades-in-dart-posted-by-gilad.html>. - Назва з екрана.
21. Dart 1.12 Released, with Null-Aware Operators and more [Электронный ресурс]. - Режим доступа: <http://news.dartlang.org/2015/08/dart-112-released-with-null-aware.html>. - Назва з екрана.
22. Codelabs & workshops - Flutter [Электронный ресурс]. - Режим доступа: <https://flutter.dev/docs/codelabs>. - Назва з екрана.
23. Why Flutter Uses Dart _ Hacker Noon [Электронный ресурс]. - Режим доступа: <https://hackernoon.com/why-flutter-uses-dart-dd635a054ebf>. - Назва з екрана.
24. Announcing Dart 2_ Optimized for Client-Side Development _ by Anders Thorhauge Sandholm _ Dart _ Medium [Электронный ресурс]. - Режим доступа: <https://medium.com/dartlang/announcing-dart-2-80ba01f43b6>. - Назва з екрана.
25. Why I moved from Java to Dart _ Hacker Noon [Электронный ресурс]. - Режим доступа: <https://hackernoon.com/why-i-moved-from-java-to-dart-8f3802b1d652>. - Назва з екрана.
26. Language tour _ Dart [Электронный ресурс]. - Режим доступа: <https://dart.dev/guides/language/language-tour>. - Назва з екрана.

27. Library tour _ Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/guides/libraries/library-tour>. - Назва з екрана.
28. Effective Dart _ Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/guides/language/effective-dart>. - Назва з екрана.
29. Dart programming language _ Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/>. - Назва з екрана.
30. Flutter - Beautiful native apps in record time [Електронний ресурс]. - Режим доступу: <https://flutter.dev/>. - Назва з екрана.
31. List class - dart_core library - Dart API [Електронний ресурс]. - Режим доступу: <https://api.dart.dev/stable/2.4.0/dart-core/List-class.html>. - Назва з екрана.
32. Set class - dart_core library - Dart API [Електронний ресурс]. - Режим доступу: <https://api.dart.dev/stable/2.4.0/dart-core/Set-class.html>. - Назва з екрана.
33. Dart language cheatsheet _ Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/guides/language/cheatsheet>. - Назва з екрана.
34. Dart programming language _Try Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/#try-dart>. - Назва з екрана.
35. dart format _ Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/tools/dart-format>. - Назва з екрана.
36. Sound null safety _ Dart #enable-null-safety [Електронний ресурс]. - Режим доступу: <https://dart.dev/null-safety#enable-null-safety>. - Назва з екрана.
37. Sound null safety _ Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/null-safety>. - Назва з екрана.
38. Codelabs _ Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/codelabs>. - Назва з екрана.
39. A tour of the Dart language [Електронний ресурс]. - Режим доступу: <https://dart.dev/guides/language/language-tour>. - Назва з екрана.
40. Dart programming language _ Dart [Електронний ресурс]. - Режим доступу: <https://dartpad.dev/>. - Назва з екрана.
41. Get the Dart SDK _ Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/get-dart>. - Назва з екрана.

СПИСОК РЕКОМЕНДОВАНОЇ ЛІТЕРАТУРИ І ЕЛЕКТРОННИХ РЕСУРСІВ

1. Walrath, K. and Ladd, S. Dart: Up and Running [Електронний ресурс]. - Режим доступу: <https://www.oreilly.com/library/view/dart-up-and/9781449330880/> - Назва з екрана.

2. Walrath, K. and Ladd, S. What is Dart [Електронний ресурс]. - Режим доступу: https://www.oreilly.com/library/view/what-is-dart/9781449333164/?_gl=1*1266jpm*_ga*NzMyMzcyODkzLjE2OTYyNTk1NTg.*_ga_092EL089CH*MTY5NjI1OTU1Ny4xLjEuMTY5NjI1OTY3NS4xNy4wLjA - Назва з екрана.

3. Dart language support and debugger for Visual Studio Code [Електронний ресурс]. - Режим доступу: <https://marketplace.visualstudio.com/items?itemName=Dart-Code.dart-code> - Назва з екрана.

4. The Java Tutorials [Електронний ресурс]. - Режим доступу: <https://docs.oracle.com/javase/tutorial/java/index.html> - Назва з екрана.

5. What's New in JDK 8 [Електронний ресурс]. - Режим доступу: <http://www.oracle.com/technetwork/java/javase/8-whats-new-2157071.html> - Назва з екрана.

6. Commonly used packages [Електронний ресурс]. - Режим доступу: <https://dart.dev/guides/libraries/useful-libraries> - Назва з екрана.

7. Codelabs & workshops [Електронний ресурс]. - Режим доступу: <https://flutter.dev/docs/codelabs> - Назва з екрана.

8. Introduction to Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/guides/language/language-tour> - Назва з екрана.

9. A tour of the core libraries [Електронний ресурс]. - Режим доступу: <https://dart.dev/guides/libraries/library-tour> - Назва з екрана.

10. Effective Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/guides/language/effective-dart> - Назва з екрана.

11. Офіційна сторінка мови програмування Dart [Електронний ресурс]. - Режим доступу: <https://dart.dev/> - Назва з екрана.

12. Flutter - Beautiful native apps in record time [Електронний ресурс]. - Режим доступу: <https://flutter.dev/> - Назва з екрана.

ВІДПОВІДІ НА КОНТРОЛЬНІ ЗАВДАННЯ

3. Основи мови Dart

34. Неправильно визначені змінні **name4** і **name5**.
35. На консоль буде виведено «45».
36. Програма завершить виконання помилкою, оскільки не можна повторно присвоїти константі **name** нове значення у функції **main**.
37. Неправильно визначені константи **kate** і **alex**.
38. Якщо змінній не присвоєно значення, то за замовчуванням вона має значення **null**.
39. Примітивними типами даних в мові Dart не є: **float**, **char**, **boolean**.
40. Так, можна, якщо оголосити змінну з ключовим словом **dynamic**:

```
1 void main() {  
2     dynamic id = "123"; // тут id має тип String  
3     print(id);  
4     id = 123; // тепер id має тип int  
5     print(id);  
6 }
```

41. Неправильно визначені наступні змінні:

```
1 String id = 123;  
2 bool isValid = 1;
```

В обох випадках є несумісність між типом змінної і типом значення, яке присвоюється. Змінна типу **String** має присвоїти значення в одинарних або подвійних лапках, тут же присвоюється число. Змінній типу **bool** має присвоюватись **true** або **false**, тут же присвоюється число 1.

42. На консоль буде виведено число 1, так як операція $\sim/$ повертає цілочисельний результат ділення першого операнда на другий.

43. На консоль буде виведено такий результат

6
29

Розглянемо по кроках вираз **a + 5 * b++**

1. Спочатку виконується вираз **b++**, який збільшує значення змінної **b** одиницю (тобто тепер воно дорівнює 6), але повертає значення змінної до збільшення (тобто 5).

2. Далі виконується вираз **5 * b++**, тобто **5 * 5 = 25**.

3. Далі виконується вираз **a + 5 * b++**. Фактично це буде **4 + 5 * 5 = 4 + 25 = 29**.

44. На консоль буде виведено такий результат:

30

29

45. Блок **default** в конструкції **switch ... case** необов'язковий.

46. На консоль нічого не буде виведено, оскільки змінній **num** не присвоєно ніякого значення, отже за замовчуванням вона має значення **null**. Відповідно її значення не відповідає жодному з блоків **case**.

47. Ні, оператор **break** в блоці **case** при певних умовах може бути відсутнім.

48. Програма завершиться з помилкою, оскільки змінна **num1** дорівнює **null**, відповідно її значення не можна порівняти зі значенням змінної **num2**.

49. На консоль буде виведено число 5.

50. На консоль буде виведено "**null**".

51. Цикл буде виконуватися 10 разів.

52. Цикл буде виконуватися один раз.

53. Цикл буде виконуватися 10 разів, на консоль будуть виведені всі парні числа від 0 до 8 (0, 2, 4, 6, 8).

4. Функціональне програмування

13. Програма завершиться з помилкою, тому що зовнішня функція намагається звернутися до змінної **message**, яка визначена у вкладеній функції **showMessage**. А для зовнішньої функції змінні вкладеної функції не доступні.

5. Об'єктно-орієнтоване програмування

38. Для спадкування одного класу від іншого застосовується оператор **extends**.

39. Оскільки в мові Dart похідні класи не успадковують конструктори, то у класу **Employee** визначено тільки конструктор за замовчуванням, який не приймає параметрів. Тобто в даному коді конструктор класу **Employee** має 0 параметрів.

40. При виконанні цієї програми вона завершиться з помилкою, відповідно на консолі побачимо тільки опис помилки. Оскільки класи в Dart не успадкують конструктори, то для створення об'єкта **Employee** не можна використовувати виклик такого вигляду як:

```
1 Employee bob = Employee("Bob");
```

41. Наведена програма завершиться з помилкою, оскільки похідні класи при визначенні конструктора повинні обов'язково викликати конструктор базового класу (якщо базовий клас явно визначає конструктори). В цьому прикладі конструктор класу **Employee** не викликає конструктор класу **Person**, відповідно отримаємо помилку.

42. На консоль буде виведений рядок **"Name: Bob"**, так як викликається метод **display**, визначений в базовому класі **Person**.

Приклади та вправи

9.1.

```
1 String stringify(int x, int y) {
2     return '$x $y';
3 }
```

9.2.

```
1 String? name = 'Jane';
2 String? address;
```

9.3.

```
1 String? foo = 'a string';
2 String? bar; // = null
3
4 // Замініть оператор аби 'a string' присвоїти baz.
5 String? baz = foo ?? bar;
6
7 void updateSomeVars() {
8     // Замініть оператор аби 'a string' присвоїти bar.
9     bar ??= 'a string';
10 }
```

9.4.

```
1 // Цей метод має повернути версію `str` з усіма
  великими літерами
2 // або null, якщо `str` є null.
3 String? upperCaseIt(String? str) {
4     return str?.toUpperCase();
5 }
```

9.5.

```
1 // Ініціалізуйте цей list, аби він містив 'a', 'b',  
  and 'c' саме в цьому порядку:  
2 final aListOfStrings = ['a', 'b', 'c'];  
3  
4 // Ініціалізуйте цей set, аби він містив 3, 4, та 5:  
5 final aSetOfInts = {3, 4, 5};  
6  
7 // Ініціалізуйте цей map зі String з int, аби  
  aMapOfStringsToInts['myKey'] повертав 12:  
8 final aMapOfStringsToInts = {'myKey': 12};  
9  
10 // Ініціалізуйте цей порожній List<double>:  
11 final anEmptyListOfDouble = <double>[];  
12  
13 // Ініціалізуйте цей порожній Set<String>:  
14 final anEmptySetOfString = <String>{};  
15  
16 // Ініціалізуйте цей порожній Map із double з int:  
17 final anEmptyMapOfDoublesToInts = <double, int>{};
```

9.6.

```
1 class MyClass {  
2     int value1 = 2;  
3     int value2 = 3;  
4     int value3 = 5;  
5  
6     // Метод, який повертає добуток, зазначених вище  
  значень:  
7     int get product => value1 * value2 * value3;  
8  
9     // Додає 1 до значення value1:  
10    void incrementValue1() => value1++;  
11  
12    // Метод, який повертає рядок із значень  
13    // list, що розділені символом кома (тобто  
  'a,b,c'):  
14    String joinWithCommas(List<String> strings) =>  
  strings.join(',');  
15 }
```

9.7.

```
1  class BigObject {
2      int anInt = 0;
3      String aString = '';
4      List<double> aList = [];
5      bool _done = false;
6
7      void allDone() {
8          _done = true;
9      }
10 }
11
12 BigObject fillBigObject(BigObject obj) {
13     return obj
14         ..anInt = 1
15         ..aString = 'String!'
16         ..aList.add(3)
17         ..allDone();
18 }
```

9.8.

```
1  class InvalidPriceException {}
2
3  class ShoppingCart {
4      List<double> _prices = [];
5
6      double get total => _prices.fold(0, (e, t) => e +
7      t);
8
9      set prices(List<double> value) {
10         if (value.any((p) => p < 0)) {
11             throw InvalidPriceException();
12         }
13
14         _prices = value;
15     }
16 }
```

9.9.

```
1 String joinWithCommas(int a, [int? b, int? c, int?
  d, int? e]) {
2   var total = '$a';
3   if (b != null) total = '$total,$b';
4   if (c != null) total = '$total,$c';
5   if (d != null) total = '$total,$d';
6   if (e != null) total = '$total,$e';
7   return total;
8 }
```

9.10

```
1 class MyDataObject {
2   final int anInt;
3   final String aString;
4   final double aDouble;
5
6   MyDataObject({
7     this.anInt = 1,
8     this.aString = 'Old!',
9     this.aDouble = 2.0,
10  });
11
12  MyDataObject copyWith({int? newInt, String?
  newString, double? newDouble}) {
13    return MyDataObject(
14      anInt: newInt ?? this.anInt,
15      aString: newString ?? this.aString,
16      aDouble: newDouble ?? this.aDouble,
17    );
18  }
19 }
```

9.11

```
1  typedef VoidFunction = void Function();
2
3  class ExceptionWithMessage {
4      final String message;
5      const ExceptionWithMessage(this.message);
6  }
7
8  abstract class Logger {
9      void logException(Type t, [String? msg]);
10     void doneLogging();
11 }
12
13 void tryFunction(VoidFunction untrustworthy, Logger
    logger) {
14     try {
15         untrustworthy();
16     } on ExceptionWithMessage catch (e) {
17         logger.logException(e.runtimeType, e.message);
18     } on Exception {
19         logger.logException(Exception);
20     } finally {
21         logger.doneLogging();
22     }
23 }
```

9.12

```
1  class MyClass {
2      final int anInt;
3      final String aString;
4      final double aDouble;
5
6      MyClass(this.anInt, this.aString, this.aDouble);
7  }
```

9.13

```
1  class FirstTwoLetters {
2      final String letterOne;
3      final String letterTwo;
4
5      FirstTwoLetters(String word)
6          : assert(word.length >= 2),
7            letterOne = word[0],
8            letterTwo = word[1];
9  }
```

9.14

```
1  class Color {
2      int red;
3      int green;
4      int blue;
5
6      Color(this.red, this.green, this.blue);
7
8      Color.black()
9          : red = 0,
10           green = 0,
11           blue = 0;
12 }
```


9.15

```
1  class IntegerHolder {
2      IntegerHolder();
3
4      factory IntegerHolder.fromList(List<int> list) {
5          if (list.length == 1) {
6              return IntegerSingle(list[0]);
7          } else if (list.length == 2) {
8              return IntegerDouble(list[0], list[1]);
9          } else if (list.length == 3) {
10             return IntegerTriple(list[0], list[1], list[2]);
11          } else {
12              throw Error();
13          }
14      }
15 }
16
17 class IntegerSingle extends IntegerHolder {
18     final int a;
19     IntegerSingle(this.a);
20 }
21
22 class IntegerDouble extends IntegerHolder {
23     final int a;
24     final int b;
25     IntegerDouble(this.a, this.b);
26 }
27
28 class IntegerTriple extends IntegerHolder {
29     final int a;
30     final int b;
31     final int c;
32     IntegerTriple(this.a, this.b, this.c);
33 }
```

9.16

```
1  class Color {
2      int red;
3      int green;
4      int blue;
5
6      Color(this.red, this.green, this.blue);
7
8      Color.black() : this(0, 0, 0);
9  }
```

9.17

```
1  class Recipe {
2      final List<String> ingredients;
3      final int calories;
4      final double milligramsOfSodium;
5
6      const Recipe(this.ingredients, this.calories,
7      this.milligramsOfSodium);
8  }
```

ПРЕДМЕТНИЙ ПОКАЖЧИК ТЕРМІНІВ ТА СИНТАКСИЧНИХ КОНСТРУКЦІЙ

			int 37	
@override	@	103		L
			List	135
	A			
abstract		105		M
async		172	main	32
await		172	Map	143
			microTask queue	161
	B			
bool		37		N
break		60	null	40
	C			P
const		36	print	32
	D			R
DartPad		9	Runes	39
do 59				
double		38		S
dynamic		35	Set	140
			statement	32
	E		static	112
enum		128	String	38
event loop		161	super	110
event queue		161	switch/case	55
Exception		153	Symbol	40
extends		101		
				T
	F		this	92
factory		121	throw	156
final		36, 98	try ... on ... catch ... finally	149
for58				
Future		162		V
			var	34
	G		void	32
Generics		116		
				W
	I		whenComplete	171
if/else		53	while	60

with		110	Т	
	А		тернарна операція	57
абстрактний клас		105	У	
абстрактний метод		107	успадковування	101
анонімна функція		78		
асинхронна операція		174	Ф	
асинхронність		161	фабрика	17
	В		фабричний конструктор	121
винятки		149		
вкладена функція		79		
	З			
замикання		80		
	І			
іменований конструктор		91		
ініціалізатор		94		
	К			
каскадна нотація		96		
колекція		143		
константи		98		
константний конструктор		100		
конструктор		86		
	М			
міксин		110		
	Н			
набір		140		
	О			
об'єкт		85		
	П			
перерахування		128		
	С			
список		135		
статична константа		113		
статичний метод		114		
статичні поля		112		

ДОДАТОК А. Приклади кросплатформної розробки мобільних застосунків

Інструментальні засоби розпізнавання поведінки людини з використанням сенсорів мобільного телефону

Описаний програмний застосунок розроблено в межах виконання дипломної роботи на здобуття ступеня бакалавра за спеціальністю 121 Інженерія програмного забезпечення¹

Метою розробки є створення мобільного застосунку для виявлення та ідентифікації нетипової поведінки при керуванні транспортним засобом. Призначення застосунку — збереження даних поїздки, розпізнавання та ідентифікація нетипової поведінки. На рисунку А.1 зображено інтерфейс розробленого мобільного застосунку.

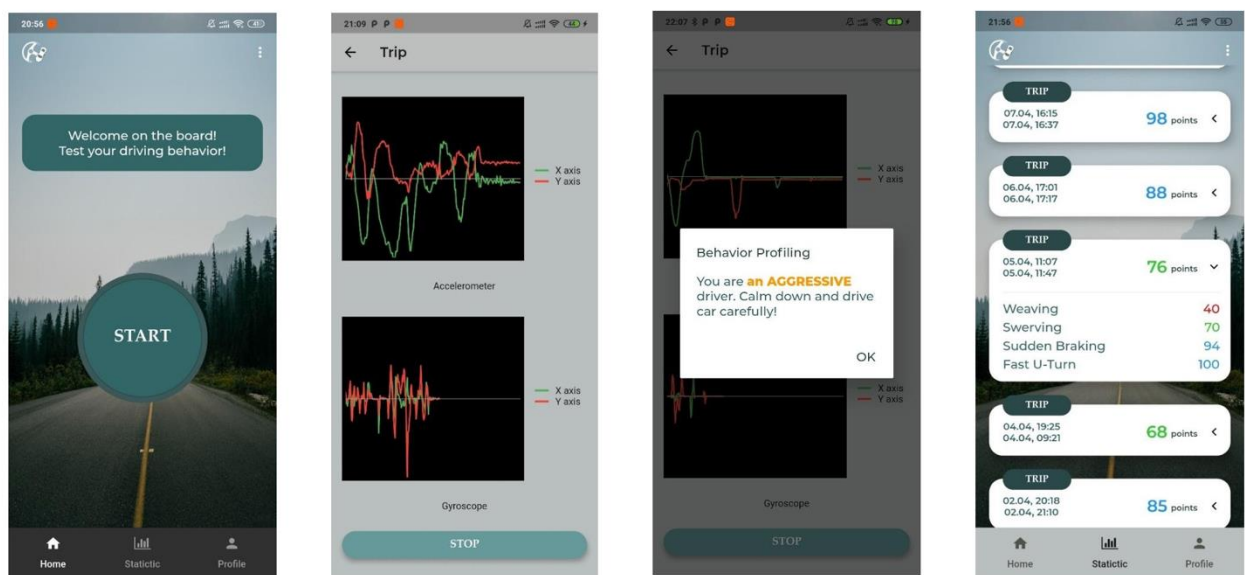


Рисунок А.1. Інтерфейс мобільного застосунку розпізнавання поведінки людини з використанням сенсорів мобільного телефону

Мобільний застосунок має функцію розпізнавання нетипової поведінки при керуванні транспортним засобом на основі даних від сенсорів мобільного телефону. Система складається з двох модулів: Моделювання поведінки водія та Аналіз поведінки водія. Архітектура мобільного застосунку виглядає так (рисунок А.2).

¹ Мовлян, Т. В. Інструментальні засоби розпізнавання поведінки людини з використанням сенсорів мобільного телефону : дипломна робота ... бакалавра : 121 Інженерія програмного забезпечення / Мовлян Тетяна Вікторівна. – Київ. 2020. – 88 с. <https://ela.kpi.ua/handle/123456789/36660>

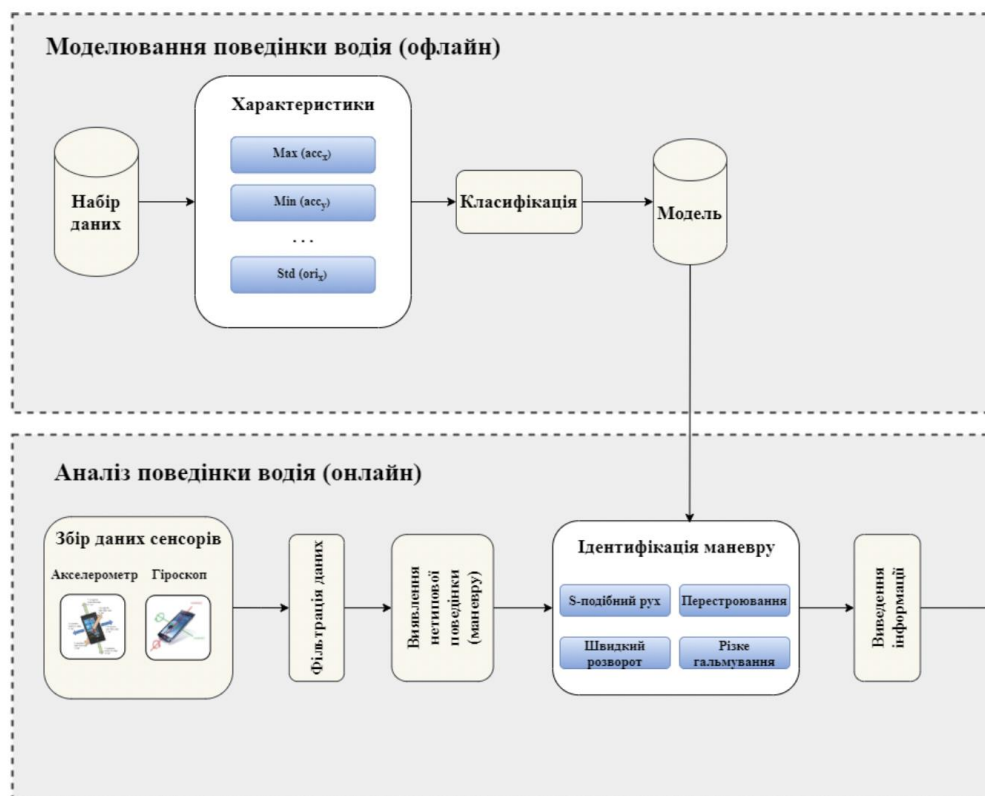


Рисунок А.2. Архітектура мобільного застосунку

Діаграма розгортання на Андроїд-пристрої розробленого мобільного застосунку зображена на рисунку А.3.

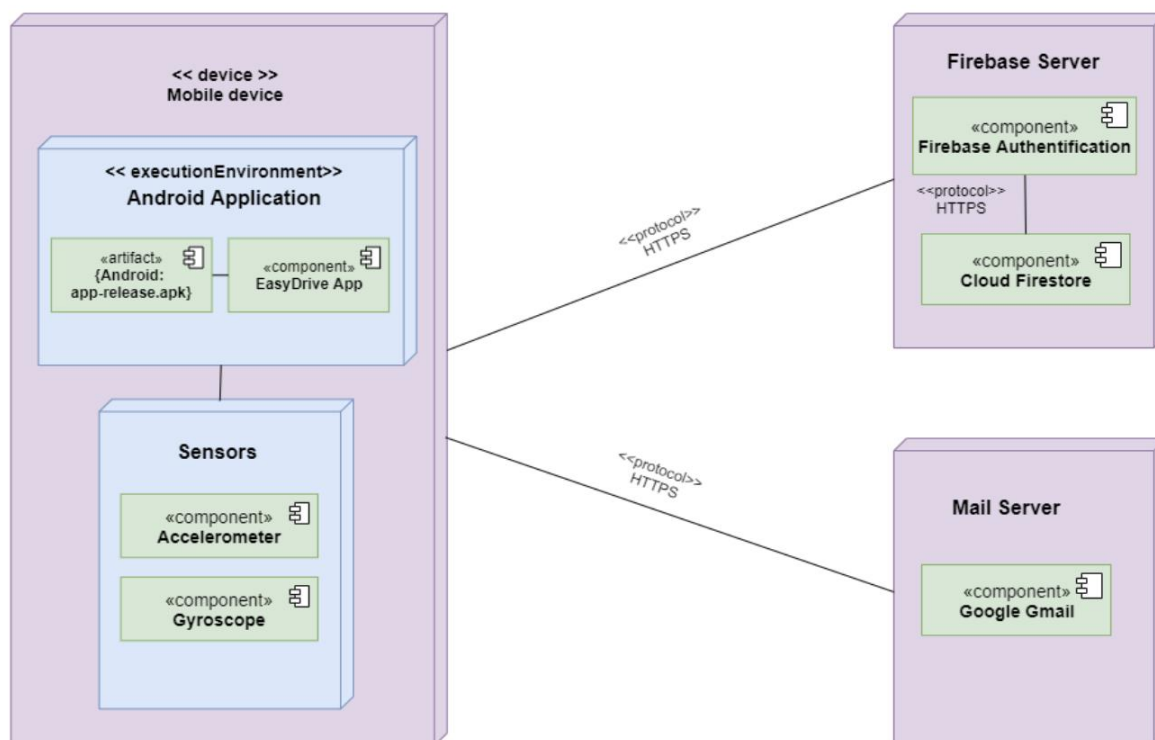


Рисунок А.3. Діаграма розгортання

Далі наведено фрагмент коду програмного застосунку, який реалізує функцію аналізу поїздки.

```
// Функція аналізу поїздки
Future<void> _analysData() async {
    List<dynamic> _abnormalClassifications = [];
    List<Map<String, dynamic>> _smoothedData = <Map<String, dynamic>>[];
    _smoothedData = smoothingSMAUnknown(_data);
    bool startAbnormal = false;
    int abnormalBeginIndex = 0;
    int abnormal = 0;
    var _dataModels =
        json.decode(await
rootBundle.loadString('models/behaviorModels.json'));
    for (var i = 1; i < _smoothedData.length; i++) {
        if (stDeviation(_smoothedData[i]["acc.x"], _smoothedData[i -
1]["acc.x"],
            mean(_smoothedData[i]["acc.x"], _smoothedData[i -
1]["acc.x"]))) >
            0.3) {
            if (startAbnormal != true && i != _smoothedData.length - 1) {
                startAbnormal = true;
                abnormalBeginIndex = i - 1;
            }
            if (i == _smoothedData.length - 1 && startAbnormal == true) {
                startAbnormal = false;
                abnormal += 1;
                _abnormalClassifications.add(classifyAbnormal(
                    _smoothedData, _dataModels, abnormalBeginIndex, i)); }
        } else if (startAbnormal == true && i == _smoothedData.length - 1) {
            startAbnormal = false;
            abnormal += 1;
            _abnormalClassifications.add(classifyAbnormal(
                _smoothedData, _dataModels, abnormalBeginIndex, i));
        } else if (startAbnormal == true) {
            if (stDeviation(
                _smoothedData[i]["acc.x"],
                _smoothedData[i + 1]["acc.x"],
                mean(
                    _smoothedData[i]["acc.x"], _smoothedData[i +
1]["acc.x"]))) >
                    0.23) {
            } else if (i != _smoothedData.length - 2 &&
                stDeviation(
                    _smoothedData[i + 1]["acc.x"],
                    _smoothedData[i + 2]["acc.x"],
                    mean(_smoothedData[i + 1]["acc.x"],
                        _smoothedData[i + 2]["acc.x"]))) >
                    0.23) {
            } else {
                startAbnormal = false;
                abnormal += 1;
                _abnormalClassifications.add(classifyAbnormal(
                    _smoothedData, _dataModels, abnormalBeginIndex, i));
            }}
        if (_smoothedData[i]['time'] % 60 == 0 || i == _smoothedData.length -
1) {
            allTripAbnormal.add(abnormal);
            abnormal = 0;
        }
    }
    if (_abnormalClassifications.length > 0) {
        for (var ab in _abnormalClassifications) {
            ab == "Weaving"
                ? _weaving++
                : ab == "Swerving"
                    ? _swerving++
                    : ab == "Fast U-turn" ? _fastUTurn++ : _suddenBraking++;
        }
    }
}
```

Інструментальні засоби розпізнавання та обробки даних двовимірного зображення

Описаний програмний застосунок розроблено в межах виконання дипломної роботи на здобуття ступеня бакалавра за спеціальністю 121 Інженерія програмного забезпечення¹

Метою розробки є створення зручного програмного забезпечення мобільного телефону для ефективного зчитування та обробки пошкодженого двовимірного зображення, а саме QR-коду.

На рисунку А.4 зображено інтерфейс розробленого мобільного застосунку.

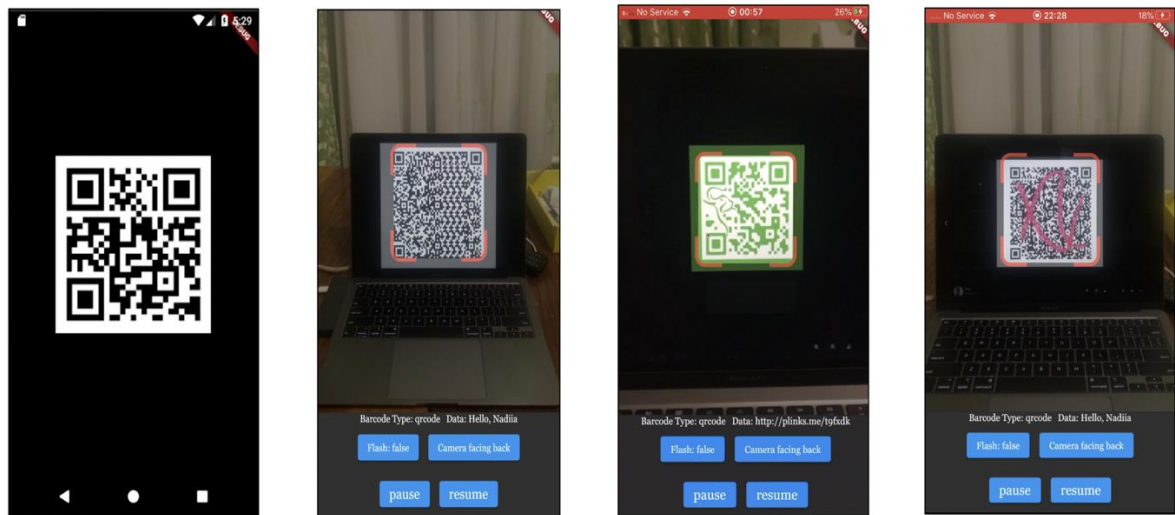


Рисунок А.4. Інтерфейс мобільного застосунку розпізнавання зображень
Діаграму користувацьких потоків зображено на рисунку А.5.

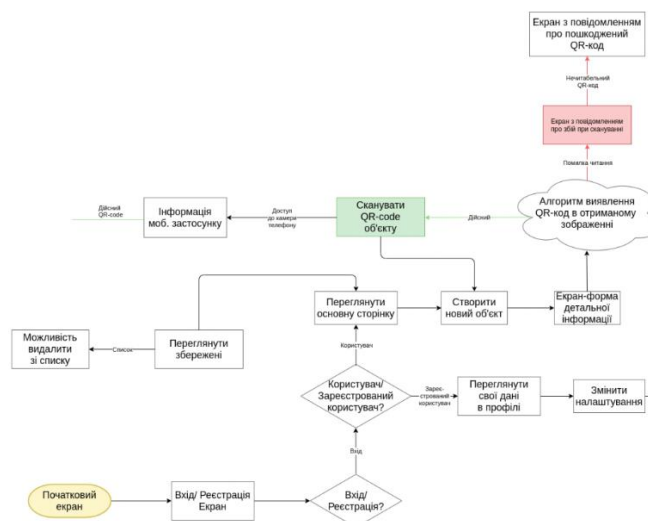


Рисунок А.5. Діаграма користувацьких потоків (user flow)

¹ Скиба, Н. В. Інструментальні засоби розпізнавання та обробки даних двовимірного зображення : дипломна робота ... бакалавра : 121 Інженерія програмного забезпечення / Скиба Надія Володимирівна. – Київ, 2021. – 86 с. <https://ela.kpi.ua/handle/123456789/44243>

Далі наведено фрагмент коду програмного застосунку, який реалізує функцію розпізнавання зображення QR-коду

```
QRScanner.dart
class QRViewExample extends StatefulWidget {
  const QRViewExample({
    Key key,
  }) : super(key: key);
  @override
  State<StatefulWidget> createState() => _QRViewExampleState();
}
class _QRViewExampleState extends State<QRViewExample> {
  Barcode result;
  QRViewController controller;
  final GlobalKey qrKey = GlobalKey(debugLabel: 'QR');
  // In order to get hot reload to work we need to pause the camera if the
platform
  // is android, or resume the camera if the platform is iOS.
  @override
  void reassemble() {
    super.reassemble();
    if (Platform.isAndroid) {
      controller.pauseCamera();
    }
    controller.resumeCamera();
  }
  @override
  Widget build(BuildContext context) {
    return Scaffold(
      body: Column(
        children: <Widget>[
          Expanded(flex: 4, child: buildQrView(context)),
          Expanded(
            flex: 1,
            child: FittedBox(
              fit: BoxFit.contain,
              child: Column(
                mainAxisAlignment: MainAxisAlignment.spaceEvenly,
                children: <Widget>[
71
                  if (result != null)
                    Text(
                      'Barcode Type: ${describeEnum(result.format)} Data: ${result.code}')
                    else
                      Text('Scan a code'),
                  Row(
                    mainAxisAlignment: MainAxisAlignment.center,
                    crossAxisAlignment: CrossAxisAlignment.center,
                    children: <Widget>[
                      Container(
                        margin: EdgeInsets.all(8),
                        child: ElevatedButton(
                          onPressed: () async {
                            await controller?.toggleFlash();
                            setState(() {});
                          },
                        child: FutureBuilder(
                          future: controller?.getFlashStatus(),
                          builder: (context, snapshot) {
                            return Text('Flash: ${snapshot.data}');
                          },
                        ),
                      ),
                      Container(
                        margin: EdgeInsets.all(8),
                        child: ElevatedButton(
                          onPressed: () async {
                            await controller?.flipCamera();
                            setState(() {});
                          },
                        child: FutureBuilder(
                          future: controller?.getCameraInfo(),
```


Інструментальні засоби розширеної навігації

Описаний програмний застосунок розроблено в межах виконання дипломної роботи на здобуття ступеня бакалавра за спеціальністю 121 Інженерія програмного забезпечення¹.

Метою даної роботи є створення мобільного застосунку - GPS-навігатора з інтеграцією технологій машинного навчання та доповненої реальності. Призначення застосунку - побудова маршруту, вказаного користувачем, навігація за допомогою віртуальних об'єктів доповненої реальності, розпізнавання учасників дорожнього трафіку.

На рисунку А.6 зображено інтерфейс розробленого мобільного застосунку.

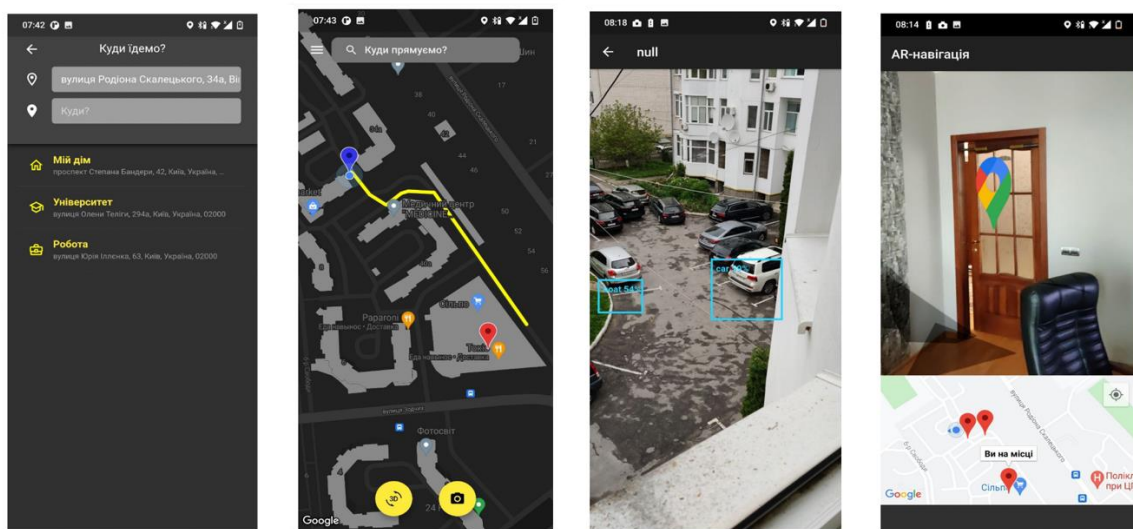


Рисунок А.6. Інтерфейс мобільного застосунку розширеної навігації

Діаграма розгортання розробленого мобільного застосунку зображена на рисунку А.7. Система містить чотири основних вузли: смартфон, сервер бази даних (Firebase RealTime Database), 2 хмарних API та їхні основні компоненти. Операційна система Android - середовище виконання мобільного додатку "ARnavigator". Для використання та встановлення необхідно мати файл "armeabi-v7a-release.apk". Для економії пам'яті смартфона використовуються хмарні API - "Places API" та "Directions API". Places API використовується для того, щоб повернути результати пошуку географічного місця за запитом від користувача, а Directions API - для того, щоб отримати всі проміжні точки між початком та кінцем маршруту користувача.

¹ Сторожук, К. С. Інструментальні засоби розширеної навігації : дипломна робота ... бакалавра : 121 Інженерія програмного забезпечення / Сторожук Костянтин Сергійович. – Київ, 2021. – 72 с. <https://ela.kpi.ua/handle/123456789/44205>

З'єднання з базою даних та хмарними API відбувається за допомогою протоколу HTTPS — комбінації протоколів SSL/TLS і HTTP. Він забезпечує зашифровану і безпечну ідентифікацію мережевого сервера та з'єднання між додатком та сервером.

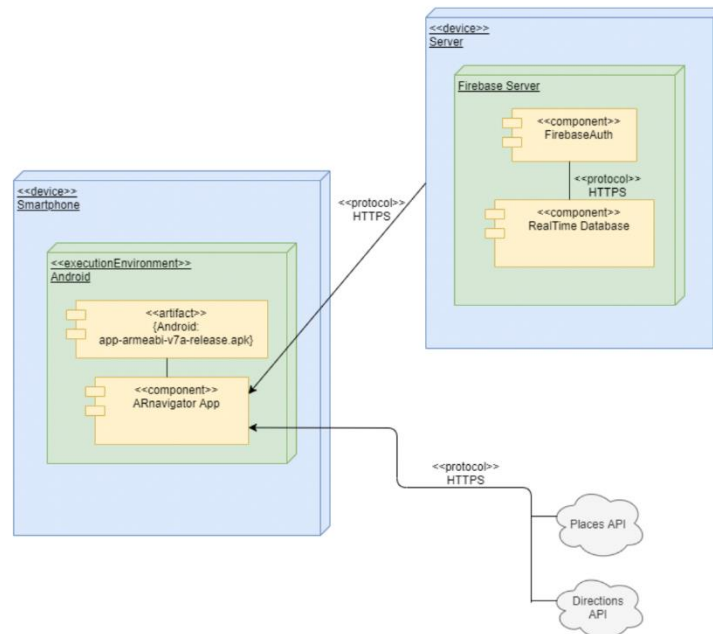


Рисунок А.7. Діаграма розгортання

Далі наведено фрагмент коду програмного застосунку, який реалізує функцію роботи з камерою мобільного пристрою.

```
camera.dart
import 'package:flutter/material.dart';
import 'package:camera/camera.dart';
import 'package:tflite/tflite.dart';
import 'dart:math' as math;
typedef void Callback(List<dynamic> list, int h, int w);
class CameraFeed extends StatefulWidget {
  final List<CameraDescription> cameras;
  final Callback setRecognitions;
  // The cameraFeed Class takes the cameras list and the setRecognitions
  // function as argument
  CameraFeed(this.cameras, this.setRecognitions);
  @override
  _CameraFeedState createState() => new _CameraFeedState();
  class _CameraFeedState extends State<CameraFeed> {
    CameraController controller;
    bool isDetecting = false;
    @override
    void initState() {
      super.initState();
      print(widget.cameras);
      if (widget.cameras == null || widget.cameras.length < 1) {
        print('No Cameras Found.');
```

```

setState(() {});
controller.startImageStream((CameraImage img) {
  if (!isDetecting) {
    isDetecting = true;
    Tflite.detectObjectOnFrame(
      bytesList: img.planes.map((plane) {return plane.bytes;}).toList(),
      model: "SSDMobileNet",
      imageHeight: img.height,
      imageWidth: img.width,
      imageMean: 127.5,
      imageStd: 127.5,
      numResultsPerClass: 1,
      threshold: 0.4,
    ).then((recognitions) {
      widget.setRecognitions(recognitions, img.height, img.width);
      isDetecting = false;});});});});});
@override
void dispose() {
  controller?.dispose();
  super.dispose();}
@override
Widget build(BuildContext context) {
  if (controller == null || !controller.value.isInitialized) {
    return Container();}
  var tmp = MediaQuery.of(context).size;
  var screenH = math.max(tmp.height, tmp.width);
  var screenW = math.min(tmp.height, tmp.width);
  tmp = controller.value.previewSize;
  var previewH = math.max(tmp.height, tmp.width);
  var previewW = math.min(tmp.height, tmp.width);
  var screenRatio = screenH / screenW;
  var previewRatio = previewH / previewW;
  return OverflowBox(
    maxHeight:
      screenRatio > previewRatio ? screenH : screenW / previewW * previewH,
    maxWidth:
      screenRatio > previewRatio ? screenH / previewH * previewW : screenW,
    child: CameraPreview(controller),
  );}}

```